Tile IR Specification

Dec 2025

Contents

1 Introduction

This document describes **Tile IR** , a portable, low-level tile virtual machine and instruction set.

Unlike PTX, which models the GPU as a data-parallel single instruction multiple thread (SIMT) processor, **Tile IR** models the GPU as a tile-based processor.

In **Tile IR** , each logical thread (tile block) computes over partial fragments (tiles) of multi-dimensional arrays (tensors).

1.1 Scalable Data Parallel Computing with Tiles on GPU

The CUDA platform provides portability through CUDA C++ and PTX, allowing code written for older GPU generations to run efficiently on newer ones. These mechanisms enable developers to write software for one family of GPUs and continue to deploy it on future generations.

The rapid evolution of hardware features, such as tensor cores, has increased programming-model complexity, requiring greater expertise and hardware understanding to write portable and performant code.

Since Volta, each new GPU generation introduces powerful new hardware features, giving rise to new programming paradigms.

To address these challenges, **Tile IR** introduces a virtual instruction set that enables native programming of the hardware in terms of tiles, allowing developers to write higher-level code that can be efficiently executed across multiple

GPUs with minimal changes.

While PTX ensures portability for SIMT programs, **Tile IR** extends the CUDA platform with native support for tile-based programs, allowing developers to focus on partitioning their data-parallel programs into tiles and tile blocks, letting **Tile IR** handle the mapping onto hardware resources such as threads, the memory hierarchy, and tensor cores.

By raising the level of abstraction, **Tile IR** enables users to build new higher-level hardware-specific DSLs, compilers, and frameworks for NVIDIA hardware.

1.2 Goals and Scope

Tile IR aims to provide a performance-portable programming model and instruction set for tile-based parallel programming.

Core goals:

- Introduce a data-parallel tile programming abstraction that aligns with programmer intent and acts as a code generation target for DSLs and compilers.
- Abstract tensor-cores and their programming model to enable hardware innovation without disrupting the NVIDIA ecosystem or customers' existing software investments.
- Abstract low-level, architecture-specific details such as CUDA threads and memory hierarchy to ensure programs can be efficiently recompiled for different GPUs.
- Minimize abstraction overhead. While portability has a cost, **Tile IR** aims to keep this cost low, introducing only a modest performance overhead.
- Provide user controls and optimization hints to recover peak performance when needed.
- Provide seamless interoperability with existing CUDA programming models like CUDA C++ and PTX.

Tile IR achieves these core goals through the following key components, in order of importance:

1. A versioned specification of the **Tile IR** abstract machine, including a versioned and portable bytecode representation.
2. An optimizing compiler for **Tile IR** programs, available as part of the CUDA driver and as a standalone tool in the CUDA toolkit, similar to PTX.
3. An MLIR dialect that existing compilers can use to target **Tile IR** as a backend compiler target.

1.3 Document Structure

4. Introduction this section.
5. Programming Model describes the programming model of **Tile IR** .
6. Syntax defines the syntax of **Tile IR** programs used throughout the specification.
7. Binary Format describes the **Tile IR** byte code and its binary format.
8. Type System defines the type system of **Tile IR** .
9. Semantics describes the semi-formal semantics of **Tile IR** .
10. Memory Model describes the **Tile IR** memory model.
11. Operations a listing of the full set of **Tile IR** operations.
12. Debug Info describes the debug information format for **Tile IR** programs.
13. Stability describes the stability guarantees of the **Tile IR** .
14. Appendix contains relevant reference materials, including full programs and references.

2 Programming Model

Tile IR extends CUDA's low-level programming model with new abstractions that differ from what has previously existed in CUDA C++ or PTX.

This section introduces the programming model of **Tile IR** and familiarizes the reader with its core concepts and abstractions. We do this by working through a series of real programs, building up to a dynamic, high-performance implementation of GEMM that makes use of the all major features of **Tile IR** .

Tile IR has an expressive tile-based programming model. We introduce users to the various ways to represent tensor computation by progressively adapting the example programs to take better advantage of **Tile IR** features which help simplify programs and enable the compiler to provide performance portability. We start by first introducing concepts that readers may be familiar with from prior art.

2.1 Tile Kernels

Tile IR programs are referred to as *tile kernels* , which like CUDA C++ or PTX, are functions which run as N copies in parallel when invoked. The primary difference is the basic unit of execution: a *tile-block* , which expresses the computation performed by a single logical tile thread operating over a multi-dimensional tile of data.

During execution, each tile kernel is referred to as a tile kernel instance.

Below is a simple **Tile IR** kernel which prints "Hello World!".

```
cuda_tile.module @hello_world_module {
    entry @hello_world_kernel() {
        print "Hello World!\n"
    }
}
```

For those familiar with CUDA threads, it is important to note that **Tile IR** 's threads are different. Before we go any further into formalisms, it is essential we highlight those differences.

Tile kernels are the entry point of the program, executing as parallel instances of tile blocks.

2.1.1 What's different about tile programming?

Tile IR is an extension to the CUDA programming model that enables first class support for tile programming.

Tiled kernels express programs as a grid of logical tile threads that operate over tiles. The mapping of both the grid and individual tile threads to the underlying hardware's threads is abstracted away from the programming model and is handled by the compiler.

The SIMT programming model of NVIDIA's streaming multiprocessor (SM) is one in which threads operate over

(relatively) small pieces of data and the user is responsible for dividing and scheduling the threads into the appropriate blocks to compute over the input data in an efficient manner. This model gives flexibility to programmers on how to map threads to data, or vice-versa. SIMT is the programming model exposed by

CUDA and PTX and has served NVIDIA GPUs well since its introduction in 2006.

The rise in importance of deep learning has both introduced a greater regularity to user workloads and an ever increasing need to deliver performance for these workloads. As discussed in the Introduction , this has led to new specialized hardware in the form of tensor cores.

Tensor cores introduce a new dimension to the SIMT programming model. Now, SM threads must cooperate with the tensor cores in order to reach peak performance. With each new generation of hardware the interplay between these two pieces of silicon has unlocked amazing new performance but with increasing programming complexity.

Tile IR has been built to aid in the implementation of high-performance algorithms that take full advantage of the underlying hardware's capabilities while mitigating the increase in programming complexity.

By abstracting thread-to-data mapping, **Tile IR** simplifies the use of specialized hardware like Tensor Cores compared to traditional SIMT models.

2.1.2 Kernel I/O

To illustrate the design of **Tile IR** we will move from our simple hello world kernel to one which implements 1-d tensor (i.e., vector) addition with a fixed block size `128` .

All examples presented in this section can be found in the Programming Model Example Programs .

Tile kernels accept inputs and outputs as parameters; this is the only mechanism for consuming and producing data, so we start by defining the kernel parameters.

```
entry @vector_block_add_128x1_kernel(
    %a_ptr_base_scalar : !cuda_tile.tile<ptr<f32>>,
    %b_ptr_base_scalar : !cuda_tile.tile<ptr<f32>>,
    %c_ptr_base_scalar : !cuda_tile.tile<ptr<f32>>)
```

The above code fragment defines a kernel named `vector_block_add_128x1_kernel` which takes three arguments, representing the two input buffers `a` and `b` , and the output buffer `c` . The types of all the arguments are scalar pointers, which are represented as zero dimensional tensors containing a single pointer.

All values in **Tile IR** are either tensors, or tensor views (see Tensor View). A tensor is an n-dimensional rectangular array, described by its rank (number of dimensions), the shape (extent along each dimension), and its primitive element type.

Tensors may have a rank of 0 or higher. Rank-0 tensors are scalars. The rank, dimensions, and element type are all part of the tensor type and are statically known. Tensor types are assigned to values which represent a logical view of a multidimensional array contained in global memory. Global memory is always accessed via tensors, and thus pointer arguments always point to CUDA device allocations in global device memory. Tile kernels do not have return values and thus omit a return type annotation (note that tile functions may have a return type, and will be discussed later).

An astute reader might now be wondering why we gave the inputs and outputs scalar pointer types instead of tensors types with statically known rank, shape, and stride. A common pattern in the **Tile IR** programming model is for kernels to take unstructured base pointers as parameters which can then be used to construct the required tensor.

This flexibility gives rise to multiple ways to use pointers depending on the desired program behavior, but we will first focus on the most flexible representation by converting our base pointer into a tensor of arbitrary pointers.

A tensor of arbitrary pointers is a flexible abstraction which allows you to perform scatter/gather style loads from a set of addresses all at once, and treat a set of addresses as a logical tile.

We must take a few steps to convert a base pointer `%a_ptr_base_scalar` into a tensor of pointers representing the `128x1` tile we want to compute on which contains addresses from `(base + 0, ..., base + 127)`

We start by creating an offset tensor which represents the inclusive `(0, 127)` interval.

We use cuda_tile.iota which constructs a range tensor that counts from `0` to `n - 1` forming an `n` sized vector.

```
    %offset = iota : tile<128xi32>
```

We then reshape from a scalar `ptr<f32>` to a 1-d tensor `1xptr<f32>` so we have the correct rank.

```
%a_ptr_base_tensor = reshape %a_ptr_base_scalar :
    tile<ptr<f32>> -> tile<1xptr<f32>>
```

We then broadcast the pointer so we have a 1-d tensor of `(base, ..., base)` containing 128 elements.

```
%a_ptr = broadcast %a_ptr_base_tensor : tile<1xptr<f32>> -> tile<128xptr<f32>>
```

Add the offset tensor to the tensor of pointers to obtain a `tile<128xptr<f32>>` that contains pointers of `(base + 0, ..., base + 127)` as its values. Now we have a tensor of pointers which represents the tile that we would like to compute on. We perform the same set of steps for `a` , `b` , and `c` .

```
%a_tensor = offset %a_ptr, %offset :
    tile<128xptr<f32>>, tile<128xi32> -> tile<128xptr<f32>>
```

Finally, we load both operands, perform the addition, and store to the output.

```
%a_val, %token_a = load_ptr_tko weak %a_tensor : tile<128xptr<f32>> -> tile<128
    xf32>, token
%b_val, %token_b = load_ptr_tko weak %b_tensor : tile<128xptr<f32>> -> tile<128
    xf32>, token
%c_val = addf %a_val, %b_val rounding<nearest_even> : tile<128xf32>
store_ptr_tko weak %c_tensor, %c_val : tile<128xptr<f32>>, tile<128xf32> ->
    token
```

We now have a complete kernel for a single tile-block that performs addition over 128 element vectors. As you can see this code is written from a single thread of control, but its level of parallelism will be determined by the compiler.

Kernels communicate with global memory via pointer arguments, which can be transformed into tensors using shape and stride manipulation for flexible data access.

2.2 Tile Grid

So far we have only examined kernels which are written for a single tile block. **Tile IR** allows tile blocks to be grouped into a **tile grid** , similar to CUDA C++, enabling users to launch sets of tile blocks that execute in parallel. Tile kernels, as with PTX, are implicitly parameterized over the tile block coordinates (which can be queried via cuda_tile.get_tile_block_id) and can be 1-d, 2-d, or 3-d.

When a tile kernel is launched the user specifies the grid size, which determines the number of tile blocks launched. The number of tile blocks launched is equal to the size of the grid. For example if we launch our previous example with a `(1, 1, 2)` grid, we will run an identical computation twice.

We can now look at an **improved** hello world program which shows off querying the grid size and coordinates.

```
cuda_tile.module @hello_world_module {
    // TileIR kernel function
    entry @hello_world_kernel() {
        // Step 1. Get the tile block ID
        %block_x_index, %block_y_index, %block_z_index = cuda_tile.
            get_tile_block_id : tile<i32>
        // Step 2. Get the tile block dimensions
        %block_dim_x, %block_dim_y, %block_dim_z = cuda_tile.get_num_tile_blocks
             : tile<i32>
        // Step 3. Print the tile block ID and dimensions. Each tile executes
            the
        // following print statement and prints a single line.
        cuda_tile.print "Hello, I am tile <%, %, %> in a kernel with <%, %, %>
            tiles.\n",
            %block_x_index, %block_y_index, %block_z_index, %block_dim_x, %
                block_dim_y, %block_dim_z
            : tile<i32>, tile<i32>, tile<i32>,
              tile<i32>, tile<i32>, tile<i32>
        }
}
```

Each tile kernel can be launched with a 1-d, 2-d, or 3-d grid. Each tile block can query its position in the grid using cuda_tile.get_tile_block_id and query the first, second, or third dimension index by varying the argument.

Tile kernels can also observe the grid dimensions in a similar way using cuda_tile.get_num_tile_blocks .

If we use a grid that is `(1, 1, 2)` we will see two prints:

```
1       "Hello Tile-Grid. I am tile <1, 1, 1> in a kernel with <1, 1, 2> tiles."
2       "Hello Tile-Grid. I am tile <1, 1, 2> in a kernel with <1, 1, 2> tiles."
```

The tile grid organizes parallel execution of tile blocks, allowing kernels to scale across the problem size by querying their coordinates within the grid.

2.2.1 Implementing GEMM

Now that we understand the basics concepts of the **Tile IR** programming model, we will introduce how to compute a 2-d GEMM for a single block, and then generalize it step by step to a full GEMM by utilizing the tile grid and introducing control flow and manual tiling.

This progression demonstrates how to compose basic tile operations into a complex, high-performance parallel algorithm.

2.2.2 GEMM with a Single Block

Let's first start by naturally moving from a single static block vector addition to a single static square block matrix multiplication.

This example proceeds much as before:

```
entry @gemm_block_64x64_kernel(
    %a_ptr_base_scalar: !cuda_tile.tile<!cuda_tile.ptr<f32>>,
    %b_ptr_base_scalar: !cuda_tile.tile<!cuda_tile.ptr<f32>>,
    %c_ptr_base_scalar: !cuda_tile.tile<!cuda_tile.ptr<f32>>
```

We start with a set of scalar pointers. To simplify this example, we assume that each of these pointers point to allocations which are at least 64 elements in length with stride equal to 1.

Then much like before we declare a square offset tensor.

```
%offset_flat = iota : tile<4096xi32>
%offset = reshape %offset_flat :
    tile<4096xi32> -> tile<64x64xi32>
```

We declare pointers to the underlying tiles the same way as before for A, B, C.

```
%a_ptr_base_tensor = reshape %a_ptr_base_scalar :
    tile<ptr<f32>> -> tile<1x1xptr<f32>>
%a_ptr = broadcast %a_ptr_base_tensor : tile<1x1xptr<f32>> -> tile<64x64xptr
    <f32>>
%a_tensor = offset %a_ptr, %offset :
    tile<64x64xptr<f32>>, tile<64x64xi32> -> tile<64x64xptr<f32>>
```

The only difference here is that we are building square tiles in 2D instead of 1D.

We then load the input arguments, compute the MMA operation (see cuda_tile.mmaf for floating-point and cuda_tile.mmai for integers) to compute the output tile, and then store it.

```
// Load a single 64x64 matrix from the tile.
%A_block, %token_a = load_ptr_tko weak %a_tensor :
    tile<64x64xptr<f32>> -> tile<64x64xf32>, token
// Load a single 64x64 matrix from the tile.
%B_block, %token_b = load_ptr_tko weak %b_tensor :
    tile<64x64xptr<f32>> -> tile<64x64xf32>, token
%C_block = mmaf %A_block, %B_block, %init_accum: tile<64x64xf32>, tile<64
    x64xf32>, tile<64x64xf32>
store_ptr_tko weak %c_tensor, %C_block :
    tile<64x64xptr<f32>>, tile<64x64xf32> -> token
```

If you are experienced in implementing matrix multiplication, you can see that this works well for a single `64x64` square multiplication, but what happens if we want to run the tile kernel over a large input problem size in parallel?

Basic matrix multiplication is achieved by constructing 2D tiles from pointers and performing matrix-multiply-accumulate (MMA)

operations within a single block.

2.2.3 GEMM Block by Block

Let us look at how to generalize this to work for a large matrix size – say `4096x4096` , where each tile block will compute a single output tile of the final matrix. Note that we are still working with the assumption that our problem shape is static and we will return to handling dynamically shaped inputs later on this section.

The kernel declaration looks identical to what we have been thus far taking scalar pointers to our inputs and outputs as arguments.

```
entry @gemm_square_4096_tile_64x64_kernel(
    %a_ptr_base_scalar: tile<ptr<f32>>,
    %b_ptr_base_scalar: tile<ptr<f32>>,
    %c_ptr_base_scalar: tile<ptr<f32>>
```

In order to do full matrix multiplication over a large matrix we need to split the problem across multiple tile blocks using the tile grid. We will use a 2-d grid and as such each thread will need to first read its 2-d block coordinates.

```
// Read Tile block id's.
%block_x_index, %block_y_index, %block_z_index = get_tile_block_id : tile<
    i32>
```

The block coordinates determine which tile of the output matrix we will be computing on this tile block. The kernel is structured as a reduction over the K dimension of the matrix, producing individual but complete output tiles. It loop over the reduction dimension computing a matrix-multiply-accumulate (MMA) operation over each input tile that contributes to the output tile.

As we have been doing before the kernel begins by setting up the initial state. As this kernel is structured as a loop we will first start by setting up the loop state.

```
%m_tile_size = constant <i32: 64> : tile<i32>
%m_stride_factor = cuda_tile.constant <i32: 64> : tile<64x64xi32> // todo
    fix the line # and restore this %n_tile_size = cuda_tile.constant <i32:
    64> : tile<i32>
%k_tile_size = cuda_tile.constant <i32: 64> : tile<i32>
```

First we specify the tile sizes as constants. In this case because we are using square tiles of square matrices everything is equivalent to 64. Constants in **Tile IR** can be tensor valued, and in this case we create 0-d tensors containing a single scalar value.

```
%range_start = cuda_tile.constant <i32: 0> : tile<i32>
%range_step = cuda_tile.constant <i32: 1> : tile<i32>
%init_accum = cuda_tile.constant <f32: 0.000000e+00> : tile<64x64xf32>
```

Next we initialize constants for the loop which we will talk about more in a moment, and zero initialize an accumulator value for the MMA. It is worth noting that the tensor here is a value, the **Tile IR** compiler will deal with allocation and execution.

```
%tile_size_range = cuda_tile.iota : tile<64xi32>
```

We also create a shared range from `(0, 63)` using cuda_tile.iota again for the reduction dimension.

```
%a_tile_base = cuda_tile.muli %block_x_index, %k_tile_size : tile<i32>
%a_tile_base_reshape = cuda_tile.reshape %a_tile_base : tile<i32> -> tile<1
    xi32>
%a_tile_base_tensor = cuda_tile.broadcast %a_tile_base_reshape :
    tile<1xi32> -> tile<64xi32>
%m_offsets_vec = cuda_tile.addi %a_tile_base_tensor, %tile_size_range : tile
    <64xi32>
```

We must first compute the tensor of offsets for A and B so that we can obtain a tensor of pointers for each in order to load them from memory.

Conceptually we start by computing the offsets of the M dimension using the below Python pseudo code:

```
m_offsets = block_x_index * k_tile_size + arange(0, k_tile_size)
```

This produces a vector starting from the "top-corner" of the tile at `(block_x_index * k_tile_size, block_x_index * k_tile_size + k_tile_size)` .

The striding of the A matrix is `(64, 1)` , meaning we can omit the extra multiplication by 1 for the K dimension as each value in the row of the offset matrix is sequential.

For this the offsets in the K dimension would be `k_offs = arange(0, k_tile_size)` , so we can reuse %tile_size_range below.

Again we can use the following Python pseudo code to compute the offset matrix:

```
a_tile = reshape(m_offsets, (64, 1)) * m_stride + reshape(k_offsets, (1, 64)) *
    k_stride
```

In a libraries like Python's NumPy this computation hides implicit broadcasting that must be expanded in **Tile IR** .

We will do this in two pieces by first computing the offsets along the `M` dimension, the relative offsets along the `K` dimension, and then adding them together to produce the correct tensor of offsets.

We broadcast the `m_offsets` into a matrix where each column is identical and scaled by stride.

```
%m_offsets_matrix = cuda_tile.reshape %m_offsets_vec :
    tile<64xi32> -> tile<64x1xi32>
%m_offsets_broadcast = cuda_tile.broadcast %m_offsets_matrix :
    tile<64x1xi32> -> tile<64x64xi32>
%m_offsets = cuda_tile.muli %m_offsets_broadcast, %m_stride_factor : tile<64
    x64xi32>
```

The matrix, which is now scaled by 64, would then contain these values:

```
   [[   0,    0,    0,  ...,   0,    0,    0],
[  64,   64,   64,  ...,   64,   64,   64],
[ 128,  128,  128,  ...,  128,  128,  128],
   ...,
[3904, 3904, 3904,  ..., 3904,  3904,  3904],
[3968, 3968, 3968,  ..., 3968,  3968,  3968],
[4032, 4032, 4032,  ..., 4032,  4032,  4032]]
```

We then broadcast the k_offsets into a matrix where row is identical and scaled by stride.

```
%ak_offsets_matrix = cuda_tile.reshape %tile_size_range :
     tile<64xi32> -> tile<1x64xi32>
%ak_offsets_broadcast = cuda_tile.broadcast %ak_offsets_matrix :
    tile<1x64xi32> -> tile<64x64xi32>
%ak_offsets = cuda_tile.muli %ak_offsets_broadcast, %m_stride_factor : tile<64
    x64xi32>
```

A is a row-major matrix (i.e the K dimension's stride is 1) so the K offsets contains sequential values.

```
   [[   0,    1,    2,  ...,   61,    62,    63],
[   0,    1,    2,  ...,   61,    62,    63],
[   0,    1,    2,  ...,   61,    62,    63],
   ...,
[   0,    1,    2,  ...,   61,    62,    63],
[   0,    1,    2,  ...,   61,    62,    63],
[   0,    1,    2,  ...,   61,    62,    63]]
```

We add the two of them together resulting in rows where the i-th row begins at the i-th`value of :code:`m_offsets and each column contains the next 63 integers.

```
%a_tile_offsets = cuda_tile.addi %m_offsets, %ak_offsets : tile<64x64xi32>
```

```
   [[   0,    1,    2,  ...,   61,   62,   63],
[  64,   65,   66,  ...,  125,  126,  127],
[ 128,  129,  130,  ...,  189,  190,  191],
   ...,
[3904, 3905, 3906,  ..., 3965, 3966, 3967],
[3968, 3969, 3970,  ..., 4029, 4030, 4031],
[4032, 4033, 4034,  ..., 4093, 4094, 4095]]
```

Note that because this is the 0-th tile of the output matrix it is centered around (0, 0), the next tile will be centered around (64, 0), then (128, 0) and so on. The above computation will result in the 64 x 64 grid at that point based on the grid coordinates.

Now we must do the same for B.

```
n_offs = j * n_tile_size + torch.arange(0, n_tile_size)
b_tile = k_offs[:, None] * k_stride + n_offs[None, :] * n_stride
```

```
%b_tile_base = cuda_tile.muli %block_y_index, %k_tile_size : tile<i32>
%b_tile_base_reshape = cuda_tile.reshape %b_tile_base :
    tile<i32> -> tile<1xi32>
%b_tile_base_tensor = cuda_tile.broadcast %b_tile_base_reshape :
    tile<1xi32> -> tile<64xi32>
%n_offsets_vec = cuda_tile.addi %b_tile_base_tensor, %tile_size_range : tile<64
    xi32>
```

```
%bk_offsets_matrix = cuda_tile.reshape %tile_size_range : tile<64xi32> -> tile
    <64x1xi32>
%bk_offsets = cuda_tile.broadcast %bk_offsets_matrix : tile<64x1xi32> -> tile<64
    x64xi32>
```

```
%n_offsets_matrix = cuda_tile.reshape %n_offsets_vec : tile<64xi32> -> tile<1
    x64xi32>
%n_offsets_broadcast = cuda_tile.broadcast %n_offsets_matrix :  tile<1x64xi32>
    -> tile<64x64xi32>
%n_offsets = cuda_tile.muli %n_offsets_broadcast, %m_stride_factor : tile<64
    x64xi32>
%b_tile_offsets = cuda_tile.muli %bk_offsets, %n_offsets : tile<64x64xi32>
```

Now that we have computed the initial offsets for the pointers we simply convert the base pointers and add the offsets like before.

```
%a_ptr_base_tensor = cuda_tile.reshape %a_ptr_base_scalar :
    tile<ptr<f32>> -> tile<1x1xptr<f32>>
%a_ptr = cuda_tile.broadcast %a_ptr_base_tensor : tile<1x1xptr<f32>> -> tile<64
    x64xptr<f32>>
%a_tile_ptr = offset %a_ptr, %a_tile_offsets :
    tile<64x64xptr<f32>>, tile<64x64xi32> -> tile<64x64xptr<f32>>
```

Large matrix operations are decomposed into smaller tiles processed by a grid of blocks, requiring careful calculation of global memory offsets for each block.

2.2.4 Looping Over Tiles

Now after all that preparation we can perform the core computation of the kernel.

```
%C_tile, %a_ptr_final, %b_ptr_final = for %k in (%range_start to %k_tile_size,
    step %range_start) : tile<i32>
    iter_values(
        %acc_prev = %init_accum,
        %a_tile_ptr_prev = %a_tile_ptr,
        %b_tile_ptr_prev = %b_tile_ptr
    ) -> (tile<64x64xf32>, tile<64x64xptr<f32>>, tile<64x64xptr<f32>>)
{
    // Load a single 64x64 matrix from the tile.
    %A_tile, %token_a = load_ptr_tko weak %a_tile_ptr :
        tile<64x64xptr<f32>> -> tile<64x64xf32>, token
    // Load a single 64x64 matrix from the tile.
    %B_tile, %token_b = load_ptr_tko weak %b_tile_ptr :
        tile<64x64xptr<f32>> -> tile<64x64xf32>, token
    %C_tile_acc = mmaf %A_tile, %B_tile, %acc_prev: tile<64x64xf32>, tile<64
        x64xf32>, tile<64x64xf32>

    // Advance by K block size.
    %block_size = constant <i32: 64> : tile<64x64xi32>
    %a_tile_ptr_next = offset %a_tile_ptr_prev, %block_size
        : tile<64x64xptr<f32>>, tile<64x64xi32>
```

```
            -> tile<64x64xptr<f32>>
    %b_tile_ptr_next = offset %b_tile_ptr_prev, %block_size
        : tile<64x64xptr<f32>>, tile<64x64xi32>
            -> tile<64x64xptr<f32>>
    // Store the partial sum to the 64x64 accumulator.
    continue %C_tile_acc, %a_tile_ptr_next, %b_tile_ptr_next : tile<64x64xf32>,
        tile<64x64xptr<f32>>, tile<64x64xptr<f32>>
}
```

After completing the reduction over the K dimension we need to store the output tile to the C matrix.

We do the same as we did with A and B, but this time the computation is a little different as we can compute both the X and Y dimensions using only the block coordinates. To think about this intuitively the tiled "column"

"rows" produce a single output tile.

```
cm_offsets = block_x_index * BLOCK_SIZE_M + arange(0, BLOCK_SIZE_M)
cn_offsets = pid_n * BLOCK_SIZE_N + arange(0, BLOCK_SIZE_N)
c_tile = c_ptr + stride_cm * reshape(cm_offsets, (64, 1)) + stride_cn * reshape(
    cn_offsets, (64, 1))
```

We omit the index computation for %c_tile_offset then add it to the base pointer like before.

```
%c_tile_offsets = muli %c_tile_x_offsets, %c_tile_y_offsets : tile<64x64xi32>
%c_ptr_base_tensor = reshape %c_ptr_base_scalar :
    tile<ptr<f32>> -> tile<1x1xptr<f32>>
%c_ptr = broadcast %c_ptr_base_tensor :
    tile<1x1xptr<f32>> -> tile<64x64xptr<f32>>
%c_tile_ptr = offset %c_ptr, %c_tile_offsets :
    tile<64x64xptr<f32>>, tile<64x64xi32> -> tile<64x64xptr<f32>>
```

We can then store the value just as we have done in all prior kernels.

```
store_ptr_tko weak %c_tile_ptr, %C_tile :
    tile<64x64xptr<f32>>, tile<64x64xf32> -> token
```

Structured control flow allows efficient iteration over reduction dimensions, accumulating results in registers before writing the final output tile to memory.

2.3 Structured Pointers

The previous examples have looked at how to define matrix multiplication using tensors of pointers and scatter/gather style loads and stores, which take arbitrary tensors of pointers to operate on. These operations give maximal expressivity to programmers but at the cost of potential performance.

These are valuable to understand; as we saw in the last section, you can build arbitrary tensors of pointers then load and store to them flexibly.

For example, if a user created a complete disjoint tensor of pointers, it is challenging for a human or a compiler to obtain meaningful performance from this program. In the worst case, each element will become a completely disjoint memory operation, preventing vectorized or tensorized operations, or code which makes use of cache or thread locality

This flexibility comes at a cost in both requiring more setup and manual computation to implement even relatively simple algorithms like a tiled GEMM, and the potential of being inefficient. Due to the fact that load/store take arbitrary tensors in the degenerate case it is quite possible that they get decomposed into a sequence of arbitrary loads and stores, which do not make good use of memory locality or the underlying hardware.

Tile IR will do its best to obtain good performance with these stores, but optimal performance can more easily be achieved by using a structured pointer, called a `tensor view` in **Tile IR** . tensor views allow us to simplify the programming model of **Tile IR** and to improve the efficiency of user programs.

As we have seen in our previous examples, when a kernel is given a tensor it is a scalar base pointer into global memory.

The base pointer to A points to an allocation that lives in global memory. #

By formulating our previous problem with the correct sizes, we completely side stepped the complex offset computation, and avoided dealing with imperfect tiling, or store/load masking.

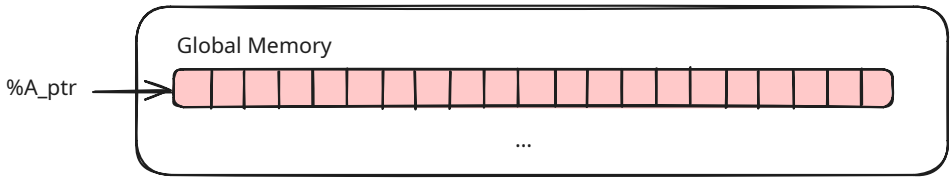

Figure 1: A view of global memory

See XX for more details about masking.

When constructing a `tensor view` we convert the raw pointer into a tensor using static or dynamic shape and stride information.

In order to both simplify the programming model of **Tile IR** and to improve the efficiency of user programs **Tile IR** also has a structured pointer type known as a tensor view . A tensor view can be constructed from raw pointers via cuda_tile.make_tensor_view and attaches shape and stride information to the pointer and effectively represents a typed pointer to a tensor.

When constructing a tensor view , we convert the raw pointer into a tensor using static or dynamic shape and stride information, which is done via cuda_tile.make_tensor_view . This op attaches shape and stride information to the pointer and effectively converts a typed pointer to a tensor.

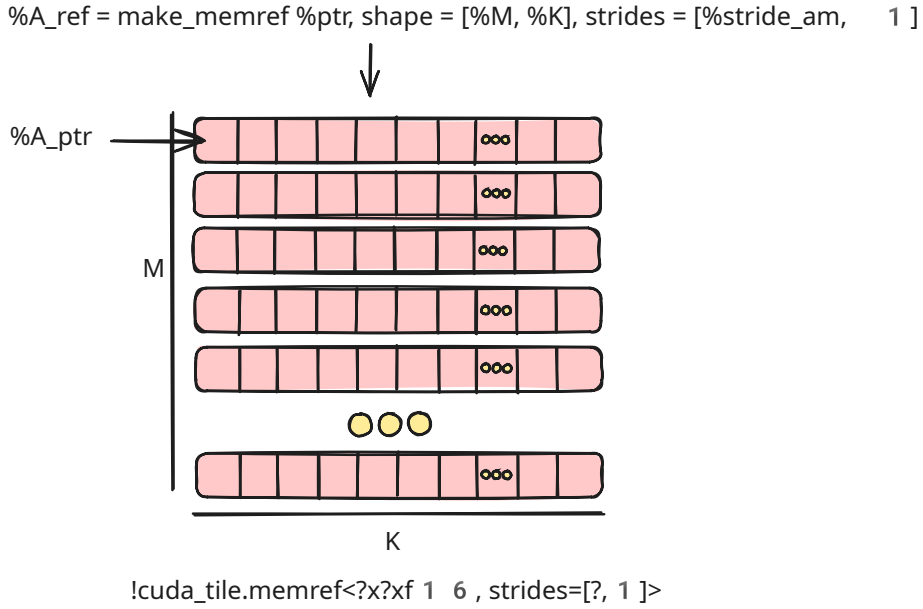

Figure 2: A view of global memory

The base pointer to A points to an allocation that lives in global memory. #

Structured pointers, or tensor views, encapsulate shape and stride information, enabling the compiler to optimize memory access and simplifying the user's code.

2.4 Tiling & Views

Once you have constructed a tensor view, we can make use of cuda_tile.make_partition_view to perform tiling of the underlying tensor.

The base pointer to A points to an allocation that lives in global memory. #

In the last section we simplified the problem by choosing tile dimensions that made the problem perfectly square, in the same layout, using simple tiling and static input dimensions, etc.

We will relax those constraints to show more complex tile mappings as we explore views and partitions.

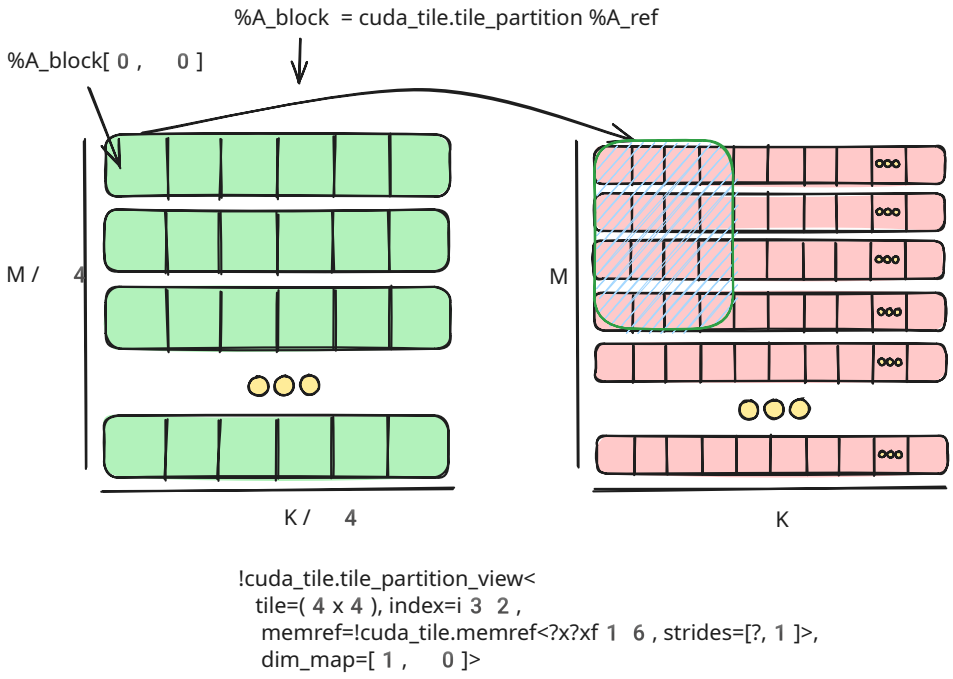

Figure 3: A view of global memory

2.4.1 Vector Addition with Views

We can now implement a more complex vector operation with SAXPY or "Single-Precision A · X Plus Y" (a common

BLAS operation). We can use `tensor view` and cuda_tile.make_partition_view to implement this operation for arbitrary sized vectors.

The kernel first defines its arguments. We now take the inputs `%X` , `%Y` , `%alpha` , as well as the dimensions of the **full** vectors, as arguments.

```
entry @saxpy_memref(%X: tile<ptr<f32>>,
                    %Y: tile<ptr<f32>>,
                    %alpha: tile<f32>,
                    %M : tile<i32>,
                    %N : tile<i32>) {
```

For those familiar with other tile programming models, you might wonder why we need to take the input tensor sizes as arguments instead of using them to control the number of blocks and remain implicit in the program.

The tensor view model differs from some existing tile programming models, wherein each kernel is unaware of the overall dimensions of the problem size beyond the need to mask loads and stores. In contrast, `tensor view` 's require the overall tensor dimensions in order to enable efficient and correct lowering of tiling and its related operations automatically.

```
%x_memref = make_tensor_view %X, shape = [%M, %N], strides = [%M, 1] : tile<i32>
     -> tensor_view<?x?xf32, strides=[?,1]>
%y_memref = make_tensor_view %Y, shape = [%M, %N], strides = [%M, 1] : tile<i32>
     -> tensor_view<?x?xf32, strides=[?,1]>
```

The tensor view's constructor takes the shapes and strides dynamically resulting in a dynamically shaped tensor view specified by ? in the type like so: `!cuda_tile.tile view<?x?xf32, strides=[?,1]>` .

We use cuda_tile.make_partition_view to create views for `%x` and `%y` which represent a `(M/128 x N/256)` tensor of tiles. Each tile will have the size specified by tile parameter of the partition type.

```
%x_view = make_partition_view %x_memref : partition_view<tile=(128x256),
    tensor_view<?x?xf32, strides=[?,1]>>
```

```
%y_view = make_partition_view %y_memref : partition_view<tile=(128x256),
    tensor_view<?x?xf32, strides=[?,1]>>
```

cuda_tile.load_view_tko allows us to load the tile specified by `%view[%x, %y]` for both %X and %Y
.

```
%x_tile, %token_x = load_view_tko weak %x_view[%tileIdX, %tileIdY] :
    partition_view<tile=(128x256), tensor_view<?x?xf32, strides=[?,1]>>, tile<
        i32> -> tile<128x256xf32>, token
```

We can then simply compute using the tiles directly to obtain our result tile.

```
// Step 6. Compute sAXPY: y = alpha * A + y
```

Finally we accumulate the result into `%Y` directly. Here we treat it as both an input and output parameter in this kernel.

```
```

Combining tensor views with partitioning simplifies kernel implementation, allowing direct loading and operating on logical tiles without manual offset arithmetic.

2.4.2 Dynamic GEMM with Views

We have now introduced the core concepts of **Tile IR** . We can put together many of these ideas to support a dynamic GEMM kernel using structured pointers. To start, we make two changes to this GEMM:

the inputs are actually transposed into column-major layout, and the inputs are in `fp16` while the output is in `fp32` .

We take the input and output tensors (as pointers), all dimension, and all strides as arguments.

```
    entry @gemm_kloop_kernel(
        %A_ptr: !cuda_tile.tile<!cuda_tile.ptr<f16>>,
        %B_ptr: !cuda_tile.tile<!cuda_tile.ptr<f16>>,
        %C_ptr: !cuda_tile.tile<!cuda_tile.ptr<f32>>,
        %M: !cuda_tile.tile<i32>, %N: !cuda_tile.tile<i32>, %K: !cuda_tile.tile<
            i32>,
        %stride_ak: !cuda_tile.tile<i32>, %stride_bn: !cuda_tile.tile<i32>, %
            stride_cm: !cuda_tile.tile<i32>
```

The **Tile IR** compiler can optimize the memory loads and stores of Tensor View if the alignment of the underlying pointers and strides are known. If these are statically known then we can infer the alignment directly but if they are dynamic, as they are in this case, we can use the cuda_tile.assume operation to inform the compiler that the pointer is properly aligned.

Here we use the `div_by` predicate to inform the compiler about the divisibility of these values which can be used to infer alignment constraints directly.

```
%A_ptr_assume = assume #cuda_tile.div_by<16>, %A_ptr : tile<ptr<f16>>
%B_ptr_assume = assume #cuda_tile.div_by<16>, %B_ptr : tile<ptr<f16>>
%C_ptr_assume = assume #cuda_tile.div_by<16>, %C_ptr : tile<ptr<f32>>
%stride_ak_assume = assume #cuda_tile.div_by<8>, %stride_ak : tile<i32>
%stride_bn_assume = assume #cuda_tile.div_by<8>, %stride_bn : tile<i32>
%stride_cm_assume = assume #cuda_tile.div_by<8>, %stride_cm : tile<i32>
```

We create a tensor view for `%A` , `%B` , and `%C` .

```
// A reference to the A tensor pointed to by A_ptr, (K x M)
%A = make_tensor_view %A_ptr_assume, shape = [%K, %M], strides = [%stride_ak, 1]
     : tile<i32> -> tensor_view<?x?xf16, strides=[?,1]>
// A reference to the B tensor pointed to by B_ptr, (N x K)
%B = make_tensor_view %B_ptr_assume, shape = [%N, %K], strides = [%stride_bn, 1]
     : tile<i32> -> tensor_view<?x?xf16, strides=[?,1]>
// A reference to the C tensor pointed to by C_ptr, (M x N)
%C = make_tensor_view %C_ptr_assume, shape = [%M, %N], strides = [%stride_cm, 1]
     : tile<i32> -> tensor_view<?x?xf32, strides=[?,1]>
```

We create a Partition View for each tensor. First A:

```
%A_block  = make_partition_view %A : partition_view<tile=(128x64), tensor_view<?
    x?xf16, strides=[?,1]>, dim_map=[1, 0]>
```

Then B:

```
%B_block  = make_partition_view %B : partition_view<tile=(64x128), tensor_view<?
    x?xf16, strides=[?,1]>, dim_map=[1, 0]>
```

And last C:

```
%C_block  = make_partition_view %C : partition_view<tile=(128x128), tensor_view
    <?x?xf32, strides=[?,1]>, dim_map=[0, 1]>
```

```
    // Load a single 64x128 matrix from the tile.
    %B_frag, %t2 = load_view_tko weak %B_block [%k, %bidy] : partition_view<tile
        =(64x128), tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]>, tile<
        i32> -> tile<64x128xf16>, token
```

We then read the the tile block grid coordinates using cuda_tile.get_tile_block_id .

```
%bidx, %bidy, %bidz = get_tile_block_id : tile<i32>
```

Due to the `K` dimension being dynamic we could do calculations ourselves to compute the size of the index space of the reduction but **Tile IR** provides an operation to do this for us. cuda_tile.get_index_space_shape to get the size of the index space, and thus the size of the reduction loop.

```
%mk_len_i32:2 = get_index_space_shape %A_block : partition_view<tile=(128x64),
    tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]> -> tile<i32>
```

No we can use a cuda_tile.for loop to iterate over the tile blocks.

A `for` loop is one of the structured control flow operations in **Tile IR** it steps a loop variable over a range from `(start, end, step)` executing the body for each value in the range.

The iter_values are loop carried variables of the body which are initialized in the loop header and are updated each iteration by yielding them with the cuda_tile.continue operation.

In order to implement the reduction we start from `0` to the size of the tiled `K` dimension, stepping uniformly by `1` on each step and a single loop carried variable representing the accumulator tile.

```
%result = for %k in (%i0 to %mk_len_i32#1, step %i1) : tile<i32>
    iter_values(%acc_prev = %cst) -> (tile<128x128xf32>)
```

We load tiles from A and B.

```
// Load a single 128x64 matrix from the tile.
%A_frag, %t1 = load_view_tko weak %A_block[%bidx, %k] : partition_view<tile=(128
    x64), tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]>, tile<i32> ->
    tile<128x64xf16>, token
// Load a single 64x128 matrix from the tile.
%B_frag, %t2 = load_view_tko weak %B_block [%k, %bidy] : partition_view<tile=(64
    x128), tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]>, tile<i32> ->
    tile<64x128xf16>, token
```

We compute the MMA, and then continue to the next loop iteration with the value.

```
    %acc = mmaf %A_frag, %B_frag, %acc_prev: tile<128x64xf16>, tile<64x128xf16>,
         tile<128x128xf32>
    continue %acc : tile<128x128xf32>
```

Finally, like our previous implementation, outside the loop we store to the tile back to `%C` this time via the view avoiding the need to compute the offsets again.

```
%t3 = store_view_tko weak %result, %C_block[%bidx, %bidy] : tile<128x128xf32>,
    partition_view<tile=(128x128), tensor_view<?x?xf32, strides=[?,1]>, dim_map
    =[0, 1]>, tile<i32> -> token
```

Tensor views support dynamic shapes and strides, enabling robust, flexible kernels that handle varying input sizes while maintaining high performance.

2.5 Cross TileBlock Communication

```
cuda_tile.module @hello_cross_block {
  global @_global_printf_mutex <i32: 1> : tile<1xi32>
  entry @hello_cross_block_kernel() {
    %idx, %idy, %idz = get_tile_block_id : tile<i32>
    %tilex, %tiley, %tilez = get_num_tile_blocks : tile<i32>
    %2 = get_global @_global_printf_mutex : tile<ptr<i32>>
    %3 = cuda_tile.constant <i32: 0> : tile<i32>
    %4 = cuda_tile.constant <i32: 1> : tile<i32>
    loop {
      %t1 = make_token : token
      %6, %t2 = atomic_cas_tko relaxed device %2, %4, %3 token=%t1: tile<!
          cuda_tile.ptr<i32>>, tile<i32> -> tile<i32>, !cuda_tile.token
      %7 = trunci %6 : tile<i32> -> tile<i1>
      if %7 {
        break
      }
    }
    print "current tile: %i / %i\0A", %idx, %tilex : tile<i32>, tile<i32>
    %5, %t4 = atomic_rmw_tko relaxed device %2, xchg, %4: tile<ptr<i32>>, tile
        <i32> -> tile<i32>, !cuda_tile.token
    return
  }
}
```

3 Syntax

Tile IR is intended to be constructed using the **Tile IR** MLIR dialect and stored as bytecode.

To enable humans to comprehend **Tile IR** bytecode programs, we provide an **unstable** textual representation based on the MLIR dialect.

This textual representation has no stability guarantees and it is not intended to be used for writing **Tile IR** programs.

3.1 Module

A **Tile IR** program consists of a **Tile IR** module which contains a series of items.

```
symbol_name := `@` identifier
cuda_tile.module @symbol_name {
    <items>*
}
```

3.2 Items

An item is a kernel definition or a global variable definition.

```
<items> ::= <kernel_definition> | <global_variable_definition>
```

3.3 Globals

A global variable definition is a variable that is defined outside of a kernel.

```
global_variable_definition ::= `global` <symbol_name> `:` <type> `=` <value>
```

3.4 Kernels

A kernel definition is a function that is defined inside a **Tile IR** module.

```
ssa_name := `%` identifier
function_signature ::= <function_parameter>*
function_parameter ::= <ssa_name> `:` <type>
<kernel_definition> ::= `func` @kernel_name `(` <function_signature> `) ->` <
    function_type> {
    <kernel_body>
}
```

A kernel definition's body is a sequence of operations.

```
kernel_body ::= <operation>*
operation ::= (ssa_name `,`?)* `=` <operation_name> <ssa_name>* attribute=
    attribute_value : type ...
```

3.5 Types

A type in **Tile IR** is a fixed pre-defined set of types.

```
element_type ::= `f32` | `f64` | `i8` | `i16` | `i32` | `i64` | `b8` | `b16` | `
    b32` | `b64`
type ::= `tile <` shape `x` element_type `>`
shape ::= `[` integer_literal (`x` integer_literal)* `]`
```

4 Binary Format

The **Tile IR** bytecode is a binary representation of **Tile IR** modules including all items (globals, kernels, functions), attributes, and instructions.

The bytecode format is stable, versioned, and provides **Tile IR** portability guarantees.

The bytecode format produced by older **Tile IR** compilers/drivers can be read by newer compilers/drivers.

A compiler/driver accepts bytecode up to its supported **Tile IR** version.

The remainder of this section describes the **Tile IR** bytecode format and its encoding.

The bytecode has a few top-level design goals:

- To represent a finite but expandable over time set of operations.
- To use a minimal set of types for the encoding.
- To enable manual construction and inspection by humans.
- To allow lazy loading of functions.
- To support forward and backward compatibility of the encoding.

The encoding of individual operations is described in Operations .

4.1 Primitives

The bytecode is composed of a handful of primitive types, each with their own encoding described below. The format uses these primitives to encode more complex data structures for representing the full set of **Tile IR** items.

4.1.1 Fixed-Width Integers

Fixed width integers are unsigned integers of a known size (in bytes). The values are stored in little-endian byte order.

Note : All multi-byte values in the **Tile IR** bytecode format use little-endian encoding, including fixed-width integers, array offsets, and type indices.

```
byte ::= `0x00`...`0xFF`
```

4.1.2 Variable-Width Integers

Variable width integers, or `VarInt` s, provide a compact representation for integers.

Each encoded `VarInt` consists of one to nine bytes, which together represent a single

64-bit value. The **Tile IR** bytecode utilizes the `PrefixVarInt` encoding for `VarInt` s.

This encoding is a variant of the `LEB128` ("Little-Endian Base 128") encoding, where each byte of the encoding provides up to `7` bits for the value, with the remaining bit used to store a tag indicating the number of bytes used for the encoding. Small unsigned integers

(less than `2^7`) may be stored in one byte, larger unsigned integers (up to `2^14`) may be stored in two bytes, and so on.

```
varint ::= `0x00`...`0xFF`
```

4.2 File Structure

The **Tile IR** bytecode file is composed of a stream of bytes; which is composed of a header followed by multiple sections. The header contains the 8-byte "magic number", and a version number.

Each section is expected to appear once within a bytecode file and is identified by a type code, the length, alignment, padding, and the payload. This design allows forward compatibility and selective parsing (tools can skip unknown sections, or unneeded sections).

```
bytecode {
  magic: "\x7FTileIR\x00",
  version: varint,
  sections: section[]
}
```

4.2.1 Magic Number

The **Tile IR** bytecode magic number consumes 8 bytes and is `\x7FTileIR\x00` . The magic number must be present at the beginning of the bytecode file to be accepted by the driver.

4.2.2 Version

Following the magic number is the version of the bytecode format. The version allows both forwards and backwards compatibility of the format.

The version is encoded as three little-endian fixed-width fields immediately following the magic number:

```
version {
  major: uint8_t,    // Major version number
  minor: uint8_t,    // Minor version number
  tag: uint16_t      // Version tag (little-endian)
}
```

Each file encodes the version which allows parsers to adapt to the specific version.

We do not break old versions of the format, if a newer file uses features an older parser can't interpret, it skips unknown optional features but will fail gracefully in cases where new features are unsupported. A new producer can also target older versions of the bytecode directly ensuring compatibility with older drivers.

Version Compatibility Guarantees

1. Backward Compatibility (a new deserializer can read old bytecode): The deserializer must support all previous versions of the bytecode format. The deserializer must handle all previously existing opcodes, types, and sections. The deserializer must maintain original program semantics.

2. Forward Compatibility (an old deserializer can read new bytecode): The deserializer must fail only if it encounters unknown required features. The deserializer must only read bytecode up to its version. The deserializer must error with a clear message if there is a version mismatch.

3. Version Targeting (a serializer can target specific older versions): The serializer may target one or more specific older versions. The serializer must validate all features are supported by the requested target version. The serializer must fail if attempting to serialize unsupported features.

4. Optional vs Required Features (required changes are clearly documented): Required changes must be introduced in a new bytecode version and gracefully failure must be designed. Optional changes must be clearly documented and must preserve the above guarantees.

Examples :

- For new ops → Add new opcodes
- For changes to existing ops → Typically keep the old format for backward compatibility and add new opcode rather than modifying existing ones

4.2.3 Sections

The remainder of the bytecode stream is composed of sections. Sections are used to group data within the bytecode and allow operations on the stream to be per-section enabling out-of-order processing of data and/or lazy-loading.

Each section contains a section ID, whose high bit indicates if the section has alignment requirements, a length and an optional alignment. A section ID is a 7-bit integer with the high bit indicating if the section has alignment requirements. When an alignment is present, a variable number of padding bytes (each byte = `0xCB`)

may appear before the section data. The alignment of a section must be a power of 2. The alignment is represented as a `VarInt` . The padding ensures the section data starts at the specified alignment boundary.

```
section {
  idAndIsAligned: byte   // low 7 bits = section ID, high bit = alignment bit
  length: varint,
  alignment: varint?,    // present only if high bit was set
  padding: byte[],       // bytes of 0xCB as needed
  data: byte[]
}
```

Deserialization The following are the implications of the section format on the deserialization process.

1. The deserializer must read the section ID and length (`idAndIsAligned`) as a single byte.
2. If the section ID has the high bit set, the deserializer must read the alignment (`alignment`) and apply the alignment by skipping `0xCB` bytes as needed.
3. The deserializer must read the length (`length`) bytes of the payload. The length is a `VarInt` .
4. The deserializer must move on to the next section until `EOF` (end of file).

The string section holds all textual names used by the module to avoid repeating them inline.

The section is encoded with the total number of strings, followed by the start index of each of the individual strings. The remaining encoding contains a single blob containing all the strings concatenated together. This design allows loading a specific string without reading the whole section. Strings in the bytecode are stored as raw byte sequences (in UTF-8 encoding) with an associated size. Finding the i-th string involves jumping to `stringStartIndex[i]` and reading until the next offset or end of the blob.

```
strings {
  numStrings: varint,
  stringStartIndex: uint32[],
  stringData: byte[]
}
```

Function Table Section The function table section enumerates the module's functions and embeds their code inline. First, `numFunctions` indicates how many functions follow; for each function `i` , `nameIndex[i]` is an index into the string section naming the function and `signatureIndex[i]` is an index into the Type section specifying its parameter, return types, global flags, etc (so multiple functions with identical signatures can share the same type entry). The field `functionLocIndex[i]` is a `VarInt` referencing an entry in the Debug Section that describes the function's definition location (e.g., source file and line). If `functionLocIndex[i]` is zero, there is no associated debug metadata for the function's definition scope. `lengthOfFunction[i]` states how many bytes of code belong to function `i` , and `functionBody[i]` contains exactly those bytes of instruction encodings. This layout avoids a separate code section and makes parsing each function straightforward: once you read the metadata for function `i` , you can either parse its instructions directly or skip them by advancing `lengthOfFunction[i]` bytes. The instruction encoding itself allows each operation to include a slot for its source location, referencing an entry in the debug section.

The instruction encodings (opcodes, operands, etc.) are described later under Operation

Opcodes and Encodings Section.

The function table encoding has been updated to include function flags and optional optimization hints:

```
functionTable {
  numFunctions : varint
  // for each function i in [0..numFunctions-1]:
  nameIndex[i] : varint                    // References the function's name in
      the StringSec
  signatureIndex[i] : varint               // References the extended function
      signature in the TypeSec
  entryFlag[i] : byte                      // Function flags (visibility, kind,
      optimization hints)
  functionLocIndex[i] : varint             // 0 means no debug location
  optimizationHints[i]? : self-contained  // Present only if
      HasOptimizationHints flag is set
  lengthOfFunction[i] : varint
  functionBody[i] : byte[lengthOfFunction[i]]
}
```

The `entryFlag` byte encodes the following information:

- **Bit 0 (0x01)** : Visibility Flag (0 = Public, 1 = Private)
- **Bit 1 (0x02)** : Function Kind Flag (0 = Device Function, 1 = Kernel Entry Point)
- **Bit 2 (0x04)** : Has Optimization Hints Flag (0 = No, 1 = Yes)
- **Bits 3-7** : Reserved for future extensions

The constant data section holds large constants (e.g., dense tensors, large arrays) separately from code.

The current implementation does not impose explicit size limits on individual constants.

Constants are stored using `uint64_t` offsets, allowing for very large constant data.

However, practical limits may be imposed by available memory and any downstream compiler or runtime limitations (such as CUBIN generation constraints).

As we have done with the string section we move the constants into their own section to avoid bloating the function table section and allow for the lazy loading of specific constants when needed.

Individual operations may reference constants by an index into this section.

The section is encoded with the total number of constants, followed by the start index of each of the individual constants. The remaining encoding contains a single blob containing all the constants concatenated together.

```
constant {
  numConstants: varint,
  constantStartIndex: uint64_t[],
  constantData: byte[]
}
```

Type Section The type section stores all type definitions (scalar types, function signatures, parametric types, etc.) used in the module. The section begins with `numTypes` specifying how many types follow, then an array of offsets (`typeStartIndex`) into the encoded blob (`typeData`). To load the i-th type definition, you use the offset contained in `typeStartIndex[i]` to index into the `typeData` blob and parse the definition from there. The `typeData` blob is a single encoded binary blob containing the type definitions concatenated together.

```
type {
  numTypes: varint
  typeStartIndex: uint32_t[] // array of offsets, length = numTypes
  typeData: byte[]           // concatenated bytes for all type definitions
}
```

Each individual type definition will consist of a `typeTag` followed by a predefined payload structure specific to that `typeTag` . This deterministic approach allows parsers to understand the `payload` layout based solely on the `typeTag` .

```
typeTag : byte
payload : byte[lengthOfPayload]
```

- `typeTag` indicates the "kind" of type. This can be a simple scalar, a function signature, or a more complex structure like a tensor.
- `payload` is interpreted based on `typeTag` . For instance, a function signature might store the number of parameters, references to their types, etc, while a tensor type might store dimensions and an element type.

Example `typeTag` Values:

`typeTag`	Meaning	Expected Payload
0x01	i1 (1-bit int)	None
0x02	i32 (32-bit int)	None
0x03	i64 (64-bit int)	None
0x04	tensor	`elementTypeIndex + rank + dimensions[]`
0x10	Function	Extended function signature
···	Future types	Depends on the type

Detailed Type Encodings The following sections describe the specific encoding format for each type tag:

Integer Types (`typeTag` = 0x00-0x04)
Integer types require no additional payload data beyond the type tag:

```
I1:   typeTag = 0x00  // 1-bit boolean
I8:   typeTag = 0x01  // 8-bit integer
I16:  typeTag = 0x02  // 16-bit integer
I32:  typeTag = 0x03  // 32-bit integer
I64:  typeTag = 0x04  // 64-bit integer
```

Float Types (`typeTag` = 0x05-0x0B)
Float types require no additional payload data beyond the type tag:

```
F16:      typeTag = 0x05  // 16-bit IEEE float
BF16:     typeTag = 0x06  // 16-bit bfloat
F32:      typeTag = 0x07  // 32-bit IEEE float
TF32:     typeTag = 0x08  // TensorFloat-32
F64:      typeTag = 0x09  // 64-bit IEEE float
F8E4M3FN: typeTag = 0x0A  // 8-bit float (E4M3FN format)
F8E5M2:   typeTag = 0x0B  // 8-bit float (E5M2 format)
```

Pointer Type (`typeTag` = 0x0C)

```
typeTag : byte = 0x0C      // Pointer type
pointeeTypeIndex : varint  // Index of the pointee type
```

Tile Type (`typeTag` = 0x0D)

```
typeTag : byte = 0x0D      // Tile type
elementTypeIndex : varint  // Index of the element type
shape : int64_t[]          // Shape dimensions (var-length array)
```

TensorView Type (`typeTag` = 0x0E)

```
typeTag : byte = 0x0E      // TensorView type
elementTypeIndex : varint  // Index of the element type
shape : int64_t[]          // Shape dimensions (var-length array)
strides : int64_t[]        // Stride values (var-length array)
indexTypeTag : byte        // Index type (I32=0x03 or I64=0x04)
```

PartitionView Type (`typeTag` = 0x0F)

```
typeTag : byte = 0x0F        // PartitionView type
tileShape : int32_t[]        // Tile shape (var-length array)
tensorViewTypeIndex : varint // Index of the TensorView type
dimMap : int32_t[]           // Dimension mapping (var-length array)
masked : byte                // Masked flag (0x00=false, 0x01=true)
```

Function Type (`typeTag` = 0x10)

```
typeTag : byte = 0x10 // Function Type
numParams : varint    // Number of input parameters
paramTypeIndices : varint[]  // Array of type indices for each parameter
numResults : varint   // Number of results
resultTypeIndices : varint[] // Array of type indices for each return value
```

The function type encoding stores only the essential type information (input and result types). Argument attributes and other function metadata are stored separately in the function table section.

Token Type (`typeTag` = 0x11)

```
typeTag : byte = 0x11  // Token type (no additional payload)
```

Attribute Encoding Attributes in **Tile IR** bytecode can be encoded in two ways:

1. **Inline encoding** - Simple attributes are encoded directly in the instruction stream
2. **Self-contained encoding** - Complex attributes include a type tag followed by their data

Self-contained attributes use the following format:

```
attributeTag : byte
attributeData : byte[]  // Format depends on attributeTag
```

The following attribute tags are supported:

Integer Attribute (`attributeTag` = 0x01)

```
attributeTag : byte = 0x01  // Integer attribute
typeIndex : varint          // Type index for the integer type
value : varint              // Integer value (zero-extended)
```

Float Attribute (`attributeTag` = 0x02)

```
attributeTag : byte = 0x02  // Float attribute
typeIndex : varint          // Type index for the float type
value : byte[]              // APFloat representation (variable length)
```

Bool Attribute (`attributeTag` = 0x03)

```
attributeTag : byte = 0x03  // Bool attribute
value : byte                // 0x00=false, 0x01=true
```

Type Attribute (`attributeTag` = 0x04)

```
attributeTag : byte = 0x04  // Type attribute
typeIndex : varint          // Index of the referenced type
```

String Attribute (`attributeTag` = 0x05)

```
attributeTag : byte = 0x05  // String attribute
stringIndex : varint        // Index into the string section
```

Array Attribute (`attributeTag` = 0x06)

```
attributeTag : byte = 0x06    // Array attribute
numElements : varint          // Number of elements
elements : self-contained[]   // Array of self-contained attributes
```

DenseElements Attribute (`attributeTag` = 0x07)

```
attributeTag : byte = 0x07  // DenseElements attribute
typeIndex : varint          // Type index for the shaped type
constantIndex : varint      // Index into constant section (for numeric data)
// OR for string data:
numStrings : varint         // Number of string elements
stringIndices : varint[]    // Indices into string section
```

DivBy Attribute (`attributeTag` = 0x08)

```
attributeTag : byte = 0x08    // DivBy attribute
divisor : varint              // Divisor value
flags : byte                  // Bit 0: unsignedInt, Bit 1: hasEvery, Bit 2:
    hasAlong
every : signed_varint?        // Present if Bit 1 set
along : signed_varint?        // Present if Bit 2 set
```

SameElements Attribute (`attributeTag` = 0x09)

```
attributeTag : byte = 0x09  // SameElements attribute
values : int64_t[]          // Array of int64 values (var-length)
```

Dictionary Attribute (`attributeTag` = 0x0A)

```
attributeTag : byte = 0x0A      // Dictionary attribute
numEntries : varint             // Number of key-value pairs
entries : dictEntry[]           // Array of dictionary entries
dictEntry {
  keyStringIndex : varint       // Index of key string
  value : self-contained        // Self-contained attribute value
}
```

OptimizationHints Attribute (`attributeTag` = 0x0B)

```
attributeTag : byte = 0x0B  // OptimizationHints attribute
dictionary : dictionary    // Dictionary attribute (without tag)
```

NonNegative Attribute (`attributeTag` = 0x0C)

```
attributeTag : byte = 0x0C  // NonNegative attribute
// No additional payload - presence indicates non-negative constraint
```

Global Section The global section stores module-level global variables. This section is optional and is only present if the module contains `cuda_tile.global` operations.

```
global {
  numGlobals: varint
  // for each global i in [0..numGlobals-1]:
  symbolNameIndex[i] : varint   // References the global's symbol name in the
      StringSec
  valueTypeIndex[i] : varint    // Type index for the global's value type
  constantValueIndex[i] : varint // Index into constant section for the global's
       initial value
}
```

Each global variable is encoded with:

- **symbolNameIndex** : Index into the string section for the global's symbol name
- **valueTypeIndex** : Index into the type section for the global's type (typically a shaped type like tensor)
- **constantValueIndex** : Index into the constant section containing the global's initial value

The debug section stores the serialized debug information (for more details about debug information see Debug Info).

This section is optional as certain tools may ignore it and serializers may leave it empty for release builds.

```
debug {
  diOpsNum: varint          // Total number of operations with debug info
  padding: bytes            // Align to 4 bytes
  diIndexOffsets: uint32_t[] // Per op offset into the debug info indices
  diIndicesNum: varint      // Total number of debug info indices
  padding: bytes            // Align to 4 bytes
  diIndices: uint64_t[]     // Array of debug indices to debug info attributes
  diAttrNum: varint         // Total number of debug info attributes
  padding: bytes            // Align to 4 bytes
  diOffsets: uint32_t[]     // Per debug info attribute offset into the debug
      info data
  diData: bytes             // Data for each debug info attribute
}
```

The debug section uses a multi-level indirection scheme:

1. **Operations** : `diOpsNum` operations have debug info, with `diIndexOffsets` pointing into the indices array
2. **Indices** : `diIndicesNum` total indices in `diIndices` , referencing debug info attributes
3. **Attributes** : `diAttrNum` debug info attributes stored in `diData` with offsets in `diOffsets`

Each debug info attribute begins with a `debugEntryType` indicating what kind of debug info it is:

- `0x00` = “Unknown”
- `0x01` = “DICompileUnit”
- `0x02` = “DIFile”
- `0x03` = “DILexicalBlock”
- `0x04` = “DILoc”
- `0x05` = “DISubprogram”
- `0x06` = “CallSite”

`debugEntryPayload` describes line, file, variable name, function index, instruction offset, etc. Each `debugEntryType` has a fixed, known structure. If `functionLocIndex[i]` or an instruction’s `locationIndex` is non-zero, it references `debugEntryOffset[...]` in this section, whose payload can store file/line info or other metadata.

Debug Entry Payload Formats Each debug entry in `diData` begins with a `debugEntryType` byte followed by type-specific data:

```
debugEntry {
  debugEntryType : byte     // Identifies the debug info type
  entryData : byte[]        // Type-specific payload (format below)
}
```

The following debug entry types are supported:
Unknown Debug Info (`debugEntryType` = 0x00)

```
debugEntryType : varint = 0x00  // Unknown debug info
// No additional payload
```

DICompileUnit (`debugEntryType` = 0x01)

```
debugEntryType : varint = 0x01  // DICompileUnit
language : varint               // Source language identifier
fileIndex : varint              // Index of associated DIFile
producer : varint               // String index for compiler producer
optimized : byte                // 0x00=false, 0x01=true
emissionKind : varint           // Emission kind enumeration
```

DIFile (`debugEntryType` = 0x02)

```
debugEntryType : varint = 0x02  // DIFile
filename : varint               // String index for filename
directory : varint              // String index for directory
```

DILexicalBlock (`debugEntryType` = 0x03)

```
debugEntryType : varint = 0x03  // DILexicalBlock
line : varint                   // Line number
column : varint                 // Column number
scopeIndex : varint             // Index of parent scope (DIFile or DISubprogram
    )
```

DILoc (`debugEntryType` = 0x04)

```
debugEntryType : varint = 0x04  // DILoc (source location)
line : varint                   // Line number
column : varint                 // Column number
scopeIndex : varint             // Index of scope (DISubprogram, DILexicalBlock,
     etc.)
inlinedAtIndex : varint         // Index of inlined location (0 if not inlined)
```

DISubprogram (`debugEntryType` = 0x05)

```
debugEntryType : varint = 0x05  // DISubprogram (function debug info)
name : varint                   // String index for function name
linkageName : varint            // String index for linkage name
fileIndex : varint              // Index of associated DIFile
line : varint                   // Line number where function is defined
typeIndex : varint              // Index of function type
scopeLineIndex : varint         // Line number where scope begins
flags : varint                  // Function flags (visibility, etc.)
unitIndex : varint              // Index of associated DICompileUnit
```

CallSite (`debugEntryType` = 0x06)

```
debugEntryType : varint = 0x06  // CallSite location
calleeIndex : varint            // Index of called location
callerIndex : varint            // Index of calling location
```

4.3 Operation Opcodes and Encodings

Each instruction in **Tile IR** bytecode is represented as:

```
opcode : byte
locationIndex : varint
instructionSpecificFields : byte[]
```

In this representation, `opcode` uniquely identifies the operation. This matches one of the operations defined in the **Tile IR** dialect. `locationIndex` is always present; if the instruction does not carry debug info, this field is 0. A non-zero value refers to an entry in the Debug Section that contains file, line, or other source-level metadata. The instruction-specific fields (operands, attributes, etc.) follow a layout defined by the opcode.

Additionally, we do not store a `resultIndex` for each producing instruction. Instead, we rely on a sequential pass to assign local value indices at parse-time.

Older parsers are expected to skip or reject unknown opcodes. Over time, new operations can be added simply by assigning new opcodes and defining their binary payload formats.

4.4 Operation Encoding Details

The general structure for operation encoding follows a consistent pattern, but varies based on the operation's characteristics:

General Operation Structure

```
opcode : byte                       // Operation identifier
locationIndex : varint              // Debug location (0 = no debug info)
resultTypes : typeIndex[]?          // Present for variadic result operations
flags : varint?                     // Optional flags for operations with optional
    fields
attributes : encoded_attr[]         // Operation-specific attributes
operands : operand_encoding         // Operation-specific operand encoding
regions : region_encoding[]?        // Present for operations with regions
```

Flags Field Encoding

For operations with optional attributes or operands, a flags field is used:

```
flags : varint                      // Bitfield encoding optional presence
```

The flags field uses individual bits to indicate the presence of optional attributes and operands:

- **Bits 0-N** : Optional attributes (in declaration order)
- **Bits N+1-M** : Optional operands (in declaration order)

UnitAttr attributes are encoded only in the flags field - no additional data is written.

Operand Encoding Patterns

Operands are encoded differently based on the operation's operand structure:

1. **Fixed Operands** : Written as sequential operand indices
2. **Variadic Operands** : Prefixed with operand count, then indices
3. **AttrSizedOperandSegments** : Each operand group encoded separately

```
// Fixed operands (e.g., binary operations)
operand1Index : varint
operand2Index : varint
// Variadic operands (e.g., function calls)
numOperands : varint
operandIndices : varint[numOperands]
// AttrSizedOperandSegments (e.g., operations with optional operand groups)
group1 : optional_operand_group
group2 : optional_operand_group
...
```

Result Type Encoding

- **Fixed Results** : No result types encoded (inferred from operation)
- **Variadic Results** : Number of results encoded, followed by type indices

```
// Variadic results
numResults : varint
resultTypeIndices : varint[numResults]
```

Region Encoding

Operations with regions encode them after operands:

```
numRegions : varint
regions : region[numRegions]
region {
  numBlocks : varint
  blocks : block[numBlocks]
```

```
}
block {
  numArgs : varint
  argTypeIndices : varint[numArgs]
  numOps : varint
  operations : operation[numOps]
}
```

Common Operation Examples
Arithmetic Operations (e.g., `cuda_tile.add`)

```
opcode : byte                    // e.g., 0x15 for add
locationIndex : varint           // Debug location
lhs : varint                     // Left operand index
rhs : varint                     // Right operand index
// Result type inferred from operands
```

Memory Operations (e.g., `cuda_tile.load`)

```
opcode : byte                    // e.g., 0x20 for load
locationIndex : varint           // Debug location
resultType : varint              // Type index for loaded value
address : varint                 // Address operand index
// Optional attributes encoded via flags
```

Control Flow Operations (e.g., `cuda_tile.if`)

```
opcode : byte                    // e.g., 0x30 for if
locationIndex : varint           // Debug location
condition : varint               // Condition operand index
numRegions : varint              // Always 2 for if (then, else)
thenRegion : region              // Then block
     elseRegion : region              // Else block (may be empty)
```

4.5 Section Ordering

Tile IR bytecode readers can handle sections in any order due to their flexible parsing design.

The reader first discovers all sections and stores their payloads, then processes them in dependency order.

Default Writing Order

Writers typically emit sections in the following order:

1. **Header** (magic number + version)
2. **Global Section** (Section ID: 0x06) - Optional, only if globals present
3. **Function Table Section** (Section ID: 0x02) - Required
4. **Constant Data Section** (Section ID: 0x04) - Optional, only if constants present
5. **Debug Section** (Section ID: 0x03) - Optional
6. **Type Section** (Section ID: 0x05) - Required
7. **String Section** (Section ID: 0x01) - Required
8. **End-of-Bytecode Marker** (0x00) - Required

However, readers are not dependent on this order and can process sections regardless of their arrangement in the file. This flexibility enables future optimizations and different writing strategies.

Reader Implementation

The reader implements this flexibility by:

1. **Discovery Phase** : Reads all section headers and stores their payloads in memory

2. **Processing Phase** : Processes sections in dependency order regardless of file order
3. **Lazy Resolution** : Resolves forward references (e.g., types, strings) on-demand

This design allows for efficient random access to any section and supports future file format optimizations.

Section Dependencies

- Function table references: types, strings, constants, debug info
- Global section references: types, strings, constants
- Debug section references: strings
- All sections may reference the string section

5 Type System

All values and operations in **Tile IR** are statically typed. This section defines **Tile IR** 's types, as well as their equivalence, layouts, and other type system details that may be relevant for DSL and compiler authors.

Notably **Tile IR** is tensor valued: all values are tensors. We have two concrete tensor types: tile, a pure tensor value, and view, a structured pointer to a tensor in memory. Additionally, we use element types in the formulation of our type system. They do not describe a value on their own.

5.1 Element Types

Element types are the native data types supported by **Tile IR** . By themselves they do not describe a value. As **Tile IR** operates over tensors, these types describe the hardware accelerated, primitive values that can be contained by a tensor. They specify how a sequence of bits are to be interpreted. Each element type has a size associated with it that represents the number of bits required to represent it.

Note

Note that this is different from a potential storage size, which is specified by the data layout of the tensor which contains these values.

Element types come in two flavors, general purpose fundamental types that come without restriction, and specialized alternative types which each come with a set of restrictions.

5.1.1 Fundamental Types

Tile IR supports a set of general purpose integer and floating-point types that are supported by all operations and have no restrictions.

These can be contained in arbitrary rank, and shape tensors, and 0-rank values of this type can be treated as scalars.

Type	Sizes	Description
i1, i8, i16, i32, i64	1, 8, 16, 32, 64	signless integer type of specified size
f16, f32, f64	16, 32, 64	IEEE floating-point type of specified size

Primary elemental types are supported in all arithmetic operations.

Warning

Integer types are signless, i.e., the type does not encode whether the represented value is to be interpreted as a signed or unsigned value. For operations where this distinction is semantically meaningful signedness is controlled via flags on each arithmetic operation.

5.1.2 Alternative Types

Tile IR also supports a set of non-standard but hardware accelerated floating-point types. Due to the nature of these types and hardware they each come with a set of restrictions.

Type	Size	Description
tf32	32	floating-point format with 8 bits for exponent and 10 bits for mantissa. Storage size is 4 bytes with 4-by
bf16	16	floating-point format with 8 bits for exponent and 7 bits for mantissa
e3m4	8	floating-point format with 3 bits for exponent and 4 bits for mantissa
e5m2	8	floating-point format with 5 bits for exponent and 2 bits for mantissa

Tensors of these types may be created, manipulated and loaded and stored from global memory, but certain computations on them are restricted.

5.2 Pointers

Pointers, or values which contain memory addresses, are typed as pointers to a specific pointee type.

Type	Size	Description
`ptr<E>`	64	a pointer to a location in memory; the data at the location represents a value of non-pointer element

The pointer points to a location in memory. The data at that location will be interpreted as being of element type E when loaded.

Pointer arithmetic also assumes the storage size of the type `E` for offset computations. Pointer types are parameterized by element types

(i.e., nested pointer types are not supported). For details about converting between different pointer types, or integers see cuda_tile.bitcast .

5.3 Tensor Types

A tensor is a multi-dimensional, rectangular array described by a shape and element type. The shape is a vector that describes the number of elements across each axis of the tensor. The length of said vector describes the rank of the tensor, i.e., the number of its dimensions.

All tensors in **Tile IR** have a statically known rank. **Tile IR** has two kinds of tensor types tiles and views.

5.3.1 Tile Type

All values in **Tile IR** are of type tile. A tile is a tensor with static shape, i.e., the extent across each dimension is known at compile time. Each extent must be a power of two.

We use `tile<MxNxKxE>` where `M` , `N` , `K` are integer numbers and `E` is an element type as syntax for tile types, see Syntax for more details.

Example Tile Type	Description
`tile<2x8xf32>`	a 2-dimensional tensor with 2 rows and 8 columns of f32 values
`tile<f32>`	a single value of type f32
`tile<ptr<f32>>`	a pointer to a value of type f32
`tile<4x8xptr<f32>>`	a 2-dimensional tensor with 4 rows and 8 columns of pointers to values of type f32

Note

The shape of a tensor of pointers defines the shape of the values that are loaded or stored. It does not imply any structure on the locations themselves. For example, two consecutive elements of the tensor may not be consecutive locations in memory. Even more, the same location may be present multiple times within a single tensor of pointers. See Memory Model for a discussion of implications.

5.3.2 Tensor View

It is common that data in global memory follows a strided structure. For example, the widely adopted row-major or column-major layouts are strided. It is beneficial for the compiler to be aware of the strided layout of data in memory. **Tile IR** features a tensor view type to describe such structure in global memory.

Conceptually, a tensor view type describes an abstracted tensor of pointers. Like a regular tensor, it is described by a shape and the type of elements it points to. It in addition has a vector of striding factors.

The striding factors describe the relative position of locations the elements of the tensor view point to. If an element is d elements apart in dimension i of the tensor view, the corresponding locations in memory will be d * s t r i d e i elements away. This information can be used by the compiler to reason about access patterns and layouts of data in memory.

Values of type tensor view are typically never materialized in memory.

Rather, they are stored as a compact description of a base-location, shape and striding factors. From this information, a tensor of pointers corresponding to the full view value can be computed using the following formula

elem[i0,...,in]=baseptr+$\sum$m=0nim*sm where the s m are the striding factors of the tensor view and b a s e p t r is the start address in global memory.

Other than the tile type, a tensor view supports dynamic extents in the shape vector. We use the `?` character to denote these. Dynamic extents are bound at runtime to values when the view type is constructed using a cuda_tile.make_tensor_view operation.

A dynamically shaped tensor view cannot be directly used to access memory. It first needs to be divided into tiles of static size.

A tensor view is constructed using the cuda_tile.make_tensor_view operation and consists of the components:

- `elementType` , the type of the elements in the view
- `shape` , an array of 64-bit integer describing the size of each dimension
- `strides` , an array of 64-bit integer describing the stride of each dimension

Examples

```
!cuda_tile.tensor_view<1024x1024xf16, strides=[1024,1]>
!cuda_tile.tensor_view<32x16x32xf16, strides=[512,1,16]>
!cuda_tile.tensor_view<?x?xf16, strides=[?,1]>
!cuda_tile.tensor_view<?x16xf32, strides=[1,?]>
```

5.3.3 Subview Types

A tensor view is often too large to be loaded as a single tile for processing. Instead, it is typically first subdivided into smaller tiles. In **Tile IR** this is expressed using subview types.

Subviews describe a mapping from an index space to a space of statically-sized tiles loaded from a tensor view. They define the necessary index computations performed by a cuda_tile.load_view_tko and cuda_tile.store_view_tko when accessing elements from a tensor view. A subview's size is defined by the cardinality of its index space.

Tile IR currently provides a single subview for partitioning a view into a grid of non-overlapping tiles but is designed to support additional subview types in the future that support different indexing patterns.

..
.. For later when we bring grid view back.
..
.. - partitioning a view into a grid of non-overlapping tiles.
.. - partitioning a view into a grid of overlapping tiles.

Partition View `partition_view` is a subview type that represents a view partitioned into a grid of non-overlapping tiles. The index space in this case is the position of the tile in the grid. A tile partition view is specified by providing the size of the loaded tile. Like all tensors, the tile size needs to be a power of two.

Partition views are particularly useful in patterns like matrix multiplication, where a large tensor in global memory is traversed as non-overlapping tiles to form the final result.

The partition view structure is created using the cuda_tile.make_partition_view constructor.

It consists of:

- `tileShape` , the shape of the individual tiles in the view
- view, the type of the view from which the subview is constructed
- `dimMap` , an array of 32-bit integers mapping the tiles dimensions to the dimensions of the view.
- `masked` , a flag defining memory operations outside of the view bounds

These fields are all encoded as part of the type.

Concretely, a tensor view type of `tensor_view<8192x128xf32, strides=[128,1]>` can be partitioned into a grid of `(64x32)` tiles of size `(128x4)` each, which when loaded produce tiles with the type `tile<128x4xf32>` . When loading a tile form such a partition view, the index is the position in the `(64, 32)` grid. Loading element `(4, 2)` hence would return the tile starting at position `(256, 64)` in the underlying view.

Formally, given a tensor view with shape [S 0 , ⋯, S n] and strides [s t 0 , ⋯, s t n] and a partition view with tile size [T 0 , ⋯, T n] , a load or store at position [I 0 , ⋯, I n] will load the elements at location

location[i0,⋯,in]=baseptr+$\sum$m=0nIm$\cdot$ceildiv(Sm,Tm)$\cdot$stm

Warning

In case where the underlying view's shape is not divisible by the tile size, tiles on the boundary might be partial. Reading or writing from and to a partial tile is undefined behavior. By specifying the masked qualifier to the partition view, boundaries can optionally be masked. In that case, out of bounds reads return a padding value and corresponding writes are skipped.

Examples

```
!cuda_tile.partition_view<
  tile=(2),
  view=!cuda_tile.tensor_view<16xf32, strides=[1]>
>
!cuda_tile.partition_view<
  tile=(2x2),
  view=!cuda_tile.tensor_view<16x16xf32, strides=[16,1]>
>
!cuda_tile.partition_view<
  tile=(2x2),
  view=!cuda_tile.tensor_view<16x16xf32, strides=[16,1]>,
  dim_map=[1, 0]
>
!cuda_tile.partition_view<
  masked tile=(2x2),
  view=!cuda_tile.tensor_view<16x16xf32, strides=[16,1]>,
>
```

5.4 Type Equivalence

Tile IR does not provide means to name types. Equivalence of types hence is a purely structural property: Two types are considered equal if they are structurally identical.

Note that some types form a natural subtype relationship. Types with dynamic shapes and strides like view cover the values that an identical type with all dynamic shapes and strides substituted with static values would cover. However, we consider these types distinct in **Tile IR** .

6 Semantics

This section provides a written English presentation of the operational semantics of **Tile IR** . These semantics are intended to provide an understanding of **Tile IR** for: 1) those interested in generating **Tile IR** as a code generation target, or 2) those interested in reading **Tile IR** programs produced by others.

It does **not** attempt to formalize every possible behavior that might be admitted by an axiomatic formulation or a small step operational semantics. For understanding an even more informal presentation of the language and its core concepts see the Programming Model section.

We first introduce the abstract machine state and language definitions before describing semantics of individual kernels and programs.

We then discuss the semantics of broad classes of operations as well, for more detailed descriptions of individual operations and their behavior see Operations for a complete listing.

6.1 The Abstract Machine

The **Tile IR** abstract machine state S is a tuple consisting of the following components, each explained below:

- A well-formed module Mod which stores one of more items, discussed below in Modules .
- A grid of tile blocks TB (or logical tile threads) each representing a single tile-kernel instance.
- A per-tile-block infinite register file, R , that maps named ”registers” to values.
- A global memory store, M , that maps addresses to scalar values.
- A set of pending memory accesses, P , that make progress asynchronously to the execution of **Tile IR** operations.

6.2 Modules

A program in **Tile IR** is represented as a module. A module is a single translation unit which contains zero or more items. An item may either be a:

- a global variable definition
- a tile kernel
- a tile function

For those familiar with CUDA, a tile kernel is the global entry point for a tile program, much like a kernel is in CUDA C++ or PTX.

Tile functions represent device side functions that can be called from the tile kernel currently with some restrictions.

6.2.1 Global Variable

A global is a named global variable that is stored in global device memory and accessible to all tile blocks.

Global variables are declared using the cuda_tile.global operation.

A global variable must be initialized upon declaration and will be initialized exactly once.

A global variable must contain a value of Tile Type .

A global variable can be modified by using the cuda_tile.get_global operation to obtain a pointer which can be used to read and write to the global variable.

6.2.2 Tile Kernel

```
entry @tile_func(%A0: T0, ..., %AN: TN) {
     %0 = op %P0, %P1, ... %PN -> R0
     ...
     return
}
```

The basic unit of execution in **Tile IR** is the tile kernel. A tile kernel is a tile function that acts as the entry point of a tile program. A tile kernel represents a function parameterized by a set of grid coordinates.

At kernel runtime, each unique grid coordinate is available to each kernel instance (tile block).

A tile kernel can query its grid coordinates via cuda_tile.get_tile_block_id and the coordinates can be one-, two-, or three-dimensional depending on the grid the kernel is launched with. A kernel may also query the total the total number of tile blocks along each dimension via cuda_tile.get_num_tile_blocks .

A tile kernel is a tile function with additional restrictions:

- can only have parameters with scalar (i.e., 0-rank) tensor types
- requires all input tensors to be provided as scalar pointers (i.e `tile<ptr<E>>`)
- produces no return value
- the kernel is only executed for its effect on global device memory

A tile kernel is otherwise a tile function and all properties of tile functions also apply to tile kernels.

6.2.3 Tile Function

A tile function consists of a name, a list of formal parameters, a return type, and a body.

A tile function's body contains a single threaded tile program (referred to as *tile block*) parameterized by formal parameters.

A tile function has N formal parameters and produces M return values. The type of the parameters can be one of the valid types described in Type System .

Note

Currently defining non-kernel tile functions is disabled with support planned for a future release.

6.2.4 Function Bodies

A function body consists of a sequence of statements that are in static-single-assignment (SSA) form.

Each statement assigns the result of a single operation to a set of unique result variables.

All operations in **Tile IR** are represented uniformly in this way, including control flow and memory operations.

6.2.5 Well-Formedness

A well-formed module is a module that satisfies the following properties:

- The module contains at least one item.
- Each item is uniquely named within the module.
- Each Tile Kernel and Tile Function has a body that is a sequence of statements in valid static-single-assignment (SSA) form.
- The program type checks according to the rules specified in Type System and the operator type signatures are specified in Operations .

Program well-formedness is required as a pre-condition and post-condition of both optimizations and operational semantic rules.

6.3 Values

Tile IR has a small set of types as described in Type System but we only have two types of values.

- Pointers , which represent a memory address.
- Tiles , or an N-dimensional array of scalars.
- Views , which represent a structured view of memory.

6.3.1 Pointers

A pointer is a 64-bit integer memory address that references a location in global device memory.

Pointers are typed as `ptr<E>` where `E` is the type of the memory location it references.

Pointers are required to be aligned to the size of the underlying datatype they point to see Element Type Encoding for specific encodings and information about allocation layout.

A pointer is a memory address that points to a location in global memory.

Data Layout Allocations pointed to by input pointer values, and by extension views (see below), must conform to the specified data layout.

We expect that the allocation pointed to by ptr<E> is a sized contiguous allocation of scalar values of element type E .

There is no padding between elements of the allocation, and we expect that for an allocation of size N will be equivalent to N * sizeof(E) bytes. The size and encoding of a element is determined by its type E and is defined in the table below.

As an aside the datatype encoding is compatible with DLPack a standard adopted by most deep learning frameworks and array libraries. We provide the equivalent

PyTorch and NumPy encodings for each datatype.

For NumPy low-precision types we provide the equivalent in terms of the ml_dtypes library a standard collection of low-precision NumPy data types.

Warning

Tile IR layouts are currently restricted to be contiguous for sub-byte types.

Tile IR Type	DLPack Type Code	DLPack Bits	DLPack Lanes	NumPy Type	PyTorch Typ
i1	`kDLInt` , `kDLUInt`	8	1	`numpy.uint8` (unpacked)	N/A
i8	`kDLInt` , `kDLUInt`	8	1	`numpy.uint8` , `numpy.int8`	`torch.bool`
i16	`kDLInt` , `kDLUInt`	16	1	`numpy.int16` , `numpy.uint16`	`torch.int16`
i32	`kDLInt` , `kDLUInt`	32	1	`numpy.int32` , `numpy.uint32`	`torch.int32`
i64	`kDLInt` , `kDLUInt`	64	1	`numpy.int64` , `numpy.uint64`	`torch.int64`
f16	`kDLFloat`	16	1	`numpy.float16`	`torch.float`
f32	`kDLFloat`	32	1	`numpy.float32`	`torch.float`
f64	`kDLFloat`	64	1	`numpy.float64`	`torch.float`
bf16	`kDLBfloat`	16	1	`ml_dtypes.bfloat16`	`torch.bfloa`
fp8 (E3M4)	`kDLFloat8_e3m4`	8	1	`ml_dtypes.float8_e4m3fn`	`torch.float`
fp8 (E5M2)	`kDLFloat8_e5m2`	8	1	`ml_dtypes.float8_e5m2`	`torch.float`

Note

Allocations are the only values where memory layout is specified in **Tile IR** .

6.3.2 Tiles

A tile is a immutable N-dimensional array of scalars characterized by:

- Its rank (number of dimensions)
- Its shape (extent along each dimension)
- Its primitive element type

These properties are part of both the tile's type and the size and shape of the value.

A tile may have any any non-negative rank, where:

- Rank-0 tiles represent scalar values.
- Rank-1 tiles represent vectors.
- Rank-2 tiles represent matrices.
- And Rank-N tiles represent higher-order N-d arrays.

For a given tile of type `tile<NxKxE>` , where the tile has shape (N, K) with elements of type E results in a value of size N × K × size (E) , containing N * K individual elements.

A tile value of this type is abstractly represented as a tuple of an array with N * K of elements of type E , and an opaque layout which provides a mapping from the index space of elements to a linear index.

Note

The physical layout and memory representation of a given tile is not visible to the program and is not specified by the language semantics.

The compiler will choose to represent a tile in memory in a way that is most efficient for the target architecture, and specific program and tiles sizes.

6.3.3 Views

Tile IR provides a set of view types as described in Tensor View .

Views provide a structured views of memory by enriching a pointer with additional data.

Views have their own set of memory operations cuda_tile.load_view_tko and cuda_tile.store_view_tko which vary in behavior based on the view type and make use of the additional metadata. Due to the face that views are not a singular type, but a family of types each concrete view type has its own value representation.

Our first-order view type is Tensor View . A tensor view value is logically a tuple of (p t r , s h a p e → , s t r i d e s → , d i m g r o u p s →) , and load and store operate on the entire tensor view. The index space of this view is rank-0 (i.e., there is only a single element in the index space).

Our primary second-order (or "sub-view") view type is Partition View .

The partition view type Partition View is a subview type which represents a tensor view partitioned into tiles.

A partition view value is logically a tuple of (t e n s o r _ v i e w , t i l e _ s i z e →) , where load and store operate over tile values of the given size. The index space of this view is:

indexSpace[i]=tensorViewShape[dimGroups[i]]/tileSize[i]

For example a tensor view with shape (1024 , 1024) may be partitioned into a grid of 64 × 32 tiles, resulting in a partition view with shape (16 , 32) . See Type System for more details on the different types of views.

Note

As with tiles, the physical layout and memory representation of a view is not visible to the program and is not specified by the language semantics.

Warning

Reading or writing of bounds of any allocation is undefined behavior and bounds checking must be performed by the programmer if desired.

6.4 Tile Grid

During execution, the abstract machine instantiates a grid of tile blocks. A tile grid is grid of tile blocks arranged in a 1-, 2-, or 3-dimensional array.

Each position of the grid corresponds to a single independent tile kernel instance.

The abstract machine stores a sequence of tile blocks, T B , that are indexed by a grid of coordinates, g → .

Each tile block is assigned unique tile block id based on the grid size and the tile block's position within the grid.

The bijective mapping between 1-, 2-, or 3-dimensional grid coordinates and flattened tile block ids is computed as follows:

tile_grid(i→)={i0where grid=(x)i0$\cdot$x+i1where grid=(x,y)i0$\cdot$x+i1$\cdot$y+i2where grid=(x,y,z)where ⋯,in)and 0$\leq$ik<gridk for all k$\leq$3

6.5 Register File

The register file, R , maps named registers to values. Each assignment (i.e., SSA variable) in the tile function's body is assigned to a unique register which eventually holds the value of the operation's result.

Registers are local to a tile block and are not visible to other tile blocks. As stated previously, the memory representation of values in registers are not visible to the program and is not specified by the language semantics.

Values will only be fetched or persisted to global memory by memory operations (see Memory).

The register file is indexed by the tile block's coordinates, g → , and the register's name, r and produces a value v .

S.R(g→,r)=v

6.6 Global Memory

The global memory, M , is a mapping from addresses to scalar values.

The global memory is used to store the values of the tile block's global variables.

The heap is abstractly modeled as map from addresses to scalar values, not tile values.

This distinction is essential to describe the memory effect of tile operations as a sequence of individual scalar memory operations. A fine-grained model enables both reasoning about aggregate operations granularly as well as a straight forward denotation into the existing PTX memory model.

Note

Global memory **is** the same global device memory that is used by CUDA programs and described in the PTX Specification .

The specification elaborates the intricacies of the **Tile IR** memory model in Memory Model .

6.7 Tile Block

A tile block is a single thread of execution that is assigned a unique coordinate in the tile grid.

Abstractly its state consists of:

- The tile kernel under execution.
- A register file, R , that maps named registers to values.
- A statement under evaluation representing by an integer index into the sequence of SSA

statements in the tile function's body.

6.8 Execution Semantics

Tile IR program execution starts with a kernel launch. The kernel launch API is uniform for all CUDA kernels and is specified in the CUDA Runtime API .

6.8.1 Initialization

A launch of tile kernel initializes the abstract machine with:

- The module representing the complete program.
- A grid of tile blocks where each tile block is instantiated using the same tile kernel, begins at statement 0, assigned a unique grid coordinate, and assigned a unique empty register file.
- A reference to global memory, where its state is the state of global memory prior to the kernel launch.
- The set of pending memory operations is initialize to be empty.

6.8.2 Forward Progress

Execution proceeds with unspecified scheduling of tile blocks. Each tile block will be executed in some order which is non-deterministic and not specified by the language semantics. We guarantee forward progress of the execution that is all tile blocks will be guaranteed to eventually be scheduled for execution. It is possible that all tile blocks run completely in parallel, completely serially, or anything in between.

6.8.3 Tile Block Execution

Execution of a single tile block is isolated from other tile blocks. Tile blocks can only observe effects of other blocks via global memory which can be used to implement forms of cooperation or communication.

Function bodies are a series of static-single-assignment (SSA) statements which assign the result of each operations to a unique variable. Each variable is mapped to a register in the abstract machine's register file. A function body is executes statements sequentially, in order.

The compiler is free to reorder statements as long as there is no effect on the program visible effects or violated program semantics.

For more detailed example programs and explanations of their execution see Programming Model or Appendix .

The one unique semantic of **Tile IR** is the partitioning of memory operations into *program ordered* and *token ordered operations* . All memory operations produce their *result values* immediately but the order in which order these operations effect memory is more subtle.

For program ordered operations, the order between any pair memory operations acting on the same address is defined by the operation's position in program. Intuitively the effect of all prior memory operations on the same address will be visible to all subsequent memory operations on the same address.

In contrast, the order between any pair of token ordered operations is undefined, and has no relation to program order. The order of a pair of any two token ordered operations

(A and B) is only defined if established by a direct or transitive relationship between A 's output token and B 's input token.

This choice importantly allows a producer of **Tile IR** to induce different memory ordering semantics by inserting the appropriate memory ordering tokens.

For example starting with a single fresh token depending on by the first operation with the result token of the first operation being depended upon by the second operation and so on threading the tokens through each operation in program order.

Token threading like this establishes a ordering of the memory operations which is consistent with the same program with each token ordered operations being replaced by a program ordered memory operation.

For a detailed discussion of the memory model and memory operations see Memory Model .

6.8.4 Termination

A tile block will terminate when the tile block's function body reaches the final statement. Tile kernels must terminate with a return operation cuda_tile.return which signals the end of the execution.

7 Memory Model

The memory model defines the legal values that loads can return from memory.

This is not as straightforward as one might expect at first glance, to enable compiler and hardware optimizations, we allow the apparent re-ordering of instructions.

This memory model is derived from the PTX memory model, and synchronization primitives are deliberately similar.

7.1 Memory model operations

The memory model is built of relations between individual element accesses of tile operations, and restrictions on cycles of those relations.

Therefore, a **Tile IR** memory instruction generates one or more memory model operations.

In particular, tile loads, stores, and atomic updates generate one memory operation per element in the tile.

When expanding a tile operation into many memory model operations, one might ask what order the memory model operations happen in.

That is deliberately left unspecified for implementation flexibility.

To generalize the memory model over **Tile IR** operations, we use the terminology of *memory model operations* throughout the specification of the memory consistency model.

We enumerate which **Tile IR** operations expand into which memory model operations in the table below.

7.2 Scopes

Memory operations in **Tile IR** may have a scope.

Operations without a scope are called `weak` .

All memory operations specify a scope or `weak` .

Any scope other than weak requires a memory ordering to be set.

Scope	Description
`tile_block`	Tile block scope, for communication within a single tile block.
`device`	Device scope, for communication within the same GPU.
`sys`	System scope, for communication anywhere in the system.

Weak operations cannot be used to communicate through memory between threads, or between fragments of the same tile block which are not ordered by token order.

The compiler may assume that tiles accessed with `weak` are not concurrently accessed by any other thread.

Note

Tile Block scope is needed when building communicating algorithms where there is communication within a single tile block.

This is necessary when communicating memory through memory between operations which are not ordered by token order, or when storing to a tile with internal aliasing.

7.3 Memory ordering

Memory operations have a memory ordering parameter which controls how that operation can be used for synchronization.

Synchronization through memory is a two-party process, which requires a releaser and an acquirer observing the same location.

When a pair of memory accesses synchronize through memory it establishes a *happens before* relationship.

See the happens before definition below.

Any ordering other than `weak` requires a scope to be set.

`weak`: No concurrent accesses to the source/destination location.

`relaxed`: There may be concurrent access to the location, but this access does not establish a happens-before relationship.

`release`: There may be concurrent access to the location. If this release is observed with an acquire

operation, then happens before is established.
`acquire`: There may be concurrent accesses to the location. If this acquire observes a release operation, then happens before is established.
`acq_rel`: There may be concurrent accesses to the location. This has the effect of both a release and acquire operation.

7.4 Moral Strength

Two accesses to the same location are morally strong if the operations are related in *restricted program order* , or each operation specifies a scope which includes the tile block executing the other operation.

7.5 Tokens and token order

In **Tile IR** we have explicit annotation of dependencies between loads and stores for the token-ordered operations. **Tile IR** produces wide loads and stores of whole tiles of data, making efficient use of various resources of the GPU in parallel.

We provide token ordered operations to explicitly inform the **Tile IR** toolchain that two operations may happen in parallel, and will not interfere with each other.

There is a family of memory operations called token ordered operations which produce and consume tokens.

Tokens are abstract values in the **Tile IR** language for building dependencies between memory operations within the same tile block thread.

They have no concrete representation at runtime, cannot be compared, computed upon, or stored/loaded to/from memory.

Program dependencies (i.e. dependencies apparent from control flow, data dependency, or address dependency) **do not** provide ordering between two memory operations.

Tokens must be used, even where the token ordering appears redundant with program dependencies.

Program dependencies may be optimized away by the **Tile IR** toolchain, whereas token dependencies are not.

7.6 Base relations

Coherence order

There exists a partial transitive order that relates overlapping write operations, determined at runtime, called *coherence order* .

Two overlapping write operations are related in *coherence order* if they are *morally strong* or if they are related in *happens before* order.

Program order

A memory operation A is program order before a memory operation B if the instruction that gave rise to A is before the instruction that gave rise to B in the program source.

Waits-for order

An operation A *waits-for* an operation B if:

- an instruction I A gave rise to A , I A produced a token t , an instruction I B gave rise to B and I B depends upon the token t ; or
- there is some operation C such that A *waits-for* C and C *waits-for* B .

Reads from

An read operation R *reads-from* a write operation W when R and W access the same location and R reads the value written by W .

Read-modify-write order

Read-modify-write atomics generate a pair of memory operations for each element location within a tile.

Each pair of memory operations is related by *read-modify-write order* .

Atomicity

When an atomic operation A and a write W overlap and are *morally strong* , then the following two communications cannot both exist in the same execution:

- A reads from a write W ' that precedes W in *coherence order* .

- A follows W in *coherence order* .

7.7 No-thin-air

It is not practical to specify a "no thin air" axiom without preventing useful compiler optimizations.

We therefore say informally that the implementation will not provide values out of thin air to satisfy program executions.

7.8 Data Races

Two accesses are said to *conflict* when they access the same location and at least one of them is a write.

Two conflicting memory accesses are said to be in a *data race* if they are not related in happens before and are they are not morally strong.

Programs with data races have **undefined behaviour** .

7.9 PTX interoperability

The axioms and relations of the **Tile IR** memory model are intended to be a strict weakening of the PTX memory model.

The **Tile IR** memory model is designed to allow communication with PTX threads.

Release and acquire patterns in **Tile IR** will match up with acquire and release patterns in PTX to build PTX *causality* and **Tile IR** *happens before* .

The same is true for data races, data races between accesses in a **Tile IR** program and a PTX program will result in undefined behavior as if the data race were all in **Tile IR** .

7.10 Hazards and intuition in the Tile IR Memory Model

7.10.1 Token ordered operations reading from the future

Store buffering and load buffering behavior are visible when using token ordered operations without token constraints.

7.10.2 Race hazards within a single tile block thread

The definition of data race relies on two operations being in happens before order.

In many programming languages this is always true within a single thread, but this is not the case in **Tile IR** .

In **Tile IR** you can construct a data race when there are two accesses to the same location within a single tile block store (because of internal overlap in the destination tile), or with two accesses within a tile block thread to the same location which are not ordered by token order.

This motivates the need for a `tile_block` scope, and is a hazard to be aware of in the language.

7.10.3 Some intuition on how to use token ordering

When you have memory accesses to tiles with no overlap between them, it is generally safe to not order these with respect to each other in token order.

When you use a release operation, you need to token-order all memory events that must stay before the release to the release itself.

Similarly, when an acquire operation is used, all memory events which must remain after the acquire need to be token ordered after the acquire operation itself.

To reiterate: a user of **Tile IR** cannot rely on program dependencies of any form other than token dependencies to enforce ordering within a Tile Block Thread.

The **Tile IR** toolchain can remove dependencies through any possible complex reasoning, breaking dependencies which appear to be in the program syntax.

8 Operations

This section describes a complete and categorized list of all **Tile IR** instructions names, signatures, and semantics.

8.1 Meta Types

Operations have arguments which are **Tile IR** values with **Tile IR** types but many operations have immediate or static arguments which correspond to attributes in the MLIR dialect. These **meta types** are not representable in the **Tile IR** type system but are used to construct **Tile IR** programs and only present at compile time. Operations in the specification are described abstractly in both the **Tile IR** IR and bytecode independent of the MLIR or bytecode encoding. For each of these types we provide a definition of them below and link to them from each operation.

Note

The convention is that the meta types are capitalized and **Tile IR** types are snake cased.

The convention is that the meta types are capitalized and the native **Tile IR** types are camel cased are snake cased.

8.1.1 Symbol

`Symbol` a symbol in the program, begins with `@` and uniquely identifies a symbol in the program.

8.1.2 Flag

`Flag` a boolean value that can be used to control the behavior of an operation.

8.1.3 Token

Token represents a memory ordering token that can be used to control the ordering of memory operations.

8.1.4 Variadic

`Variadic` represents an argument which can accept a statically sized, but variable, number of arguments.

8.1.5 Any

`Any` represents a value of any valid **Tile IR** type.

8.1.6 Name

`Name` represents a name in the program, begins with `#` and uniquely identifies a name in the program.

8.1.7 Type

`Type` represents a **Tile IR** type and are attached as attributes to operations which define IR items.

8.1.8 Array

`Array` represents a statically sized array of values that can be passed to attributes.

8.1.9 String

`String` represents a string value that can be passed to attributes.

8.1.10 bool

`bool` represents a boolean value that can be passed to attributes.

8.1.11 DenseConstant

`DenseConstant` represents a dense constant value that can be passed to attributes.

8.1.12 view_type

`view_type` represents a type which implements the view interface, currently this is only implemented by partition_view but will have new implementers in future releases.

8.2 Operation Design Considerations

The design of **Tile IR** has a set of design considerations that apply to all operations in the dialect this section introduces some of the common design considerations that apply to all operations, or to classes of operations generically.

8.2.1 Explicit Broadcast

There are no implicit broadcast performed by operations in the **Tile IR** dialect all operations that require operands of the same shape must be explicitly broadcasted.

For example to use the cuda_tile.offset operation to add an offset tile to a pointer, the pointer and offset must be reshaped or broadcasted to have the same shape using the cuda_tile.reshape or cuda_tile.broadcast operations.

8.2.2 Distinct Floating-Point and Integer Operations

Numeric ooerations are split across integer and floating-point types due to differences in flags such as rounding modes, `NaN` handling, and fast math.

For example, the cuda_tile.addf operation supports a rounding attribute, but the addi operation does not.

8.2.3 Explicit Overflow Annotations

Some operations such as cuda_tile.addi support an explicit overflow annotation that expresses the expected overflow behavior of the operation.

These attributes serve as assumptions that an implementation may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

We recommend that generators of **Tile IR** programs utilize these annotations to help the implementation reason about the overflow behavior of the operation, enabling extra optimization opportunities.

8.3 Core

8.3.1 cuda_tile.broadcast

Broadcast tile to new shape

```
cuda_tile.broadcast %source
```

Parameters

- **source** (tile) - The tile to broadcast. 13.1
- **result** (tile) - The broadcasted tile. 13.1

The `broadcast` operation expands each unary (`1`) dimension in the input tile by duplicating the data along that dimension.

Expansion happens only for dimensions of size one that are stretched or “copied” to match the size of the dimension implied by the result type of the operation. The operation does not change the rank of the source tile. Any change to the rank of the source tile must be made using reshape-like operations before broadcasting.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same element type (tile).
- `source` and `result` must have the same rank.

8.3.2 cuda_tile.cat

Concatenate tiles along specified dimension

```
cuda_tile.cat %lhs %rhs %dim
```

Parameters

- **lhs** (tile) - The left hand side operand. 13.1
- **rhs** (tile) - The right hand side operand. 13.1
- **dim** (i64) - The dimension along which to concatenate. 13.1
- **result** (tile) - The concatenated result tile. 13.1

The `cat` operation concatenates the two input tiles. The input tiles must have the same shape in all but the concatenating dimension. Concatenation happens along the dimension specified by the the attribute `dim` the resulting dimension is the sum of the the two input tiles concatenating dimension.

cat(x,y,dimcat)[i→]={x[...,icat,...,in]if icat<dcaty[...,icat-dcat,...,in]if icat$\geq$dcat

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same rank.
- `lhs` , `rhs` and `result` must have the same element type (tile).

Examples

```
// A valid invocation of cat.
%0 = cat %arg0, %arg1 dim = 1
  : tile<2x4xf32>, tile<2x4xf32> -> tile<2x8xf32>
// >>> %arg0 = tile([[ A, B, C ],
//                   [ D, E, F ]])
// >>> %arg1 = tile([[ 1, 2, 3 ],
//                   [ 4, 5, 6 ]])
// >>> %0 = tile([[ A, B, C, 1, 2, 3 ],
//                [ D, E, F, 4, 5, 6 ]])
// A valid invocation of cat.
%1 = cat %arg0, %arg1 dim = 0
  : tile<2x4xf32>, tile<2x4xf32> -> tile<4x4xf32>
// >>> %arg0 = tile([[ A, B, C ],
//                   [ D, E, F ]])
```

```
//
// >>> %arg1 = tile([[ 1, 2, 3 ],
//                   [ 4, 5, 6 ]])
//
// >>> %1 = tile([[ A, B, C ],
//                [ D, E, F ],
//                [ 1, 2, 3 ],
//                [ 4, 5, 6 ]])
```

See cuda_tile.cat_0 for the full example listing.

8.3.3 cuda_tile.cmpf

Element-wise floating-point comparison

```
cuda_tile.cmpf %comparison_predicate %comparison_ordering %lhs %rhs
```

Parameters

- **comparison_predicate** (ComparisonPredicate) - The comparison predicate. 13.1
- **comparison_ordering** (ComparisonOrdering) - The comparison ordering. 13.1
- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **result** (tile < i1 >) - The result of the comparison. 13.1

The `cmpf` operation is a generic comparison for float-like types. The operands must have the same shape and type, and this type must be a float type.

The result is `1` if the comparison is true and `0` otherwise. The comparison is performed element-wise and the element of the result indicates whether the comparison is true for the operand elements with the same indices as those of the result.

cmpf(x,y,pred)i={1if xi pred yi0otherwise

The `comparison_predicate` attribute specifies the kind of comparison to be performed.

- `equal` - Equal comparison.
- `not_equal` - Not equal comparison.
- `less_than` - Less than comparison.
- `less_than_or_equal` - Less than or equal comparison.
- `greater_than` - Greater than comparison.
- `greater_than_or_equal` - Greater than or equal comparison.

The `comparison_ordering` attribute specifies the kind of ordering to be performed in the comparison operation.

- `unordered` - Unordered comparison.
- `ordered` - Ordered comparison.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` and `rhs` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- Result type has i1 element type and same shape as operands
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%lhs0 = constant <f16: 0.0> : tile<f16>
%rhs0 = constant <f16: 0.0> : tile<f16>
// Custom form of scalar "ordered equal" comparison.
%x0 = cmpf equal ordered %lhs0, %rhs0 : tile<f16> -> tile<i1>
%lhs1 = constant <f16: 0.0> : tile<2x2xf16>
%rhs1 = constant <f16: 0.0> : tile<2x2xf16>
// Custom form of scalar "unordered less than" comparison.
%x2 = cmpf less_than unordered %lhs1, %rhs1 : tile<2x2xf16> -> tile<2x2xi1>
%lhs2 = constant <f64: 0.0> : tile<2x2xf64>
%rhs2 = constant <f64: 0.0> : tile<2x2xf64>
```

See cuda_tile.cmpf_0 for the full example listing.

8.3.4 cuda_tile.cmpi

Element-wise integer comparison

```
cuda_tile.cmpi %comparison_predicate %lhs %rhs %signedness
```

Parameters

- **comparison_predicate** (ComparisonPredicate) - The comparison predicate. 13.1
- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **result** (tile < i1 >) - The result of the comparison. 13.1

The `cmpi` operation is a generic comparison for integer-like types. The operands must have the same shape and type, and this type must be an integer type.

The result type has i1 element type and the same shape as the operands.

The result is `1` if the comparison is true and `0` otherwise. The comparison is performed element-wise and the element of the result indicates whether the comparison is true for the operand elements with the same indices as those of the result.

cmpi(x,y,pred)i={1if xi pred yi0otherwise

The `comparison_predicate` attribute specifies the kind of comparison to be performed.

- `equal` - Equal comparison.
- `not_equal` - Not equal comparison.
- `less_than` - Less than comparison.

- `less_than_or_equal` - Less than or equal comparison.
- `greater_than` - Greater than comparison.
- `greater_than_or_equal` - Greater than or equal comparison.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` and `rhs` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- Result type has i1 element type and same shape as operands
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%lhs0 = constant <i16: 0> : tile<i16>
%rhs0 = constant <i16: 0> : tile<i16>
// Scalar "signed less than" comparison.
%x0 = cmpi less_than %lhs0, %rhs0, signed : tile<i16> -> tile<i1>
%lhs1 = constant <i64: 0> : tile<2x2xi64>
%rhs1 = constant <i64: 0> : tile<2x2xi64>
// Tile equality comparison.
// There is no difference between "signed" and "unsigned" when performing
    equality and inequality comparison.
%x1 = cmpi equal %lhs1, %rhs1, signed : tile<2x2xi64> -> tile<2x2xi1>
```

See cuda_tile.cmpi_0 for the full example listing.

8.3.5 cuda_tile.constant

Construct a constant tile

```
cuda_tile.constant %value
```

Parameters

- **value** (DenseConstant) - The constant value to create. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 | f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The constant tile. 13.1

The `constant` operation creates a tile initialized by `$value` .

There are two main forms of using the operation:

- One where the value is a single constant specified by `<D: c>` and the tile is filled with identical values for all elements with element type `D` .
- One where the value is a list of constants specified by `dense<D: [c0, c1, c2, ...]>` and the constant value's shape must match the tile's shape with the element type `D` .

The annotated type of the tile constrains its rank, shape, and element type.

Constraints

- The operation has no operands and may be constant folded.
- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `value` and `result` must have the same shape and element type (DenseConstant).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%c0 = constant <i32: 0> : tile<i32>
%c1 = constant <i64: 1> : tile<i64>
%c2 = constant <i32: [0, 1, 2, 3]> : tile<4xi32>
%c3 = constant <f32: 0.0> : tile<2x4xf32>
%c4 = constant <f64: [0.0, 1.0, 2.0, 3.0]> : tile<4xf64>
```

See cuda_tile.constant_0 for the full example listing.

8.3.6 cuda_tile.entry

Define a tile kernel

```
cuda_tile.entry %sym_name %function_type %arg_attrs %res_attrs %
    optimization_hints
```

Parameters

- **sym_name** (Symbol) - The name of the function. 13.1
- **function_type** (Type) - The type of the function. 13.1
- **arg_attrs** (Attributes) - The argument attributes of the function: none of these are supported by TileIR at the moment. 13.1
- **res_attrs** (Attributes) - The result attributes of the function: none of these are supported by TileIR at the moment. 13.1
- **optimization_hints** (OptimizationHints) - Compiler architecture-specific optimization hints 13.1

No results.

Description The `entry` operation defines a tile kernel; a kernel is a function that can serve as the program entry point. It has a unique name per-module. A kernel can not return any value. It must be launched from the host side using `cuLaunchKernel` or similar CUDA runtime API functions.

Tile kernels require that the user specifies the 3-d grid dimensions at launch which defines the number of tile blocks (or kernel instances) that will execute the kernel in parallel.

For detailed semantics of tile kernels see Tile Kernel .

The `optimization_hints` attribute provides architecture-specific compiler hints in the form of nested dictionaries.

The hints are specified for each architecture (e.g., `sm_100` , `sm_120`) and for each architecture the user can specify specific hints for each operation.

- `num_cta_in_cga` - suggest the number of CTAs in a CGA (which must be the power of 2 less than or equal to 16) for cuda_tile.entry .
- `allow_tma` - suggest whether to use TMA for cuda_tile.load_view_tko and cuda_tile.store_view_tko .
- `latency` - latency hint for cuda_tile.load_view_tko and cuda_tile.store_view_tko .

For example they can be annotated as:

```
optimization_hints=<
  sm_100 = {num_cta_in_cga = 8},
  sm_120 = {num_cta_in_cga = 16}
>
```

Constraints

- The operation must be a symbol in the global symbol table.
- The operation must implement callable target interface.
- The operation must implement function-like behavior interface.
- The region must not capture SSA values defined above the operation.
- The operation must provide custom parsing and printing methods.
- Each provided region must contain exactly one block.

8.3.7 cuda_tile.extract

Extract a subtile from a tile

```
cuda_tile.extract %source %indices
```

Parameters

- **source** (tile) - The source tile to extract from. 13.1
- **indices** (Variadic < tile < i32 > >) - The indices of the slice to extract. 13.1
- **result** (tile) - The extracted subtile. 13.1

The `extract` operation extracts a subtile from the given source tile.

The shape of the result tile must divide the shape of the source tile evenly e.g., `tile<4xf32>` is a valid extraction from `tile<8xf32>` , but `tile<3xf32>` is not.

The `$indices` indicate the number of the slice to extract, but *importantly* not the offsets used to construct the subtile for extraction. The semantics of extract means that only full size slices can be extracted.

Slices of a source tile with the same shape are non-overlapping by definition for unique indices.

The `indices` operands are interpreted as unsigned integers.

Warning

If the `indices` specify a non-existent (i.e., out-of-bounds) slice, the behavior of the operation is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same rank.

Examples

```
// Extract a subtile from %t at dim_0 = [4;8) and dim_1 = [4;6).
%c1 = constant <i32: 1> : tile<i32>
%c2 = constant <i32: 2> : tile<i32>
%t = constant <f32: 0.0> : tile<32x8xf32>
// Valid indices are: [ {0, 1, 2, 3, 4, 5, 6, 7}, {0, 1, 2, 3} ]
%0 = extract %t[%c1, %c2]
    : tile<32x8xf32> -> tile<4x2xf32>
```

See cuda_tile.extract_0 for the full example listing.

8.3.8 cuda_tile.get_global

Get a pointer to a global variable

```
cuda_tile.get_global %name
```

Parameters

- **name** (Symbol) - The name of the global variable. 13.1
- **result** (tile < ptr >) - The result of the get_global operation. 13.1

The `get_global` operation returns a pointer to the specified `global` variable. A global variable is a form of static global memory allocation that can be declared using the cuda_tile.global operation.

The element type of the returned pointer will be of the same type as the element type of the declared global variable.

For detailed semantics of global variables see Global Variable .

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

Examples

```
global @val <f32: [0.1, 0.2, 0.3, 0.4]> : tile<4xf32>
entry @example() {
  %ptr = get_global @val : tile<ptr<f32>>
  return
}
```

See cuda_tile.get_global_0 for the full example listing.

8.3.9 cuda_tile.get_num_tile_blocks

Get total number of tile blocks

```
cuda_tile.get_num_tile_blocks
```

Parameters No parameters.

Results

- **gridSize_x** (tile < i32 >) - The number of tile blocks in dimension `x` . 13.1
- **gridSize_y** (tile < i32 >) - The number of tile blocks in dimension `y` . 13.1
- **gridSize_z** (tile < i32 >) - The number of tile blocks in dimension `z` . 13.1

The `get_num_tile_blocks` operation queries the total number of tile blocks in the form of a 3-tuple specifying the extent of each grid dimension.

A tile `id` is a coordinate in 3-space and therefore the must also be a 3-tuple containing the extent of each dimension: `x` , `y` and `z` .

When launching 1- or 2-dimensional grids, the unspecified dimensions will have a cardinality of 1.

For example if the grid used to launch the kernel is `(1024, 1024)` then the result of this operation will be `(1024, 1024, 1)` .

Note

Grid Dimension Limitation : Grid dimensions are limited to 2^24-1 (16,777,215)

per axis. Larger dimensions may result in incorrect tile block ID calculations. Use multiple kernel launches for larger workloads.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
entry @example() {
  %x, %y, %z = get_num_tile_blocks : tile<i32>
  // print "x: %, y: %, z: %\n", %x, %y, %z : tile<i32>, tile<i32>, tile<
      i32>
}
```

See cuda_tile.get_num_tile_blocks_0 for the full example listing.

8.3.10 cuda_tile.get_tile_block_id

Get the currently executing tile block coordinates

```
cuda_tile.get_tile_block_id
```

Parameters No parameters.

Results

- **blockId_x** (tile < i32 >) - The tile block ID for dimension `x` . 13.1
- **blockId_y** (tile < i32 >) - The tile block ID for dimension `y` . 13.1
- **blockId_z** (tile < i32 >) - The tile block ID for dimension `z` . 13.1

`get_tile_block_id` returns a 3-d tile block coordinates (or ID) of the currently executing tile block.

A tile ID has three dimensions: `x` , `y` , and `z` . This operation returns all three of them simultaneously. The value of each dimension returned by this operation is between `0` (including) and the value returned by `get_num_tile_blocks` for the respective axis (excluding), represented by the inclusive interval `[0, get_num_tile_blocks(dim) - 1]` . Grid dimensions unspecified at kernel launch (i.e., a 1-d or 2-d grid) will always be `0` for all tile blocks.

Note

Grid Dimension Limitation : Grid dimensions are limited to 2^24-1 (16,777,215)

per axis. Larger dimensions may result in incorrect tile block ID calculations. Use multiple kernel launches for larger workloads.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- The operation's result type may be inferred from its operands and attributes.

8.3.11 cuda_tile.global

Allocate static global memory

```
cuda_tile.global %sym_name %value %alignment
```

Parameters

- **sym_name** (Symbol) - The name of the global variable. 13.1
- **value** (DenseConstant) - The value to initialize the allocation with. 13.1
- **alignment** (i64) - The alignment of the buffer. 13.1

No results.

Description The `global` operation statically allocates a mutable 1-dimensional location in global memory and initializes it using `value` . The initialization of the allocation is performed at CUDA module load time. The lifetime of the allocation is the same as the lifetime of the module.

The allocation may be read or written to by first using cuda_tile.get_global to obtain a pointer to the the memory and then read using cuda_tile.load_ptr_tko or written to using cuda_tile.store_ptr_tko .

The initial values are stored in memory in linear order, so the pointer returned by cuda_tile.get_global points to the first element, and offsetting the pointer by x would allow to load element at position x .

`global` operations must be directly nested within the **Tile IR** module. They cannot be defined inside functions.

As globals are defined at the module scope their names are globally unique symbols and must not collide with any other symbol in the module.

For more detailed semantics of global variables see Global Variable .

Constraints

- The operation must be a symbol in the global symbol table.

Examples

```
global @val alignment = 128 <f32: [0.1, 0.2, 0.3, 0.4]> : tile<4xf32>
entry @example() {}
```

See cuda_tile.global_0 for the full example listing.

8.3.12 cuda_tile.iota

Generate a 1-d tile range from 0 to n-1

```
cuda_tile.iota
```

Parameters No parameters.

Results

- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The result of the iota operation. 13.1

The `iota` operation generates a 1-d tile with a sequence of integer values. The starting value is `0` and the stride is `1` . If the shape of the result tile is `(n)` , then the generated values are `[0, n - 1]` .

iota(n)i=ifor i$\in$[0,n-1]

The result values should be interpreted as unsigned integers.

Note

The number of elements in the result tile must not exceed the maximum value that the element type can express.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.3.13 cuda_tile.mmaf

Floating-point matrix-multiply-accumulate

```
cuda_tile.mmaf %lhs %rhs %acc
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 | tile < tf32 > >) - The left hand side matrix operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 | tile < tf32 > >) - The right hand side matrix operand. 13.1
- **acc** (tile < f16 | f32 | f64 >) - The accumulator matrix operand. 13.1

- **result** (tile < f16 | f32 | f64 >) - The result matrix after multiplication and accumulation. 13.1

The `mmaf` operation implements an MMA (matrix-multiply-accumulate) operation for floating-point tiles.

It performs matrix multiplication on the floating-point tiles `lhs` and `rhs` , then adds the tile `acc` to the result. `lhs` , `rhs` , and `acc` must be 2D tiles or 3D tiles. The latter case indicates a batched matrix multiplication.

mmaf(A,B,C)ij=$\sum$k=0K-1Aik×Bkj+Cij

The types of all operands must be a supported combination (see mmaf Supported Data Types).

Shapes must be a valid matrix multiplication configuration. Unbatched (2D)

MMA expects the operands `lhs` , `rhs` , and `acc` to have shapes `M x K` , `K x N` , and `M x N` (respectively). Batched (3D) MMA expects the operands to have shapes `B x M x K` , `B x K x N` , and `B x M x N` (respectively).

The table below shows the supported output types for each possible `mmaf` input type. Input operands must be of the same element type.

Input Type	Supported Output Types
`f8E4M3FN`	`f16` or `f32`
`f8E5M2`	`f16` or `f32`
`f16`	`f16` or `f32`
`bf16`	`f32`
`tf32`	`f32`
`f32`	`f32`
`f64`	`f64`

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `acc` and `result` must have the same shape and element type (tile < f16 | f32 | f64 >).
- `lhs` and `rhs` must have the same element type (tile < f16 | bf16 | f32 | f64 | tile < tf32 > >).
- `lhs` , `rhs` and `acc` must have the same rank.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%lhs0 = constant <f16: 0.0> : tile<4x8xf16>
%rhs0 = constant <f16: 0.0> : tile<8x2xf16>
%acc0 = constant <f32: 0.0> : tile<4x2xf32>
%0 = mmaf %lhs0, %rhs0, %acc0
    : tile<4x8xf16>, tile<8x2xf16>,
      tile<4x2xf32>
%lhs1 = constant <f16: 0.0> : tile<2x4x8xf16>
%rhs1 = constant <f16: 0.0> : tile<2x8x2xf16>
%acc1 = constant <f32: 0.0> : tile<2x4x2xf32>
%1 = mmaf %lhs1, %rhs1, %acc1
    : tile<2x4x8xf16>, tile<2x8x2xf16>,
      tile<2x4x2xf32>
```

See cuda_tile.mmaf_0 for the full example listing.

8.3.14 cuda_tile.mmai

Integer matrix-multiply-accumulate

```
cuda_tile.mmai %lhs %rhs %acc %signedness_lhs %signedness_rhs
```

Parameters

- **lhs** (tile < i8 >) - The left hand side matrix operand. 13.1
- **rhs** (tile < i8 >) - The right hand side matrix operand. 13.1
- **acc** (tile < i32 >) - The accumulator matrix operand. 13.1
- **signedness_lhs** (Signedness) - The signedness of the `lhs` operand. 13.1
- **signedness_rhs** (Signedness) - The signedness of the `rhs` operand. 13.1
- **result** (tile < i32 >) - The result matrix after multiplication and accumulation. 13.1

The `mmai` operation implements an MMA (matrix-multiply-accumulate) operation for integer tiles.

It performs matrix multiplication on the integer tiles `lhs` and `rhs` , then adds the tile `acc` to the result. `lhs` , `rhs` , and `acc` must be 2D tiles or 3D tiles. The latter case indicates a batched matrix multiplication.

mmai(A,B,C)ij=$\sum$k=0K-1Aik×Bkj+Cij

Input tiles `lhs` and `rhs` must be of integer type `i8` . The signedness of `lhs` and `rhs` are specified separately by the `signedness_lhs` and `signedness_rhs` attributes, respectively. The accumulator tile `acc` must be of type `i32` and is always interpreted as signed. The output tile `result` is of type `i32` and is always interpreted as signed.

Shapes must be a valid matrix multiplication configuration. Unbatched (2D)

MMA expects the operands `lhs` , `rhs` , and `acc` to have shapes `M x K` , `K x N` , and `M x N` (respectively). Batched (3D) MMA expects the operands to have shapes `B x M x K` , `B x K x N` , and `B x M x N` (respectively).

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `acc` and `result` must have the same shape and element type (tile < i32 >).
- `lhs` and `rhs` must have the same element type (tile < i8 >).
- `lhs` , `rhs` and `acc` must have the same rank.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%lhs0 = cuda_tile.constant <i8: 0> : tile<4x8xi8>
%rhs0 = cuda_tile.constant <i8: 0> : tile<8x2xi8>
%acc0 = cuda_tile.constant <i32: 0> : tile<4x2xi32>
%0 = mmai %lhs0, %rhs0, %acc0 signed signed
    : tile<4x8xi8>, tile<8x2xi8>,
      tile<4x2xi32>
%lhs1 = cuda_tile.constant <i8: 0> : tile<2x4x8xi8>
%rhs1 = cuda_tile.constant <i8: 0> : tile<2x8x2xi8>
%acc1 = cuda_tile.constant <i32: 0> : tile<2x4x2xi32>
```

```
%1 = mmai %lhs1, %rhs1, %acc1 unsigned unsigned
    : tile<2x4x8xi8>, tile<2x8x2xi8>,
      tile<2x4x2xi32>
```

See cuda_tile.mmai_0 for the full example listing.

8.3.15 cuda_tile.module

Top-level module containing a series of defined items.

```
cuda_tile.module %sym_name
```

Parameters

- **sym_name** (Symbol) - The name of the module. 13.1

No results.

Description A `module` operation represents a single compilation unit and contains zero or more items (global variables, functions, or kernels).

For detailed description of the semantics of modules, and the full definition of each item type see Modules .

The `module` operation is the top-level operation in a **Tile IR** module and must contain only **Tile IR** operations and no other dialects.

Constraints

- The region must not capture SSA values defined above the operation.
- The operation must provide custom parsing and printing methods.
- All regions must have zero arguments.
- Each provided region must contain exactly one block.
- The operation must define a symbol scope.
- The region must not require explicit terminator operations.
- The operation must specify whether regions are SSACFG or Graph kind.
- The operation must contain only dataflow graph regions.

8.3.16 cuda_tile.offset

Offsets a tile of pointers

```
cuda_tile.offset %ptr %offset
```

Parameters

- **ptr** (ptr) - The base pointer tile to advance. 13.1
- **offset** (tile < i1 | i8 | i16 | i32 | i64 >) - The offset tile to add to the pointer. 13.1
- **result** (ptr) - The resulting pointer tile after advancement. 13.1

`offset` advances a tile of pointers. It takes `ptr` as base and `offset` as increment, and performs element-wise addition of `ptr` by `offset` :

offset(ptr,offset)i=ptri+offseti×bitwidth

```
result[i,j] = ptr[i,j] + offset[i,j] * bitwidth
```

`ptr` is interpreted as an unsigned integer. `offset` is interpreted as a signed integer. `bitwidth` is the storage bitwidth of the pointee type. The multiplication must not overflow (wrap-around) in a signed sense. The addition must not overflow (wrap-around) in an unsigned sense. In case of an overflow, the result is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- The operation must apply element-wise to its operands.
- `ptr` , `offset` and `result` must have the same shape.
- `result` and `ptr` must have the same shape and element type (ptr).
- The operation's result type may be inferred from its operands and attributes.

8.3.17 cuda_tile.pack

Pack a tile into a byte array

```
cuda_tile.pack %source
```

Parameters

- **source** (tile < i1 | i8 | i16 | i32 | i64 | f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The input tile. 13.2
- **result** (tile < i8 >) - The packed tile. 13.2

The `pack` operation takes a rank-1 numeric tile and produces a rank-1 `tile<i8>` .

Similar to `bitcast` , underlying bit-values are not changed. However, `pack` does not operate element-twise, instead reinterpreting the entire tile as a byte array.

Input and output tiles must be rank-1 to eliminate packing ambiguity. The size of the output tile must match the number of bytes in the input tile.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same rank.

Examples

```
%arg0 = constant <f16: 0.0> : tile<64xf16>
%0 = pack %arg0 : tile<64xf16> -> tile<128xi8>
```

See cuda_tile.pack_0 for the full example listing.

```
%arg0 = constant <f4E2M1FN: 0.0> : tile<64xf4E2M1FN>
%0 = pack %arg0 : tile<64xf4E2M1FN> -> tile<32xi8>
```

See cuda_tile.pack_1 for the full example listing.

8.3.18 cuda_tile.permute

Permute tile dimensions

```
cuda_tile.permute %source %permutation
```

Parameters

- **source** (tile) - The input tile. 13.1
- **permutation** (Array < i32 >) - The permutation of the dimensions. 13.1
- **result** (tile) - The permuted tile. 13.1

Permute the dimensions of the input tile `source` according to the `permutation` array.

The `permutation` array is a list of integers that specify the new order of the dimensions.

For example, if the input tile has shape `[2, 4, 8]` , and the permutation is `[2, 0, 1]` , the output tile will have shape `[8, 2, 4]` .

This operation logically is a change in the indexing of the tile.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same element type (tile).
- `source` and `result` must have the same rank.

Examples

```
%arg0 = constant <f16: 0.0> : tile<2x4x8xf16>
%0 = permute %arg0 [2, 0, 1] : tile<2x4x8xf16> -> tile<8x2x4xf16>
```

See cuda_tile.permute_0 for the full example listing.

8.3.19 cuda_tile.reduce

Variadic tile reduction across dimensions

```
cuda_tile.reduce %operands %dim %identities
```

Parameters

- **operands** (Variadic < tile >) - The set of tiles to reduce. 13.1
- **dim** (i32) - The index of the dimension to perform reduction on. 13.1
- **identities** (Array) - The reduction identities for each operand. 13.1
- **results** (Variadic < tile >) - The set of reduced tiles. 13.1

The `reduce` operation applies a custom reduction function along a specified dimension of one or more input tiles, producing the same number of output tiles.

The reduction function must be an associative operation defined within the `reduce` operation's region. A single reduction operation can reduce over any number of input tiles in parallel, producing a reduced output tile for each.

All input tiles must have the same shape. The output tiles will have a matching shape in every dimension except the one being reduced, which is removed.

For each input tile, a constant identity value must be provided that matches the element type of the input tile. Identity `i` of `identities` corresponds to input tile `i` of `operands` . The correct identity value is a property of the reduction function in the `body` . (For example, if the reduction function performs `min` , the identity is `+inf` , while if the reduction function performs a `sum` , the identity is `0` .)

The reduction function must expect `2N` arguments, where `N` is the number of input tiles.

Each pair of reduction arguments `2i` and `2i+1` will correspond to the `i` -th input tile.

The first argument of each pair is an element of the input tile; the second is the accumulator from all prior reductions along the specified dimension. This second value might be input element, the identity value, or the result of a previous reduction iteration. The reduction function should yield the new accumulator value for each input tile.

Note

There are no guarantees on the order of element reduction along the specified dimension.

However, the result is deterministic across different runs of the same kernel on the same device.

Constraints

- The operation must provide custom parsing and printing methods.
- The operation only has an effect if and only if it the region's operation have an effect.
- All operands must have identical shapes.
- Each provided region must contain exactly one block.

Examples

```
%input = constant <f32: 0.0> : tile<8xf32>
%0 = reduce %input dim=0 identities=[0.000000e+0 : f32] : tile<8xf32> ->
    tile<f32>
  (%input_arg: tile<2xf32>, %input_accum: tile<f32>) {
    %add_result = addf %input_arg, %input_accum : tile<f32>
    yield %add_result : tile<f32>
  }
```

See cuda_tile.reduce_0 for the full example listing.

```
%input = constant <f32: 0.0> : tile<8x64xf32>
%0 = reduce %input dim=0 identities=[0.000000e+0 : f32] : tile<8x64xf32> -> tile
    <8xf32>
  (%input_arg: tile<f32>, %input_accum: tile<f32>) {
    %add_result = addf %input_arg, %input_accum : tile<f32>
    yield %add_result : tile<f32>
  }
```

See cuda_tile.reduce_1 for the full example listing.

8.3.20 cuda_tile.reshape

Reshape tile dimensions

```
cuda_tile.reshape %source
```

Parameters

- **source** (tile) - The source tile to reshape. 13.1
- **result** (tile) - The reshaped tile. 13.1

The `reshape` operation changes the shape of the `source` operand. `reshape` is only a change in the indexing of the tile. The number of elements and element type must remain unchanged.

0-d tiles (i.e., scalars) contain precisely one element and thus are the one exception where a 0-d tile can be reshaped to shape where the `size(shape) == 1` .

Conceptually reshaping a tile is equivalent to first creating a 1-d tile from the data of the source assuming a row-major layout and then converting the 1-d tile into the new shape in a row-major layout.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same element type (tile).

Examples

```
%cst = constant <i8: 0> : tile<i8>
%0 = reshape %cst
    : tile<i8> -> tile<1x1x1xi8>
%t = constant <f32: 0.0> : tile<8x2xf32>
%1 = reshape %t
    : tile<8x2xf32> -> tile<2x2x4x1xf32>
```

See cuda_tile.reshape_0 for the full example listing.

```
  %cst = constant <i32: [[0, 1, 2, 3], [4, 5, 6, 7]]>
      : tile<2x4xi32>
  %r0 = reshape %cst
: tile<2x4xi32> -> tile<2x2x2xi32>
// Step 1: Turn source into 1D tile. Use row-major by convention.
// %tmp: [0, 1, 2, 3, 4, 5, 6, 7]
%tmp = reshape %cst
    : tile<2x4xi32> -> tile<8xi32>
// Step 2: Turn 1D tile into result tile. Use row-major by convention.
// %r: [[[0, 1], [2, 3]], [[4, 5], [6, 7]]]
%r1 =  reshape %tmp
        : tile<8xi32> -> tile<2x2x2xi32>
```

See cuda_tile.reshape_1 for the full example listing.

8.3.21 cuda_tile.scan

A parallel prefix sum operation

```
cuda_tile.scan %operands %dim %reverse %identities
```

Parameters

- **operands** (Variadic < tile >) - The a set of tiles to scan. 13.1
- **dim** (i32) - The index of the dimension along which to scan. 13.1
- **reverse** (bool) - Whether to scan in reverse order. 13.1
- **identities** (Array) - The identities of the scan operation. 13.1
- **results** (Variadic < tile >) - The resulting tiles from the scan operation. 13.1

The `scan` operation computes an inclusive parallel prefix along a given dimension of the input tiles using a binary associative function and an identity.

The `scan` operation applies a scan function defined over a tile of elements for a given type, utilizing an associative operation and an identity value. It operates on `operands` and `identities` across the specified `dim` , producing new `results` tile values. The exact evaluation order within each prefix is implementation-defined but the result remains deterministic across different runs of the same kernel on the same device.

scan(X,dim,identity,f)i1,⋯,id[j]=fold(f,identity,(Xi1,⋯,idim-1,0,idim+1,⋯,id,⋯,Xi1,⋯,idim-1,j,idim+1,⋯,id))

The scan preserves all intermediate accumulator values:

result[0]=f(identity,X[⋯,0,⋯])result[1]=f(result[0],X[⋯,1,⋯])$\vdots$result[j]=f(result[j-1],X[⋯,j,⋯])

When `reverse` is `true` , the prefix is taken in decreasing index order.

Let N be the size of the scanned dimension; then:

scanrev(X)[j]= fold(f,identity,(X[⋯,N-1,⋯],⋯,X[⋯,j,⋯]))

The `identities` attribute is a list of identity elements for each input tile; the identity at position `i` binds with the operand tile at the same position. The correct identity is a property of the scan function in the `body` (e.g., `sum` uses 0, `prod` uses 1, `min` uses +inf, `max` uses -inf).

The `body` region represents the binary associative operation. The region must contain **Tile IR** operations with 0-rank tile types. Region arguments are bound in operand order as `[op_0_current_iter, op_0_prev_iter, op_1_current_iter, op_1_prev_iter, ...]` , where `op_i_current_iter` is the current element along `dim` and `op_i_prev_iter` is the running accumulator for operand `i` . On the first step, the accumulator is the corresponding identity element.

Note

Associativity of the binary operation permits the compiler to reorganize the applications of the operation to achieve efficient parallel prefix scans on the GPU.

Warning

The scan operation is restricted to only support single tile input.

Constraints

- The operation must provide custom parsing and printing methods.
- The operation only has an effect if and only if it the region's operation have an effect.
- All operands must have identical shapes.
- Each provided region must contain exactly one block.

Examples

```
%input = constant <f32: 0.0> : tile<8x16xf32>
%result = scan %input dim=1 reverse=false identities=[1.0 : f32] : tile<8
    x16xf32> -> tile<8x16xf32>
(%acc: tile<f32>, %elem: tile<f32>) {
  %prod = mulf %acc, %elem rounding<nearest_even>: tile<f32>
  yield %prod : tile<f32>
}
```

See cuda_tile.scan_0 for the full example listing.

8.3.22 cuda_tile.select

Select values based on condition

```
cuda_tile.select %cond %val_if_true %val_if_false
```

Parameters

- **cond** (tile < i1 >) - The condition tile. 13.1
- **val_if_true** (tile) - The value if true tile. 13.1
- **val_if_false** (tile) - The value if false tile. 13.1
- **result** (tile) - The tile of selected values. 13.1

The `select` op chooses values based on the binary conditions supplied as the `cond` operand. The `val_if_true` operand contains the value(s) to use if the condition is 1. The `val_if_false` operand contains the value(s) to use if the condition is 0. The choice is made element-wise according to the values in the condition tile.

select(cond,x,y)i={xiif condi=1yiif condi=0

All tiles must have the same shape. The tiles `val_if_true` , `val_if_false` , and the result must have the same element type. The `cond` tile must be a tile of `i1` values.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `val_if_true` , `val_if_false` and `result` must have the same shape and element type (tile).
- The operation's result type may be inferred from its operands and attributes.

8.3.23 cuda_tile.unpack

Unpack a byte array into a tile

```
cuda_tile.unpack %source
```

Parameters

- **source** (tile < i8 >) - The input i8 tile. 13.2
- **result** (tile < i1 | i8 | i16 | i32 | i64 | f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The output unpacked tile. 13.2

The `unpack` operation takes a rank-1 `tile<i8>` and produces a rank-1 numeric tile.

Similar to `bitcast` , underlying bit-values are not changed. However, `unpack` does not operate elementwise, instead reinterpreting the entire tile as a numeric tile of different element type.

Input and output tiles must be rank-1 to eliminate unpacking ambiguity. The size of the input tile must match the number of bytes in the output tile.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same rank.

Examples

```
%arg0 = constant <i8: 0> : tile<64xi8>
%0 = unpack %arg0 : tile<64xi8> -> tile<32xf16>
```

See cuda_tile.unpack_0 for the full example listing.

```
%arg0 = constant <i8: 0> : tile<64xi8>
%0 = unpack %arg0 : tile<64xi8> -> tile<128xF4E2M1FN>
```

See cuda_tile.unpack_1 for the full example listing.

8.4 Conversions

There are no implicit type conversions in **Tile IR** thus we expose a set of explicit conversion operations for interconverting between types which have compatible representations or rules for conversion.

cuda_tile.bitcast preserves the contents of the input but allows for changing of element types, cuda_tile.exti and cuda_tile.trunci change the width of integer tiles, cuda_tile.ftoi and cuda_tile.itof convert floating-point tiles to integer tiles and vice versa, and cuda_tile.ftof converts between different floating-point types.

For more details on conversions and their rules see the individual operation's documentation.

8.4.1 cuda_tile.bitcast

Bitcast a tile from one element type to another

```
cuda_tile.bitcast %source
```

Parameters

- **source** (tile < i1 | i8 | i16 | i32 | i64 | f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The source tile to cast. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 | f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The casted tile. 13.1

The `bitcast` operation casts the input tile from one element type to another without modifying the underlying bits.

Only non-pointer types of the same bit width are allowed (e.g., `i32` to `f32`).

Pointer types must use cuda_tile.ptr_to_int or cuda_tile.int_to_ptr instead.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.2 cuda_tile.exti

Extend the width of an integer tile

```
cuda_tile.exti %from %signedness
```

Parameters

- **from** (tile < i1 | i8 | i16 | i32 | i64 >) - The input integer tile to extend. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **to** (tile < i1 | i8 | i16 | i32 | i64 >) - The extended integer tile. 13.1

The `exti` operation converts a tile of integers of a given width to a strictly larger width. Zero-extension is used for `unsigned` integers and sign-extension is used for `signed` integers.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.3 cuda_tile.ftof

Convert between floating-point types

```
cuda_tile.ftof %from %rounding_mode
```

Parameters

- **from** (tile < f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The input floating-point tile. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **to** (tile < f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The result floating-point tile. 13.1

The `ftof` operation converts a tile of a given floating-point element type into one of a different floating-point element type (for example, from `f32` to `f64`).

The source type and the result type must be different.

The `rounding_mode` attribute specifies the rounding behavior for the operation.

Only `NEAREST_EVEN` rounding mode is supported.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.4 cuda_tile.ftoi

Convert a tile from floating-point values to integer values

```
cuda_tile.ftoi %from %signedness %rounding_mode
```

Parameters

- **from** (tile < f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The input floating-point tile. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1

- **to** (tile < i1 | i8 | i16 | i32 | i64 >) - The result integer tile. 13.1

The `ftoi` operation converts a floating-point tile into an integer tile.

In contrast to a cuda_tile.bitcast which is bits preserving, this preserves the numerical value of the tile, rounded towards zero to the nearest integer of the provided type.

The `rounding_mode` attribute specifies the rounding behavior for the operation.

Only `NEAREST_INT_TO_ZERO` rounding mode is supported.

Warning

If the input floating-point value, after being rounded, is outside the

(signed or unsigned) range of the target integer type, the closest representable value is used instead. `NaN` values are converted to 0.

Input `Inf` values are undefined behavior.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.5 cuda_tile.itof

Convert integer to floating-point

```
cuda_tile.itof %from %signedness %rounding_mode
```

Parameters

- **from** (tile < i1 | i8 | i16 | i32 | i64 >) - The input integer tile. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **to** (tile < f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The converted floating-point tile. 13.1

The `itof` operation converts an integer tile into a float tile.

In contrast to cuda_tile.bitcast , this preserves the numerical value of the tile, rounded to the nearest floating-point number of the provided type.

Warning

If the input integer value, after being rounded, is outside the range of the target floating-point type, it is converted to `Inf` for types that support that value, and `NaN` otherwise.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.6 cuda_tile.int_to_ptr

Convert a tile of integers to a tile of pointers

```
cuda_tile.int_to_ptr %source
```

Parameters

- **source** (tile < i64 >) - The input tile of integers. 13.1
- **result** (ptr) - The output tile of pointers. 13.1

The `int_to_ptr` operation converts a tile of integers to a tile of pointers.
The `source` operand is interpreted as an unsigned integer.
The inverse of this operation is cuda_tile.ptr_to_int .

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.7 cuda_tile.ptr_to_int

Convert a tile of pointers to a tile of integers

```
cuda_tile.ptr_to_int %source
```

Parameters

- **source** (ptr) - The input tile of pointers. 13.1
- **result** (tile < i64 >) - The output tile of integers. 13.1

The `ptr_to_int` operation converts a tile of pointer-type elements to a tile of `i64` elements.
The result values should be interpreted as unsigned integers.
The inverse of this operation is cuda_tile.int_to_ptr .

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.8 cuda_tile.ptr_to_ptr

Reinterpret a tile of one pointer type as another

```
cuda_tile.ptr_to_ptr %source
```

Parameters

- **source** (ptr) - Tile with source pointer element type. 13.1
- **result** (ptr) - Tile with target pointer element type. 13.1

The `ptr_to_ptr` operation casts a tile of pointers from a pointer of one element type to another element. Casts between pointer and non-pointer types are disallowed.
In order to perform those conversions, use cuda_tile.ptr_to_int or cuda_tile.int_to_ptr .
These operations are distinct to enable future compiler reasoning about pointer provenance.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.4.9 cuda_tile.trunci

Truncates the width of an integer tile

```
cuda_tile.trunci %from %overflow
```

Parameters

- **from** (tile < i1 | i8 | i16 | i32 | i64 >) - The input integer tile to truncate. 13.1
- **overflow** (IntegerOverflow) - The overflow behavior of the operation. 13.1

- **to** (tile < i1 | i8 | i16 | i32 | i64 >) - The truncated integer tile. 13.1

The `trunci` operation converts a tile of integers of a given element type to one with a strictly smaller width.

The optional overflow attribute specifies whether an overflow can occur when interpreting the operand as a signed and/or unsigned integer. In case of "no signed wrap", all truncated bits must have the same value as the most significant bit of the truncated result. In case of "no unsigned wrap", the truncated bits must be zero.

The `overflow` attribute is used to instruct the compiler on how to reason about the overflow behavior of the specific operation.

These attributes serve as assumptions that the compiler may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

- `none` - The compiler makes no assumptions regarding overflow behavior.
- `no_signed_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands signed integers.
- `no_unsigned_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands unsigned integers.
- `no_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands as signed or unsigned integers.

If an overflow occurs at runtime despite the value of overflow stating otherwise, the behavior is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

8.5 Control Flow

Tile IR contains a standard set of control flow operations that enable conditionals, and loops.

The operations are designed in the style of the MLIR Control Flow dialect .

A notable difference is that we allow the nesting of control flow operations for example a cuda_tile.if may appear inside a cuda_tile.loop or cuda_tile.for .

The main control structures are:

- cuda_tile.if which implements conditional branching.
- cuda_tile.loop which implements a loop with arbitrary exit conditions.
- cuda_tile.for which implements a range-based loop with a fixed number of iterations.

These operations and their supporting operations are described in the following section.

8.5.1 cuda_tile.assert

Terminate kernel execution with an error message if condition is false-y

```
cuda_tile.assert %condition %message
```

Parameters

- **condition** (tile < i1 >) - The condition tile to check. 13.1
- **message** (String) - The error message to display if assertion fails. 13.1

No results.

Description The `assert` operation takes as `condition` a tile of `i1` values. For each value that is `0` , it prints the given error message, along with the index of the value within the tile.

If at least one value is `0` , an error is signalled to the host side. The kernel, including the tile block that failed the assertion, may keep running.

Assertions are for debugging purposes. They can affect performance and it is therefore recommended to remove them in production code.

Constraints No constraints.

Examples

```
assert %arg0, "assertion failed" : tile<i1>
```

See cuda_tile.assert_0 for the full example listing.

8.5.2 cuda_tile.break

Break from loop

```
cuda_tile.break %operands
```

Parameters

- **operands** (Variadic < Any >) - The operands to yield to the parent loop upon termination. 13.1

No results.

Description The `break` operation is a terminator operation of a cuda_tile.loop .

It may yield any number of `$operands` to the parent loop upon termination. The number of values yielded and the execution semantics of how they are yielded are determined by the parent loop.

The `break` operation always returns control to the innermost enclosing loop operation, even when it is nested within other control constructs such as `if` or additional loops.

Constraints

- The operation must terminate its parent basic block.

Examples

```
// Break from the body of a loop.
loop {
    break
}
// Break from an if nested within the loop.
loop  {
    %condition = constant <i1: 1> : tile<i1>
    if %condition  {
        break
    }
    // ...
}
%initValue0 = constant <f32: 0.0> : tile<f32>
```

```
// Break from an if nested within the loop, while yielding values.
%results = loop iter_values(%var0 = %initValue0): tile<f32> -> tile<f32> {
    %condition = constant <i1: 1> : tile<i1>
    if %condition  {
        // ...
        yield
    } else {
        // %if.loopValue0 = ...
        %loopValue0 = constant <f32: 1.0> : tile<f32>
        break %loopValue0 : tile<f32>
    }
    %loopValue1 = constant <f32: 1.0> : tile<f32>
    continue %loopValue1 : tile<f32>
}
```

See cuda_tile.break_0 for the full example listing.

8.5.3 cuda_tile.continue

Continue to next loop iteration

```
cuda_tile.continue %operands
```

Parameters

- **operands** (Variadic < Any >) - The values to yield to the parent loop. 13.1

No results.

Description The `continue` operation represents a block terminator that returns control to a loop operation, such as cuda_tile.for and cuda_tile.loop . The operation may yield any number of `$operands` to the parent loop upon termination.

The requirements and semantics of the `continue` operation are defined by the parent loop operation, see the loop operation's description for particular semantics.

The `continue` operation always returns control to the innermost enclosing loop operation, even when it is nested within other control constructs such as `if` or additional loops.

Constraints

- The operation must terminate its parent basic block.

Examples

```
  %lowerBound = constant <i32: 0> : tile<i32>
  %upperBound = constant <i32: 10> : tile<i32>
  %step = constant <i32: 1> : tile<i32>
  %condition = constant <i1: 1> : tile<i1>
  // Continue from the body of a loop.
  for %iv in (%lowerBound to %upperBound, step %step) : tile<i32> {
      continue
  }
  // Continue from an if nested within the loop.
  for %iv in (%lowerBound to %upperBound, step %step) : tile<i32> {
      if %condition  {
          continue
      }
      // ...
  }
// Continue from an if nested within the loop, while yielding values.
%initVar0 = constant <f32: 0.0> : tile<f32>
```

```
%results = for %iv in (%lowerBound to %upperBound, step %step) : tile<i32>
          iter_values(%var0 = %initVar0) -> (tile<f32>)
  {
      if %condition {
          // ...
          yield
      } else {
          %loopValue0 = constant <f32: 1.0> : tile<f32>
          continue %loopValue0 : tile<f32>
      }
      %loopValue1 = constant <f32: 1.0> : tile<f32>
      continue %loopValue1 : tile<f32>
  }
```

See cuda_tile.continue_0 for the full example listing.

8.5.4 cuda_tile.for

For loop over integer range

```
cuda_tile.for %lowerBound %upperBound %step %initValues %unsignedCmp
```

Parameters

- **lowerBound** (tile < any >) - The lower bound of the loop. 13.1
- **upperBound** (tile < any >) - The upper bound of the loop. 13.1
- **step** (tile < any >) - The step of the loop. 13.1
- **initValues** (Variadic < Any >) - The initial values of the loop-carried values. 13.1
- **unsignedCmp** (Flag) - If present, use unsigned integer comparison for loop termination. 13.2
- **resultValues** (Variadic < Any >) - The values of the loop-carried variables after loop termination. 13.1

The `for` operation is a structured range-based sequential loop.

The loop operation consists of (1) a range formed by `lowerBound` , `upperBound` , and `step` ,

(2) a set of loop-carried values which are initialized by `initValues` and updated by each iteration of the loop, and

(3) a region which represents the loop body.

The iteration space is defined by the interval [l o w e r B o u n d , u p p e r B o u n d) with each value separated by `step` .

range(Lb,Ub,S)={Lb+i$\cdot$S$|$i$\in$Z,Lb+i$\cdot$S<Ub}

`lowerBound` , `upperBound` , and `step` must be of the same type. `lowerBound` and `upperBound` specify a half-open (or exclusive) range: the range includes the `lowerBound` but does not include the `upperBound` . `step` must be positive but the bounds may be negative or zero.

The `lowerBound` , `upperBound` , and `step` operands are interpreted as signed integers.

The first iteration of the loop receives the induction variable initialized to the value of `lowerBound` and the loop-carried values initialized to the values of `initValues` .

The loop body is executed for each value in the range, receiving an integer induction variable incremented by `step` on each iteration and the loop-carried values which correspond to the loop-carried values yielded by the previous loop iteration.

The loop terminates when the induction variable is greater than or equal to `upperBound` . By default, signed comparison is used between the upperBound and the induction variable. To use unsigned comparison instead, specify the optional `unsigned` unit attribute.

The body of the loop must be terminated by a cuda_tile.continue that yields the next iteration's value for each loop carried variable.

The for operation produces one return value for each loop carried variable. The type of the i -th return value is that of the i -th loop carried variable and its value is the final value of the i -th loop carried variable.

Warning

- Loop carried variables can not be a tensor_view or view type.
- `for` operations cannot terminate early and must end in a cuda_tile.continue .

- The operation must define scope when stack allocations are freed automatically.
- `lowerBound` , `upperBound` and `step` must have the same shape and element type (tile < any >).
- `initValues` and `resultValues` must have the same shape and element type (Variadic < Any >).
- The operation must provide custom parsing and printing methods.
- The operation only has an effect if and only if it the region's operation have an effect.
- Each provided region must contain exactly one block.

Examples

```
%lowerBound = constant <i32: 0> : tile<i32>
%upperBound = constant <i32: 10> : tile<i32>
%step = constant <i32: 1> : tile<i32>
// A simple loop iterating over an i32 range.
for %iv in (%lowerBound to %upperBound, step %step) : tile<i32> {
    continue
}
%initVal0 = constant <f32: 0.0> : tile<f32>
// A similar loop to the above, but with a loop carried value, val0.
%results = for %iv in (%lowerBound to %upperBound, step %step) : tile<i32>
                    iter_values(%val00 = %initVal0) -> (tile<f32>) {
  %loopVal0 = constant <f32: 1.0> : tile<f32>
  continue %loopVal0 : tile<f32>
}
```

See cuda_tile.for_0 for the full example listing.

8.5.5 cuda_tile.if

Conditional execution

```
cuda_tile.if %condition
```

Parameters

- **condition** (tile < i1 >) - The condition of the if operation. 13.1
- **results** (Variadic < Any >) - The results of the if operation. 13.1

The `if` operation represents an if-then-else construct.

The if operation consists of (1) a control operand which is a `tile<i1>` value, (2) a true branch `thenRegion` and (3) an optional false branch `elseRegion` .

The `if` operation may produce results by yielding values in each branch using cuda_tile.yield .

If yielding value(s) the types of yielded values must match and the result result type of the `if` operation will be the same as the yielded values.

If yielding values the else branch is required and must also yield a value.

The values returned will be dependent on which branch is taken.

Warning

The `if` operation has a set of additional restrictions today:

- Results of `if` must not be a tensor_view or view type.

- All regions must have zero arguments.
- The operation must provide custom parsing and printing methods.
- The operation only has an effect if and only if it the region's operation have an effect.
- Each provided region must contain exactly one block.

Examples

```
%condition = constant <i1: 1> : tile<i1>
// A simple if operation that conditionally executes a region.
if %condition  {
  // ...
}
// An if operation with an "else" branch.
if %condition  {
  // ...
} else {
  // ...
}
// An if operation that returns mixed types (f32,i32)
%x, %y = if %condition -> (tile<f32>, tile<i32>) {
  %x_then = constant <f32: 1.0> : tile<f32>
  %y_then = constant <i32: 2> : tile<i32>
  yield %x_then, %y_then : tile<f32>, tile<i32>
} else {
  %x_then = constant <f32: 1.0> : tile<f32>
  %y_then = constant <i32: 42> : tile<i32>
  yield %x_then, %y_then : tile<f32>, tile<i32>
}
```

See cuda_tile.if_0 for the full example listing.

8.5.6 cuda_tile.loop

Loop until a break operation

```
cuda_tile.loop %initValues
```

Parameters

- **initValues** (Variadic < Any >) - The initial values of the loop. 13.1

- **resultValues** (Variadic < Any >) - The result values of the loop. 13.1

The `loop` operation represents an, unstructured, infinite loop that executes until a cuda_tile.break is reached.

The loop consists of a (1) a set of loop-carried values which are initialized by `initValues` and updated by each iteration of the loop, and

(2) a region which represents the loop body.

The loop will execute the body of the loop until a cuda_tile.break is dynamically executed.

Each control path of the loop must be terminated by:

- a cuda_tile.continue that yields the next iteration's value for each loop carried variable.
- a cuda_tile.break that terminates the loop and yields the final loop carried values.

As long as each loop iteration is terminated by one of these operations they may be combined with other control flow operations to express different control flow patterns.

The loop operation produces one return value for each loop carried variable. The type of the i th return value is that of the i th loop carried variable and its value is the final value of the i th loop carried variable.

Warning

Loop operations have a set of additional restrictions today:

- Early returns from inside loops are not supported, a code generator must first terminate the loop and then return if they wish to end the function execution entirely.
- Loop carried variables can not be a tensor_view or view type.

- The operation must define scope when stack allocations are freed automatically.
- The operation must provide custom parsing and printing methods.
- The operation only has an effect if and only if it the region's operation have an effect.
- Each provided region must contain exactly one block.

Examples

```
// A simple "while-do" loop.
loop {
    %cond = constant <i1: 1> : tile<i1>
    if %cond {
        continue
    }
    break
}
```

See cuda_tile.loop_0 for the full example listing.

```
// A simple "do-while" loop.
loop {
    //... body of the loop.
    %cond = constant <i1: 1> : tile<i1>
    if %cond {
        continue
    }
    break
}
```

See cuda_tile.loop_1 for the full example listing.

```
%initValue0 = constant <f32: 0.0> : tile<f32>
// A loop that yields carried-iteration values, returning the final values.
%results = loop iter_values(%value0 = %initValue0) : tile<f32> -> tile<f32> {
    %cond = constant <i1: 1> : tile<i1>
    if %cond {
        %loopValue0 = constant <f32: 0.0> : tile<f32>
        continue %loopValue0 : tile<f32>
    }
    break %value0 : tile<f32>
}
```

See cuda_tile.loop_2 for the full example listing.

```
%initValue0 = constant <i32: 0> : tile<i32>
// A loop that uses loop-carried values and returns a different type.
%results = loop iter_values(%value0 = %initValue0) : tile<i32> -> tile<f32> {
    %cond = constant <i1: 1> : tile<i1>
    if %cond {
        %newLoopValue = constant <i32: 0> : tile<i32>
        continue %newLoopValue : tile<i32>
    }
    %finalReturnValue = constant <f32: 0.0> : tile<f32>
    break %finalReturnValue : tile<f32>
}
```

See cuda_tile.loop_3 for the full example listing.

8.5.7 cuda_tile.return

Return value(s) from a function

```
cuda_tile.return %operands
```

Parameters

- **operands** (Variadic < Any >) - The values to return. 13.1

No results.

Description The `return` operation returns control to the caller of a function.

Warning

Currently `return` implements restricted return semantics, notably:

- cuda_tile.entry operations do not produce return value(s) and thus `return` may be used to terminate the execution of the kernel by invoking the operation with no operands
- `return` can not be directly used inside of loop bodies to terminate the the execution of the kernel
- The operation must terminate its parent basic block.

Examples

```
experimental$func @foo() -> (tile<i32>, tile<f16>) {
  %0 = constant <i32: 0> : tile<i32>
  %1 = constant <f16: 0.0> : tile<f16>
  // ...
  return %0, %1 : tile<i32>, tile<f16>
}
```

See cuda_tile.return_0 for the full example listing.

```
entry @foo() {
  %0 = constant <i32: 0> : tile<i32>
  %1 = constant <f16: 0.0> : tile<f16>
  // ...
  return
}
```

See cuda_tile.return_1 for the full example listing.

8.5.8 cuda_tile.yield

Yield a value from the block

```
cuda_tile.yield %operands
```

Parameters

- **operands** (Variadic < Any >) - The operands to yield to the parent operation. 13.1

No results.

Description The `yield` operation terminates a block that must yield control back to the parent operation such as `if` , `scan` , `reduce` .

The operation may yield any number of `$operands` to the parent upon termination. The number of values yielded and the execution semantics of how they are yielded are determined by the parent operation.

Note

Unlike standard MLIR control flow dialects `yield` is not used for loop control flow, see cuda_tile.break and cuda_tile.continue for loop control flow.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- The operation must terminate its parent basic block.

Examples

```
%condition = constant <i1: true> : tile<i1>
// Yield from the body of an if conditional.
if %condition  {
    yield
}
// Yield values from within an if conditional.
%x, %y = if %condition -> (tile<f32>, tile<f32>) {
    %x_then = constant <f32: 0.0> : tile<f32>
    %y_then = constant <f32: 1.0> : tile<f32>
    yield %x_then, %y_then : tile<f32>, tile<f32>
} else {
    %x_else = constant <f32: 2.0> : tile<f32>
    %y_else = constant <f32: 3.0> : tile<f32>
    yield %x_else, %y_else : tile<f32>, tile<f32>
}
```

See cuda_tile.yield_0 for the full example listing.

8.6 Memory

Tile IR contains a set of memory operations which enable loading, storing, and manipulating memory.

There are a few families of memory operations in **Tile IR** :

- Tile of pointer based memory operations such as cuda_tile.load_ptr_tko and cuda_tile.store_ptr_tko which load and store tiles from and to global memory.
- View based memory operations such as cuda_tile.load_view_tko and cuda_tile.store_view_tko which load and store tiles from and to views.
- Atomic memory operations such as cuda_tile.atomic_rmw_tko and cuda_tile.atomic_cas_tko which perform atomic operations on global memory.

Currently all memory operations are token-ordered; the ordering between any pair of memory operations is undefined unless connected by tokens. For more discussion on token-ordered operations see Memory Model .

Warning

Reading or writing of bound of any allocation is undefined behavior. Examples of out of bounds access are:

- Pointer memory operations to tiles containing elements outside the allocation, for example offseting passed the end of the allocation.
- Associating an invalid layout with a base pointer, that describes a striding or shape that over runs the allocation and then indexing into the view.
- Indexing into a view with indices that are out of bounds.

Note

The rules of what consititues out of bounds is modified when using padded views or masking, see Type System for more details on specific types.

8.6.1 cuda_tile.join_tokens

Product a new token which depends on the input tokens

```
cuda_tile.join_tokens %tokens
```

Parameters

- **tokens** (Variadic < token >) - The input tokens to join. 13.1
- **result** (token) - The joined token. 13.1

The `join_tokens` operation produces a fresh token which depends on all input tokens.

Token-ordered operations which consume the new token will then be ordered with respect to all joined tokens.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- The operation's result type may be inferred from its operands and attributes.

8.6.2 cuda_tile.load_ptr_tko

Load and gather data from global memory using a pointer tile without ordering guarantees

```
cuda_tile.load_ptr_tko %memory_ordering_semantics %memory_scope %source %mask %
    paddingValue %token %optimization_hints
```

Parameters

- **memory_ordering_semantics** (MemoryOrderingSemantics) - The memory ordering semantics for the load operation. 13.1
- **memory_scope** (MemoryScope) - The memory scope for the atomic operation. 13.1
- **source** (ptr) - The source tile of pointers. 13.1
- **mask** (tile < i1 >) - The mask for the load operation. 13.1
- **paddingValue** (tile < i1 | i8 | i16 | i32 | i64 | f16 | bf16 | f32 | f64 | fp8e4m3fn | fp8e5m2 | tf32 >) - The padding value for the load operation. 13.1
- **token** (token) - The token for the load operation. 13.1
- **optimization_hints** (OptimizationHints) - Optimization hints for operation 13.1

- **result** (tile) - The result of the load operation. 13.1
- **result_token** (token) - The result token of the load operation. 13.1

This `load` OP performs a gather operation by loading a tile of data from global memory into a result tile based on a tile of pointers provided by the `source` operand.

The `source` operand is a tile of pointers, which specifies the memory locations from which the data is gathered. The operation loads this data and returns it as the `result` tile. When loading i1 values, each value is loaded from a full byte in memory. Any nonzero byte is canonicalized to 0x01, and zero bytes become 0x00.

Optionally, a `mask` operand can be provided to control the gathering of elements. If present, only the elements specified by the `mask` are loaded.

The shape of the `mask` must match the shape of the `result` .

When `mask` is present one `paddingValue` can be optionally present as well.

The `paddingValue` must have the same shape of the `source` tile. If it is not present, the value of masked elements are undefined.

Token-ordered operations are not constrained by program order.

The compiler may reorder them (i.e. place them earlier or later in program order) unless further constrained by tokens.

The `memory_ordering_semantics` attribute specifies the concurrency assumption between memory accesses in different threads, which controls the synchronization required.

For example, `weak` ordering allows the compiler to assume that there are no concurrent accesses to any accessed location.

For more information, refer to the memory model section of the specification.

- `weak` - No concurrent accesses to the source/destination location.
- `relaxed` - There may be concurrent access to the location, but this access does not establish a happens-before relationship.
- `acquire` - There may be concurrent accesses to the location. If this acquire observes a release operation, then *happens before* is established.

Note: The following variants are not supported by this operation: `release` , `acq_rel` .

The `memory_scope` attribute specifies a communication scope for memory operations.

When communicating with other concurrent threads in the system, the scope must be broad enough to encompass all other threads which are participating in the communication, or data races may occur.

- `tl_blk` - There may be concurrent accesses from within the same tile block.
- `device` - There may be concurrent accesses from within the same device (i.e., GPU).
- `sys` - There may be concurrent accesses from anywhere within the system (i.e., all devices).

The `optimization_hints` attribute provides architecture-specific compiler hints in the form of nested dictionaries.

The hints are specified for each architecture (e.g., `sm_100` , `sm_120`) and for each architecture the user can specify specific hints for each operation.

- `num_cta_in_cga` - suggest the number of CTAs in a CGA (which must be the power of 2 less than or equal to 16) for cuda_tile.entry .
- `allow_tma` - suggest whether to use TMA for cuda_tile.load_view_tko and cuda_tile.store_view_tko .
- `latency` - latency hint for cuda_tile.load_view_tko and cuda_tile.store_view_tko .

For example they can be annotated as:

```
optimization_hints=<
  sm_100 = {num_cta_in_cga = 8},
  sm_120 = {num_cta_in_cga = 16}
>
```

Constraints

- The operation must encode variadic operand segment sizes in attributes.
- source type is expected a pointer type of result type
- shape of 'mask' must match the shape of 'source'
- type of 'paddingValue' must match the type of 'result'

Examples

```
%mask = constant <i1: 1> : tile<i1>
%padding = constant <f32: 0.0> : tile<f32>
  // Load without token.
  %result0, %res_token0 = load_ptr_tko weak %ptr, %mask, %padding
      : tile<ptr<f32>>, tile<i1>, tile<f32> -> tile<f32>, token
  // Load with token.
  %token0 = make_token : token
  %result1, %res_token1 = load_ptr_tko weak %ptr, %mask, %padding token=%
      token0
      : tile<ptr<f32>>, tile<i1>, tile<f32> -> tile<f32>, token
  return
```

See cuda_tile.load_ptr_tko_0 for the full example listing.

8.6.3 cuda_tile.make_token

Create a fresh token with no prior dependencies

```
cuda_tile.make_token
```

Parameters No parameters.

Results

- **result** (token) - A fresh token with no prior dependencies. 13.1

The `make_token` operation creates a fresh token with no prior dependencies.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- The operation's result type may be inferred from its operands and attributes.

8.6.4 cuda_tile.store_ptr_tko

Store and scatter data from pointer of tile to global memory without ordering guarantees

```
cuda_tile.store_ptr_tko %memory_ordering_semantics %memory_scope %destination %
    value %mask %token %optimization_hints
```

Parameters

- **memory_ordering_semantics** (MemoryOrderingSemantics) - The memory ordering semantics. 13.1
- **memory_scope** (MemoryScope) - The optional memory scope. 13.1
- **destination** (ptr) - The destination pointer tile. 13.1
- **value** (tile) - The value tile to store. 13.1
- **mask** (tile < i1 >) - The optional mask for selective storage. 13.1
- **token** (token) - The optional token for operation ordering. 13.1
- **optimization_hints** (OptimizationHints) - Optimization hints for operation 13.1
- **result_token** (token) - The result token for synchronization. 13.1

The `store` operation performs a scatter by storing a tile of data from a tile into global memory.

The `destination` operand is a tile of pointers indicating the global memory locations where data from the `value` tile will be stored. When storing i1 values, each value occupies a full byte in memory. Any nonzero byte is canonicalized to 0x01, and zero bytes become 0x00.

Additionally, the operation supports an optional `mask` operand, which allows selective scattering of elements. If provided, only the elements specified by the `mask` are stored. The shape of the `mask` must align with the shape of the `value` tile.

The `memory_ordering_semantics` attribute specifies the concurrency assumption between memory accesses in different threads, which controls the synchronization required.

For example, `weak` ordering allows the compiler to assume that there are no concurrent accesses to any accessed location.

For more information, refer to the memory model section of the specification.

- `weak` - No concurrent accesses to the source/destination location.
- `relaxed` - There may be concurrent access to the location, but this access does not establish a happens-before relationship.
- `release` - There may be concurrent access to the location. If this release is observed with an acquire operation, then *happens before* is established.

Note: The following variants are not supported by this operation: `acquire` , `acq_rel` .

The `memory_scope` attribute specifies a communication scope for memory operations.

When communicating with other concurrent threads in the system, the scope must be broad enough to encompass all other threads which are participating in the communication, or data races may occur.

- `tl_blk` - There may be concurrent accesses from within the same tile block.
- `device` - There may be concurrent accesses from within the same device (i.e., GPU).
- `sys` - There may be concurrent accesses from anywhere within the system (i.e., all devices).

The `optimization_hints` attribute provides architecture-specific compiler hints in the form of nested dictionaries.

The hints are specified for each architecture (e.g., `sm_100` , `sm_120`) and for each architecture the user can specify specific hints for each operation.

- `num_cta_in_cga` - suggest the number of CTAs in a CGA (which must be the power of 2 less than or equal to 16) for cuda_tile.entry .
- `allow_tma` - suggest whether to use TMA for cuda_tile.load_view_tko and cuda_tile.store_view_tko .
- `latency` - latency hint for cuda_tile.load_view_tko and cuda_tile.store_view_tko .

For example they can be annotated as:

```
optimization_hints=<
  sm_100 = {num_cta_in_cga = 8},
  sm_120 = {num_cta_in_cga = 16}
>
```

Constraints

- The operation must encode variadic operand segment sizes in attributes.
- destination type is expected a pointer type of value type
- shape of 'destination' must match the shape of 'mask'
- The operation's result type may be inferred from its operands and attributes.

8.7 Floating Point

Tile IR contains a set of typed arithmetic operations which implement familiar arithmetic operations on floating-point types for integer operations see Integer .

All operations are implemented in a manner that is efficient for the target architecture and device family. In most common cases this means utilizing the underlying hardware's native floating-point operations. Due to **Tile IR** 's stability guarantees and higher-level programming model some types on some hardware may be emulated, see Stability for more information about the stability guarantees and information about per device behavior.

8.7.1 Floating-Point Arithmetic

Standard floating-point types implement the IEEE-754 standard for floating-point arithmetic. On NVIDIA hardware, certain types are non-standard and *do not* implement the IEEE-754 standard, see Element Types for more details about the different floating-point types, their precision, storage, and formats.

Supports 16-bit, 32-bit, and 64-bit floating-point data types.

8.7.2 Floating-Point Math

Tile IR contains a set of standard math library operations which implement familiar mathematical functions over tensors supporting 16-bit, 32-bit, and 64-bit floating-point data types.

Note

32-bit and 64-bit operations typically leverage efficient hardware-specific instructions. Some 16-bit operations are emulated using wider intermediate computations, and may not offer the same performance.

Warning

There are some restrictions based on data type support which are detailed in the Type System section.

8.7.3 cuda_tile.absf

Element-wise floating-point absolute value

```
cuda_tile.absf %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input float tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The absolute value of the input tile. 13.1

The `absf` operation computes the element-wise absolute value of the input float tile.

absf(x)i=|x|i

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.4 cuda_tile.addf

Element-wise floating-point addition

```
cuda_tile.addf %lhs %rhs %rounding_mode %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **rounding__mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **flush__to__zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The sum of the input tiles. 13.1

The `addf` operation computes the element-wise addition of two tiles with floating-point element type.

addf(x,y)i=xi+yi

The addition of individual elements is performed by the target architecture's native floating-point addition for the given element type unless otherwise specified.

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers and rounding modes for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no
`rounding<nearest_even>`	yes	yes	yes	yes
`rounding<zero>`	yes	yes	yes*	yes*
`rounding<negative_inf>`	yes	yes	yes*	yes*
`rounding<positive_inf>`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.5 cuda_tile.atan2

Element-wise atan2

```
cuda_tile.atan2 %x %y
```

Parameters

- **x** (tile < f16 | bf16 | f32 | f64 >) - The input x float tile. 13.2
- **y** (tile < f16 | bf16 | f32 | f64 >) - The input y float tile. 13.2
- **result** (tile < f16 | bf16 | f32 | f64 >) - The element-wise result tile. 13.2

The `atan2` operation calculates the principal value of the arc tangent of the ratio of first and second input arguments x / y. The quadrant of the result is determined by the signs of inputs x and y.

(atan2(x,y))i=atan2(xi,yi)

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `x` , `y` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%x = constant <f32: [1.0, -1.0, 0.0, 2.0]> : tile<4xf32>
%y = constant <f32: [1.0,  1.0, 1.0, 0.0]> : tile<4xf32>
%res = atan2 %x, %y : tile<4xf32>
```

See cuda_tile.atan2_0 for the full example listing.

8.7.6 cuda_tile.ceil

Element-wise ceiling

```
cuda_tile.ceil %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input float tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The ceiling of the input tile. 13.1

The `ceil` operation computes the element-wise ceiling on the input floating-point tile. The ceiling operation rounds each element up to the largest integer value that is greater than or equal to the input value.

ceil(x)i=min{n$\in$Z$|$n$\geq$xi}

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%result = ceil %source : tile<f32>
```

See cuda_tile.ceil_0 for the full example listing.

8.7.7 cuda_tile.cosh

Element-wise hyperbolic cosine

```
cuda_tile.cosh %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input floating-point tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The hyperbolic cosine of the input tile. 13.1

The `cosh` operation computes the element-wise hyperbolic cosine of the input tile with floating-point element type.

cosh(x)i=coshxi

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.8 cuda_tile.cos

Element-wise cosine

```
cuda_tile.cos %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input float tile. 13.1

- **result** (tile < f16 | bf16 | f32 | f64 >) - The cosine of the input tile. 13.1

The `cos` operation computes the element-wise cosine of the input floating-point tile.

cos(x)i=cos(xi)

This operation is emulated in :code:f32‘ `when executed on half-precision inputs (` :code:`f16 and` :code:`bf16‘). See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = cos %in : tile<4xf32>
```

See cuda_tile.cos_0 for the full example listing.

8.7.9 cuda_tile.divf

Element-wise floating-point division

```
cuda_tile.divf %lhs %rhs %rounding_mode %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The dividend input floating-point tile. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The divisor input floating-point tile. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the `divf` operation. 13.1

The `divf` operation computes the element-wise division of two input tiles with floating-point element types.

The `approx` rounding mode implements a fast approximation of divide, computed as a multiplication by reciprocal. For `|rhs|` in normalized range `[2^(-126), 2^(126)]` the maximum ULP (Unit in the Last Place) error is `2` .

For `2^(126) < |rhs| < 2^(128)` , if `lhs` is infinity the operation returns `NaN` , otherwise `0` .

The `full` rounding mode implements a relatively fast, full-range approximation that scales operands to achieve better accuracy, but is not fully

IEEE 754 compliant. The maximum ulp error is 2 across the full range of inputs.

div(lhs, rhs)i=lhsi/rhsi

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers and rounding modes for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no
`approx`	yes	no	no	no
`full`	yes	no	no	no
`rounding<nearest_even>`	yes	yes	yes*	yes*
`rounding<zero>`	yes	yes	yes*	yes*
`rounding<negative_inf>`	yes	yes	yes*	yes*
`rounding<positive_inf>`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.10 cuda_tile.exp2

Element-wise power of two

```
cuda_tile.exp2 %source %flush_to_zero
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input floating-point tile. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of raising 2 to the power of the input tile. 13.1

The `exp2` operation computes the element-wise power of two of the input floating-point tile.

exp2(x)i=2xi

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

The below table shows the supported modifiers for each data type.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = exp2 %in : tile<4xf32>
```

See cuda_tile.exp2_0 for the full example listing.

8.7.11 cuda_tile.exp

Element-wise exponential

```
cuda_tile.exp %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input float tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The exponential of the input tile. 13.1

The `exp` operation computes the element-wise exponential of the input floating-point tile.

exp(x)i=exi

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = exp %in : tile<4xf32>
```

See cuda_tile.exp_0 for the full example listing.

8.7.12 cuda_tile.floor

Element-wise floor rounding

```
cuda_tile.floor %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input tile to the floor operation. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the floor operation. 13.1

The `floor` operation computes the element-wise floor on the input floating-point tile rounding each element down to the largest integer that is less than or equal to the element.

floori(xi)=max{n$\in$Z$|$n$\leq$xi}

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%source = constant <f32: 1.5> : tile<f32>
%result = floor %source : tile<f32>
```

See cuda_tile.floor_0 for the full example listing.

8.7.13 cuda_tile.fma

Floating point fused multipy-add

```
cuda_tile.fma %lhs %rhs %acc %rounding_mode %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **acc** (tile < f16 | bf16 | f32 | f64 >) - The accumulator operand. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1

- **result** (tile < f16 | bf16 | f32 | f64 >) - The fused multiply-add of the input tiles. 13.1

Takes three operands `lhs` , `rhs` and `acc` , returns `result = lhs * rhs + acc` .
fma(x,y,z)i=xi×yi+zi
The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers and rounding modes for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no
`rounding<nearest_even>`	yes	yes	yes	yes
`rounding<zero>`	yes	yes	yes*	yes*
`rounding<negative_inf>`	yes	yes	yes*	yes*
`rounding<positive_inf>`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` , `acc` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.14 cuda_tile.log2

Element-wise base-2 logarithm

```
cuda_tile.log2 %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input floating-point tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the log2 operation. 13.1

The log2 operation computes the element-wise base-2 logarithm of a floating-point tile.
log2(x)i=log2(xi)
This operation is emulated in :code:f32‘ when executed on half-precision inputs (:code:f16 and :code:bf16‘). See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- source and result must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = log2 %in : tile<4xf32>
```

See cuda_tile.log2_0 for the full example listing.

8.7.15 cuda_tile.log

Element-wise natural logarithm

```
cuda_tile.log %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input floating-point tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the log operation. 13.1

The log operation computes the element-wise natural logarithm of a floating-point tile.
log(x)i=ln(xi)
This operation is emulated in :code:f32‘ when executed on half-precision inputs (:code:f16 and :code:bf16‘). See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- source and result must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.16 cuda_tile.maxf

Element-wise floating-point maximum

```
cuda_tile.maxf %lhs %rhs %propagate_nan %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **propagate_nan** (Flag) - When set, `maxf` (or `minf`) returns a `NaN` if either of the two compared elements is `NaN` . 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the `maxf` operation. 13.1

The `maxf` operation computes the element-wise maximum of two input tiles with floating-point element types.

The `propagate_nan` controls how `maxf` will interpret `NaN` . If the `propagate_nan` modifier is set, `maxf` returns a canonical `NaN` if either of the compared elements is `NaN` (IEEE 754-2019's maximum). While if the `propagate_nan` modifier is not set, `maxf` returns a canonical `NaN` only if both elements are `NaN` ; otherwise, it returns the non- `NaN` element (IEEE

754-2019's maximumNumber).

If neither element is `NaN` , `maxf` will return the greater of the inputs. `+0.0` is considered greater than `-0.0` .

If the `flush_to_zero` modifier is specified, denormal numbers are flushed to sign-preserving zero. The `flush_to_zero` modifier applies only to the f32 data type.

maxi(x,y)i={xiif xi$\geq$yiyiif xi<yi

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// Create tensor view from a pointer to global memory
%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>
%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>
// Convert tensor views to partition views and load tiles from partition
    views.
%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>
%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>
%c0 = constant <i32: 0> : tile<i32>
%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token
%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token
// IEEE 754-2019's maximum
%4 = maxf %2, %3 propagate_nan : tile<2x4xf32>
// IEEE 754-2019's maximumNumber
%5 = maxf %2, %3 : tile<2x4xf32>
```

```
// flush denormal to positive zero
%6 = maxf %2, %3 flush_to_zero : tile<2x4xf32>
```

See cuda_tile.maxf_0 for the full example listing.

8.7.17 cuda_tile.minf

Element-wise floating-point minimum

```
cuda_tile.minf %lhs %rhs %propagate_nan %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **propagate_nan** (Flag) - When set, `maxf` (or `minf`) returns a `NaN` if either of the two compared elements is `NaN` . 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The minimum of the input tiles. 13.1

The `minf` operation computes the element-wise minimum of two input tiles with floating-point element types.

The `propagate_nan` controls how `minf` will interpret `NaN` . If the `propagate_nan` modifier is set, `minf` returns a canonical `NaN` if either of the compared elements is `NaN` (IEEE 754-2019's minimum). While if the `propagate_nan` modifier is not set, `minf` returns a canonical `NaN` only if both elements are `NaN` ; otherwise, it returns the non- `NaN` element (IEEE

754-2019's minimumNumber).

If neither element is `NaN` , `minf` will return the lowest of the inputs. `-0.0` is considered less than `+0.0` .

If the `flush_to_zero` modifier is specified, denormal numbers are flushed to sign-preserving zero. The `flush_to_zero` modifier applies only to the f32 data type.

minf(x,y)i={xiif xi$\leq$yiyiif xi>yi

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// Create tensor view from a pointer to global memory
%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>
%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>
// Convert tensor views to partition views and load tiles from partition
    views.
%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>
%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>
%c0 = constant <i32: 0> : tile<i32>
%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token
%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token
// IEEE 754-2019's minimum
%4 = minf %2, %3 propagate_nan : tile<2x4xf32>
// IEEE 754-2019's minimumNumber
%5 = minf %2, %3 : tile<2x4xf32>
// flush denormal to positive zero
%6 = minf %2, %3 flush_to_zero : tile<2x4xf32>
```

See cuda_tile.minf_0 for the full example listing.

8.7.18 cuda_tile.mulf

Element-wise floating-point multiplication

```
cuda_tile.mulf %lhs %rhs %rounding_mode %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The product of the input tiles. 13.1

The `mulf` operation computes the element-wise product between the two input tiles with with floating-point element types.

If the `flush_to_zero` modifier is specified, denormal numbers are flushed to positive zero.

If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result.

mulf(x,y)i=xi×yi

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).

- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers and rounding modes for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no
`rounding<nearest_even>`	yes	yes	yes	yes
`rounding<zero>`	yes	yes	yes*	yes*
`rounding<negative_inf>`	yes	yes	yes*	yes*
`rounding<positive_inf>`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.19 cuda_tile.negf

Element-wise floating-point negation

```
cuda_tile.negf %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The negated floating-point tile. 13.1

`negf` is an element-wise operation that negates the sign of `source` .

negf(x)i=-xi

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%source = constant <f32: 0.0> : tile<4xf32>
%result = negf %source : tile<4xf32>
```

See cuda_tile.negf_0 for the full example listing.

8.7.20 cuda_tile.pow

Element-wise floating-point exponentiation

```
cuda_tile.pow %source %exponent
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The base tile. 13.1
- **exponent** (tile < f16 | bf16 | f32 | f64 >) - The exponent tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the pow operation. 13.1

The `pow` operation computes the element-wise exponentiation of the source floating-point tile raised to the power of the exponent floating-point tile.

pow(x,y)i=xiyi

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `result` , `source` and `exponent` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- `source` , `exponent` and `result` must have the same rank.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%source = constant <f32: 0.0> : tile<4xf32>
%exponent = constant <f32: 2.0> : tile<4xf32>
%result = pow %source, %exponent : tile<4xf32>
```

See cuda_tile.pow_0 for the full example listing.

8.7.21 cuda_tile.remf

Element-wise floating-point remainder

```
cuda_tile.remf %lhs %rhs
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The remainder after division. 13.1

The `remf` operation computes the element-wise floating-point remainder using truncated division (rounding towards zero).

remf(x,y)i=xi-trunc(xi/yi)×yi

The result has the same sign as the dividend (`lhs`) and its magnitude is less than the magnitude of divisor (`rhs`).

Special cases:

- If `y` is zero, returns `NaN`
- If `x` is infinite and `y` is finite, returns `NaN`
- If `x` is finite and `y` is infinite, returns `x`
- If either argument is `NaN` , returns `NaN`

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.22 cuda_tile.rsqrt

Element-wise reciprocal square root

```
cuda_tile.rsqrt %source %flush_to_zero
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input tile to compute the reciprocal square root of. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1

- **result** (tile < f16 | bf16 | f32 | f64 >) - The reciprocal square root of the input tile. 13.1

The `rsqrt` operation computes the element-wise reciprocal square root of the input floating-point tile.

This operation supports: `flush_to_zero` : if set by the user, will flush subnormal inputs and results to sign-preserving zero.

rsqrt(x)i=1xi

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

The below table shows the supported modifiers for each data type.

Modifier	Float32	Float64
`flush_to_zero`	yes	no

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = rsqrt %in : tile<4xf32>
// Rsqrt op with flush to zero modifier
%ftz_res = rsqrt %in flush_to_zero : tile<4xf32>
```

See cuda_tile.rsqrt_0 for the full example listing.

8.7.23 cuda_tile.sinh

Element-wise hyperbolic sine

```
cuda_tile.sinh %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input float tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The hyperbolic sine of the input tile. 13.1

The `sinh` operation computes the element-wise hyperbolic sine of the input floating-point tile.

sinh(x)i=sinh(xi)

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.24 cuda_tile.sin

Element-wise sine

```
cuda_tile.sin %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input float tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The sine of the input tile. 13.1

The `sin` operation computes the element-wise sine of the input floating-point tile.
sin(x)i=sin(xi)
This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = sin %in : tile<4xf32>
```

See cuda_tile.sin_0 for the full example listing.

8.7.25 cuda_tile.sqrt

Element-wise square root

```
cuda_tile.sqrt %source %rounding_mode %flush_to_zero
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input tile to compute the square root of. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1

- **result** (tile < f16 | bf16 | f32 | f64 >) - The square root of the input tile. 13.1

The `sqrt` operation computes the element-wise square root of a floating-point tile.
sqrt(x)i=xi
The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers and rounding modes for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no
`approx`	yes	no	no	no
`rounding<nearest_even>`	yes	yes	yes	yes
`rounding<zero>`	yes	yes	yes*	yes*
`rounding<negative_inf>`	yes	yes	yes*	yes*
`rounding<positive_inf>`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.26 cuda_tile.subf

Element-wise floating-point subtraction

```
cuda_tile.subf %lhs %rhs %rounding_mode %flush_to_zero
```

Parameters

- **lhs** (tile < f16 | bf16 | f32 | f64 >) - The left hand side operand. 13.1
- **rhs** (tile < f16 | bf16 | f32 | f64 >) - The right hand side operand. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.1
- **flush_to_zero** (Flag) - If set, flushes subnormal inputs and results to sign-preserving zero. 13.1

- **result** (tile < f16 | bf16 | f32 | f64 >) - The result of the subtraction. 13.1

The `subf` operation computes the element-wise subtraction of the input floating-point tiles.

subf(x,y)i=xi-yi

Element-wise floating-point arithmetic operations are performed by the target architecture's native floating-point instructions. If the `rounding` modifier is specified, the particular rounding mode will be applied to each element of the result. See Floating Point for more details.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers and rounding modes for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`flush_to_zero`	yes	no	no	no
`rounding<nearest_even>`	yes	yes	yes	yes
`rounding<zero>`	yes	yes	yes*	yes*
`rounding<negative_inf>`	yes	yes	yes*	yes*
`rounding<positive_inf>`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.7.27 cuda_tile.tanh

Element-wise hyperbolic tangent

```
cuda_tile.tanh %source %rounding_mode
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input floating-point tile. 13.1
- **rounding_mode** (RoundingMode) - The rounding mode for the operation. 13.2

- **result** (tile < f16 | bf16 | f32 | f64 >) - The hyperbolic tangent of the input floating-point tile. 13.1

The `tanh` operation computes the element-wise hyperbolic tangent of the input floating-point tile. Default rounding mode is full .

The `approx` rounding mode implements a fast approximation to hyperbolic tangent.

Subnormal results of this fast approximation are not flushed to zero.

The `full` rounding mode implements a relatively fast full-range approximation.

The maximum ulp error is 2 across the full range of inputs in FP32 and 1 in FP64.

tanh(x)i=tanh(xi)

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

The below table shows the supported modifiers for each data type. Entries with '*' are emulated in f32.

Modifier	Float32	Float64	BFloat16	Float16
`approx`	yes	no	no	no
`full`	yes	yes	yes*	yes*

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res0 = tanh %in : tile<4xf32>
// tanh with approx modifier
%res1 = tanh %in rounding<approx> : tile<4xf32>
```

See cuda_tile.tanh_0 for the full example listing.

8.7.28 cuda_tile.tan

Element-wise tangent

```
cuda_tile.tan %source
```

Parameters

- **source** (tile < f16 | bf16 | f32 | f64 >) - The input floating-point tile. 13.1
- **result** (tile < f16 | bf16 | f32 | f64 >) - The tangent of the input floating-point tile. 13.1

The `tan` operation computes the element-wise tangent of the input floating-point tile.

tan(x)i=tan(xi)

This operation is emulated in `:code:f32‘ when executed on half-precision inputs ( :code:f16 and :code:bf16‘ )`. See Floating Point for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < f16 | bf16 | f32 | f64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8 Integer

Tile IR contains a set of typed arithmetic operations which implement familiar arithmetic operations on tiles of integers, for floating-point operations see Floating Point .

All operations are implemented in a manner that is efficient for the target architecture and device family. In most common cases this means utilizing the underlying hardware's native floating-point operations. Due to **Tile IR** 's stability guarantees and higher-level programming model some types on some hardware may be emulated, see Stability for more information about the stability guarantees and information about per device behavior.

8.8.1 Integer Arithmetic

Integer types in **Tile IR** are signless, which is importantly not the same as unsigned. We store all integers in a two's complement representation and with required operations supporting a signed or unsigned flag as needed. This design allows us to not have to differentiate between signed and unsigned integer types at the IR level and keeps sign information local to the operation.

For the `i1` type, unsigned operations see values 0/1, while signed operations see values 0/-1, with all i1 values canonicalized to 0x00 (false) or 0x01 (true)

for consistent LSB-only semantics.

8.8.2 cuda_tile.absi

Element-wise integer absolute value

```
cuda_tile.absi %source
```

Parameters

- **source** (tile < i1 | i8 | i16 | i32 | i64 >) - The input integer tile. 13.1

- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The absolute value of the input tile. 13.1

The `absi` operation computes the absolute value of the input integer tile.

The input tile is always interpreted as a signed integer.

The output tile is always interpreted as an unsigned integer.

absi(x)=|x|

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape.
- `source` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.3 cuda_tile.addi

Element-wise integer addition

```
cuda_tile.addi %lhs %rhs %overflow
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **overflow** (IntegerOverflow) - The overflow behavior of the operation. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The sum of the input tiles. 13.1

The `addi` operation computes the element-wise addition of two tiles with integer element types.

addi(x,y)i=xi+yi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `overflow` attribute is used to instruct the compiler on how to reason about the overflow behavior of the specific operation.

These attributes serve as assumptions that the compiler may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

- `none` - The compiler makes no assumptions regarding overflow behavior.
- `no_signed_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands signed integers.

- `no_unsigned_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands unsigned integers.
- `no_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands as signed or unsigned integers.

If an overflow occurs at runtime despite the value of overflow stating otherwise, the behavior is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.4 cuda_tile.divi

Element-wise integer division

```
cuda_tile.divi %lhs %rhs %signedness %rounding
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **rounding** (RoundingMode) - Set the rounding direction (implementing floordiv/ceildiv). 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The result of the division. 13.1

The `divi` operation computes the element-wise division of two tile values with integer element type.

The default rounding is towards zero. The rounding mode can be set to positive_inf ("ceiling division"), or negative_inf ("floor division"), other values are illegal.

The use of the rounding flag negative_inf with unsigned is not a valid combination.

If the unsigned flag is provided, the operands are treated as unsigned integers, otherwise they are treated as signed integers.

The behavior is undefined if the right hand side is zero. A signed division overflow (minimum value divided by -1) is undefined behavior.

div(lhs, rhs)i=lhsi/rhsi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

The `rounding` attribute specifies the rounding mode to use for the operation.

- `nearest_even` - Round to nearest (ties to even).
- `zero` - Round towards zero (truncate).
- `negative_inf` - Round towards negative infinity.
- `positive_inf` - Round towards positive infinity.
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - Round towards zero to the nearest integer.

- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.5 cuda_tile.maxi

Element-wise integer maximum

```
cuda_tile.maxi %lhs %rhs %signedness
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The result of the maxi operation. 13.1

The `maxi` operation computes the element-wise maximum between two input integer tiles.
maxi(x,y)i={xiif xi$\geq$yiyiif xi<yi
Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.
The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// Create tensor view from a pointer to global memory
%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>
%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>
// Convert tensor views to partition views and load tiles from them.
%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>
%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>
%c0 = constant <i32: 0> : tile<i32>
%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token
%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token
// Signless i32 treated as unsigned
%4 = maxi %2, %3 unsigned : tile<2x4xi32>
// Signless i32 treated as signed
%5 = maxi %2, %3 signed : tile<2x4xi32>
```

See cuda_tile.maxi_0 for the full example listing.

8.8.6 cuda_tile.mini

Element-wise integer minimum

```
cuda_tile.mini %lhs %rhs %signedness
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The minimum of the input tiles. 13.1

The `mini` operation computes the element-wise minimum between the two input tiles with integer element types.

mini(x,y)i={xiif xi$\leq$yiyiif xi>yi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// Create tensor view from a pointer to global memory
%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>
%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>
// Convert tensor views to partition views and load tiles from partition
    views.
%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>
%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>
%c0 = constant <i32: 0> : tile<i32>
%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token
%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
     tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token
// Signless i32 treated as unsigned
%4 = mini %2, %3 unsigned : tile<2x4xi32>
// Signless i32 treated as signed
%5 = mini %2, %3 signed : tile<2x4xi32>
```

See cuda_tile.mini_0 for the full example listing.

8.8.7 cuda_tile.muli

Element-wise integer multiplication

```
cuda_tile.muli %lhs %rhs %overflow
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side input integer tile. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side input integer tile. 13.1
- **overflow** (IntegerOverflow) - The overflow behavior of the operation. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The product of the input tiles. 13.1

The `muli` operation computes the element-wise product between the two input tiles with integer element types.

muli(x,y)i=xi×yi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `overflow` attribute is used to instruct the compiler on how to reason about the overflow behavior of the specific operation.

These attributes serve as assumptions that the compiler may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

- `none` - The compiler makes no assumptions regarding overflow behavior.
- `no_signed_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands signed integers.
- `no_unsigned_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands unsigned integers.
- `no_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands as signed or unsigned integers.

If an overflow occurs at runtime despite the value of overflow stating otherwise, the behavior is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.8 cuda_tile.mulhii

Element-wise high bits of integer multiplication

```
cuda_tile.mulhii %x %y
```

Parameters

- **x** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side input integer tile. 13.1
- **y** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side input integer tile. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The most significant bits of the product of the input tiles. 13.1

The `mulhii` operation produces the most significant N bits of the 2N-bit product of two N-bit integer tiles. For `i64` , this is the most significant 64

bits of the full 128-bit product; for `i8` , it is the most significant 8

bits of the full 16-bit product; etc.

This is in contrast to `muli` , which produces the lower N bits of the 2N-bit product.

The `mulhii` operation is only defined for unsigned integers.

mulhii(xi,yi)=xi×yi»bitwidth(type(xi))

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `x` , `y` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// 2^31 * 2 = 2^32, or 0x100000000.
// The most significant 32 bits of the product are 0x00000001.
// The lower 32 bits of the product are 0x00000000.
%a = constant <i32: 2147483648> : tile<i32>  // %a = 2^31
%b = constant <i32: 2> : tile<i32>           // %b = 2
%res_hi = mulhii %a, %b : tile<i32>          // %res_hi = 1
%res_lo = muli %a, %b : tile<i32>            // %res_lo = 0
```

See cuda_tile.mulhii_0 for the full example listing.

8.8.9 cuda_tile.negi

Element-wise integer negation

```
cuda_tile.negi %source %overflow
```

Parameters

- **source** (tile < i1 | i8 | i16 | i32 | i64 >) - The input integer tile. 13.1
- **overflow** (IntegerOverflow) - The overflow behavior of the operation. 13.2
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The negated integer tile. 13.1

The `negi` operation computes the element-wise negation of the input integer tile.

The input and output tiles are always interpreted as signed integers.

negi(xi)=-xi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `overflow` attribute is used to instruct the compiler on how to reason about the overflow behavior of the specific operation.

These attributes serve as assumptions that the compiler may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

- `none` - The compiler makes no assumptions regarding overflow behavior.
- `no_signed_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands signed integers.
- `no_unsigned_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands unsigned integers.
- `no_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands as signed or unsigned integers.

If an overflow occurs at runtime despite the value of overflow stating otherwise, the behavior is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `source` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%source = constant <i16: [0, 1, 2, 3]> : tile<4xi16>
%result = negi %source : tile<4xi16>
// %result = [0, -1, -2, -3]
```

See cuda_tile.negi_0 for the full example listing.

8.8.10 cuda_tile.ori

Element-wise bitwise OR

```
cuda_tile.ori %lhs %rhs
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The bitwise OR of the input tiles. 13.1

The `ori` operation computes the element-wise bitwise OR of two tiles with integer element types.

ori(x,y)i=xi|yi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.11 cuda_tile.remi

Element-wise integer remainder

```
cuda_tile.remi %lhs %rhs %signedness
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The remainder after division. 13.1

The `remi` operation computes the element-wise remainder of the input tiles with integer element types using truncated division (rounding towards zero).

Division by zero is undefined behavior.

remi(x,y)i=xi-trunc(xi/yi)×yi

If the operation is signed, the sign of the result matches the sign of the dividend (`lhs`). For example:

- `remi(7, 3) = 1`
- `remi(7, -3) = 1`
- `remi(-7, 3) = -1`
- `remi(-7, -3) = -1`

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `result` , `lhs` and `rhs` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.12 cuda_tile.shli

Element-wise shift-left

```
cuda_tile.shli %lhs %rhs %overflow
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand (shift amount). 13.1
- **overflow** (IntegerOverflow) - The overflow behavior of the operation. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The result of the left shift operation. 13.1

The `shli` operation computes the element-wise left shift of the `lhs` integer operand by the `rhs` operand. The lower-order bits on the right are filled with zeros.

shli(x,y)i=xi$\ll$yi

The `rhs` operand is interpreted as an unsigned integer.

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `overflow` attribute is used to instruct the compiler on how to reason about the overflow behavior of the specific operation.

These attributes serve as assumptions that the compiler may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

- `none` - The compiler makes no assumptions regarding overflow behavior.
- `no_signed_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands signed integers.
- `no_unsigned_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands unsigned integers.
- `no_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands as signed or unsigned integers.

If an overflow occurs at runtime despite the value of overflow stating otherwise, the behavior is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.13 cuda_tile.shri

Element-wise shift-right

```
cuda_tile.shri %lhs %rhs %signedness
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand (shift amount). 13.1
- **signedness** (Signedness) - Interpret integer(s) as `signed` or `unsigned` 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The result of the right shift operation. 13.1

The `shri` operation computes the element-wise right shift of the `lhs` integer operand by the value of the `rhs` operand for tiles with integer element types.

shri(x,y)i=xi$\gg$yi

When `unsigned` , higher-order bits are zero-filled; when `signed` , the higher-order bits are filled with the sign bit.

The `rhs` operand is always interpreted as an unsigned integer.

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `signedness` attribute specifies the signedness of operand(s).

- `unsigned` - Treat the operands as unsigned integers.
- `signed` - Treat the operands as signed integers.

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.14 cuda_tile.subi

Element-wise integer subtraction

```
cuda_tile.subi %lhs %rhs %overflow
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **overflow** (IntegerOverflow) - The overflow behavior of the operation. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The result of the subtraction. 13.1

The `subi` operation computes the element-wise subtraction of two input integer tiles.

subi(x,y)i=xi-yi

Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

The `overflow` attribute is used to instruct the compiler on how to reason about the overflow behavior of the specific operation.

These attributes serve as assumptions that the compiler may use to reason about the operation. It is the responsibility of the code generator to ensure that the operation respects these assumptions dynamically during execution.

- `none` - The compiler makes no assumptions regarding overflow behavior.
- `no_signed_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands signed integers.
- `no_unsigned_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands unsigned integers.
- `no_wrap` - The compiler assumes that overflow (wrap-around) will not occur when interpreting the operands as signed or unsigned integers.

If an overflow occurs at runtime despite the value of overflow stating otherwise, the behavior is undefined.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

8.8.15 cuda_tile.xori

Element-wise bitwise XOR

```
cuda_tile.xori %lhs %rhs
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The bitwise XOR of the input tiles. 13.1

The `xori` operation computes the element-wise bitwise exclusive or (XOR)
of two tile values with integer element types.
xori(x,y)i=xi$\oplus$yi
Element-wise integer arithmetic operations are performed by the target architecture's native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%lhs = constant <i32: [0, 1, 2, 3]> : tile<4xi32>
%rhs = constant <i32: [4, 5, 6, 7]> : tile<4xi32>
// This computes the bitwise XOR of each element in `%lhs` and `%rhs`,
    which
// are tiles of shape `4xi32`, and returns the result as `%result`.
%result = xori %lhs, %rhs : tile<4xi32>
```

See cuda_tile.xori_0 for the full example listing.

8.9 Bitwise

8.9.1 cuda_tile.andi

Element-wise bitwise logical AND

```
cuda_tile.andi %lhs %rhs
```

Parameters

- **lhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The left hand side operand. 13.1
- **rhs** (tile < i1 | i8 | i16 | i32 | i64 >) - The right hand side operand. 13.1
- **result** (tile < i1 | i8 | i16 | i32 | i64 >) - The bitwise AND of the input tiles. 13.1

The `andi` operation produces a value that is the result of an element-wise, bitwise “and” of two tiles with integer element type.

andi(x,y)i=xi$\wedge$yi

Element-wise integer arithmetic operations are performed by the target architecture’s native integer instructions. The default semantics are wrap-around semantics on overflow or underflow. See Integer for more details.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.
- `lhs` , `rhs` and `result` must have the same shape and element type (tile < i1 | i8 | i16 | i32 | i64 >).
- The operation’s result type may be inferred from its operands and attributes.

8.10 Atomics

Warning

Atomic operations are limited in **Tile IR** as of early-access and will be updated in coming releases in accordance with the incoming memory model and memory operation updates.

8.10.1 cuda_tile.atomic_cas_tko

Atomic compare-and-swap on global memory

```
cuda_tile.atomic_cas_tko %memory_ordering_semantics %memory_scope %pointers %cmp
    %val %mask %token
```

Parameters

- **memory_ordering_semantics** (MemoryOrderingSemantics) - The memory ordering semantics for the atomic operation. 13.1
- **memory_scope** (MemoryScope) - The memory scope for the atomic operation. 13.1
- **pointers** (ptr) - The pointers to the memory locations to perform the atomic compare-and-swap operation on. 13.1
- **cmp** (tile) - The values to compare against. 13.1
- **val** (tile) - The values to swap in. 13.1
- **mask** (tile < i1 >) - The mask for the atomic operation. 13.1
- **token** (token) - The token for the atomic operation. 13.1
- **result** (tile) - The result of the atomic operation. 13.1
- **result_token** (token) - The result token of the atomic operation. 13.1

The `atomic_cas` operation performs element-wise, atomic compare-and-swaps at the specified global memory `pointers` . The data in memory is compared to `cmp` and the data written to memory is specified by `val` . The operation returns the original value that was stored in memory before the atomic operation was performed.

The shape (and the element type) of `pointers` , `cmp` , `val` and `result` must match. The `atomic_cas` operation performs the following steps for every `(pointer, cmp, val)` tuple in one atomic transaction. (One atomic transaction per tuple.)

```
  atomic() {
    x = *pointer
    if x == cmp {
    *pointer = val
  }
  return x
}
```

An optional parameter, `mask` , allows specifying which elements participate in the atomic operation. A false value at position i masks out the corresponding element in `pointers` , excluding it from the operation. The returned value for a masked element at position i is `cmp[i]` . If no mask is provided, all elements are included in the computation by default. The shape of mask must match that of `pointers` , `cmp` , and `val` .

A token-ordered atomic compare-and-swap is not constrained by program order. The compiler may reorder it (i.e. place them earlier or later in program order) unless constrained by tokens.

Supported data types:

- i32, i64: integer values
- f32, f64: floating-point values

For floating-point types, the comparison uses bitwise equality rather than

IEEE-754 semantics. This means different `NaN` bit patterns are treated as distinct values, and `+0.0` and `-0.0` are considered different if their bit representations differ.

The `memory_ordering_semantics` attribute specifies the concurrency assumption between memory accesses in different threads, which controls the synchronization required.

For example, `weak` ordering allows the compiler to assume that there are no concurrent accesses to any accessed location.

For more information, refer to the memory model section of the specification.

- `relaxed` - There may be concurrent access to the location, but this access does not establish a happens-before relationship.
- `acquire` - There may be concurrent accesses to the location. If this acquire observes a release operation, then *happens before* is established.
- `release` - There may be concurrent access to the location. If this release is observed with an acquire operation, then *happens before* is established.
- `acq_rel` - There may be concurrent accesses to the location. This has the effect of both a release and acquire operation.

Note: The following variants are not supported by this operation: `weak` .

The `memory_scope` attribute specifies a communication scope for memory operations.

When communicating with other concurrent threads in the system, the scope must be broad enough to encompass all other threads which are participating in the communication, or data races may occur.

- `tl_blk` - There may be concurrent accesses from within the same tile block.
- `device` - There may be concurrent accesses from within the same device (i.e., GPU).
- `sys` - There may be concurrent accesses from anywhere within the system (i.e., all devices).

- `cmp` , `val` and `result` must have the same shape and element type (tile).
- The operation must encode variadic operand segment sizes in attributes.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%ptr_1x = reshape %ptr : tile<ptr<i32>> -> tile<1xptr<i32>>
%ptr_vec = broadcast %ptr_1x : tile<1xptr<i32>> -> tile<8xptr<i32>>
%offsets = iota : tile<8xi32>
%ptrs = offset %ptr_vec, %offsets : tile<8xptr<i32>>, tile<8xi32> -> tile<8
    xptr<i32>>
%cmp = constant <i32: [0, 1, 2, 3, 4, 5, 6, 7]> : tile<8xi32>
%val = constant <i32: [7, 6, 5, 4, 3, 2, 1, 0]> : tile<8xi32>
%mask = constant <i1: [0, 1, 0, 1, 0, 1, 0, 1]> : tile<8xi1>
// Atomic CAS without input token.
%0, %token = atomic_cas_tko relaxed device %ptrs, %cmp, %val :
  tile<8xptr<i32>>, tile<8xi32> -> tile<8xi32>, token
// Atomic CAS without input token.
%1, %token1 = atomic_cas_tko relaxed device %ptrs, %cmp, %val, %mask :
  tile<8xptr<i32>>, tile<8xi32>, tile<8xi1> -> tile<8xi32>, token
// Atomic CAS with input token.
%token2 = make_token : token
%2, %token3 = atomic_cas_tko relaxed device %ptrs, %cmp, %val token=%token2
     :
  tile<8xptr<i32>>, tile<8xi32> -> tile<8xi32>, token
return
```

See cuda_tile.atomic_cas_tko_0 for the full example listing.

8.10.2 cuda_tile.atomic_rmw_tko

Atomic read-modify-write on global memory

```
cuda_tile.atomic_rmw_tko %memory_ordering_semantics %memory_scope %pointers %
    mode %arg %mask %token
```

Parameters

- **memory_ordering_semantics** (MemoryOrderingSemantics) - The memory ordering semantics for the load operation. 13.1
- **memory_scope** (MemoryScope) - The memory scope for the atomic operation. 13.1
- **pointers** (ptr) - The pointer tile to perform atomic operation on. 13.1
- **mode** (AtomicRMWMode) - The atomic operation mode (e.g., add, max, min, etc.). 13.1
- **arg** (tile) - The value tile to use in the atomic operation. 13.1
- **mask** (tile $<$ i1 $>$) - The mask for the load operation. 13.1
- **token** (token) - The token for the atomic operation. 13.1
- **result** (tile) - The result of the atomic operation. 13.1
- **result_token** (token) - The result token of the load operation. 13.1

The `atomic_rmw_tko` operation performs element-wise, atomic read-modify-write operations at the global memory locations specified by `pointers` . The values written to memory are determined by `mode` and `arg` . The operation returns the original value stored at each location before the atomic update.

The shapes of `pointers` , `arg` , and `result` must match. The element type of the pointer type must match the element types of both `arg` and `result` . Each (`pointer` , `arg`) pair is processed in a single atomic transaction.

```
atomic {
  x = *pointer
  y = mode(x, arg)
  *pointer = y
  return x
}
```

An optional parameter, `mask` , specifies which elements participate in the atomic operation. A False value at position `i` excludes the corresponding element in `pointers` from the operation.

The value returned for a masked-out element is implementation-defined.

The shape of `mask` must match the shape of `pointers` .

The `atomic_addf` operation is defined to round to the nearest even value.

.. note::

The current implementation of the compiler flushes denormals to zero. This behavior will be fixed in a future version of the compiler and users should not rely on it.

Token-ordered atomic read-modify-write operations are not constrained by program order. The compiler may reorder them (i.e., move them earlier or later in the program) unless further constrained by tokens.

Supported data types by `mode` :

>
>
> - ADD, AND, MAX, MIN, OR, UMAX, UMIN, XOR: i32, i64
>
> - ADDF: f16, f32, f64
>
> - XCHF: i32, i64, f32, f64
>
>

The `U` prefix in UMAX and UMIN distinguishes these from their signed counterparts (MAX and MIN) by interpreting the comparison as unsigned.

The `memory_ordering_semantics` attribute specifies the concurrency assumption between memory accesses in different threads, which controls the synchronization required.

For example, `weak` ordering allows the compiler to assume that there are no concurrent accesses to any accessed location.

For more information, refer to the memory model section of the specification.

- `relaxed` - There may be concurrent access to the location, but this access does not establish a happens-before relationship.
- `acquire` - There may be concurrent accesses to the location. If this acquire observes a release operation, then *happens before* is established.
- `release` - There may be concurrent access to the location. If this release is observed with an acquire operation, then *happens before* is established.
- `acq_rel` - There may be concurrent accesses to the location. This has the effect of both a release and acquire operation.

Note: The following variants are not supported by this operation: `weak` .

The `memory_scope` attribute specifies a communication scope for memory operations.

When communicating with other concurrent threads in the system, the scope must be broad enough to encompass all other threads which are participating in the communication, or data races may occur.

- `tl_blk` - There may be concurrent accesses from within the same tile block.
- `device` - There may be concurrent accesses from within the same device (i.e., GPU).
- `sys` - There may be concurrent accesses from anywhere within the system (i.e., all devices).

The `mode` attribute specifies the mode of the atomic read-modify-write operation.

- `and` - Perform bitwise AND as the modification operation.
- `or` - Perform bitwise OR as the modification operation.
- `xor` - Perform bitwise XOR as the modification operation.
- `add` - Perform integer addition as the modification operation.
- `addf` - Perform floating-point addition as the modification operation.
- `max` - Perform maximum as the modification operation.
- `min` - Perform minimum as the modification operation.
- `umax` - Perform unsigned maximum as the modification operation.
- `umin` - Perform unsigned minimum as the modification operation.
- `xchg` - Perform exchange as the modification operation.

The `mode` attribute has a default value of `add` .

Constraints

- `arg` and `result` must have the same shape and element type (tile).
- The operation must encode variadic operand segment sizes in attributes.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// Reshape the input pointer tile to have a 1d shape
%ptr_1x = reshape %ptr : tile<ptr<f32>> -> tile<1xptr<f32>>
// Broadcast the reshaped tile to a tile with 8 rows, effectively
    replicating the pointer 8 times
%ptr_vec = broadcast %ptr_1x : tile<1xptr<f32>> -> tile<8xptr<f32>>
// Create a tile of offsets [0, 1, 2, ..., 7] to index into memory
%offsets = iota : tile<8xi32>
// Add the offsets to each pointer in the vector to create 8 unique
    pointers
%ptrs = offset %ptr_vec, %offsets : tile<8xptr<f32>>, tile<8xi32> -> tile<8
    xptr<f32>>
%vals = constant <f32: [7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0, 0.0]> : tile<8
    xf32>
// Perform atomic addf operations on the memory locations pointed by %ptrs
// without requiring an input token. Returns the original values and a
    result token
%0, %res_token0 = atomic_rmw_tko relaxed device %ptrs, addf, %vals :
    tile<8xptr<f32>>, tile<8xf32> -> tile<8xf32>, token
// Perform atomic add operations again, this time using the explicit input
    token
%token = make_token : token
%1, %res_token1 = atomic_rmw_tko relaxed device %ptrs, addf, %vals, token =
     %token :
    tile<8xptr<f32>>, tile<8xf32> -> tile<8xf32>, token
```

See cuda_tile.atomic_rmw_tko_0 for the full example listing.

8.11 Views

Views are a structured way to interact with tensors in memory. They are described in both the types section Tensor View and the semantics section Views .

Views are the primary way to interact with global memory in **Tile IR** . A common pattern is to construct a Tensor View from a pointer with cuda_tile.make_tensor_view and then use the cuda_tile.load_view_tko and cuda_tile.store_view_tko operations to read and write to them. For larger tensors, loading the entire tensor is not efficient and therefore we have a sub-view Partition View which allows a user to tile a `tensor_view` .

8.11.1 cuda_tile.get_index_space_shape

Query the index space dimension size

```
cuda_tile.get_index_space_shape %src
```

Parameters

- **src** (view_type) - The source view type. 13.1

- **result** (Variadic < tile < any > >) - The shape of the index space, each value representing the size of the corresponding dimension. 13.1

The `get_index_space_shape` operation returns the shape of the index space of `src` .

The result tile has the same rank as the view's index space with the elements representing the size of the corresponding dimension.

The result values should be interpreted as unsigned integers.

Warning

If the individual index space dimension do not fit in the result tile's element type the behavior is undefined.

Constraints

- The operation is pure and does not perform any memory side effects.

Examples

```
%tensor_view = make_tensor_view %base,
    shape = [2, 2, 4], strides = [2, 2, 1]
    : tensor_view<2x2x4xf32, strides=[2,2,1]>
%partition_view = make_partition_view %tensor_view :
  partition_view<tile=(2x2x4), tensor_view<2x2x4xf32, strides=[2,2,1]>>
%dim0, %dim1, %dim2 = get_index_space_shape %partition_view :
  partition_view<tile=(2x2x4), tensor_view<2x2x4xf32, strides=[2,2,1]>> ->
      tile<i64>
```

See cuda_tile.get_index_space_shape_0 for the full example listing.

8.11.2 cuda_tile.get_tensor_shape

Query the shape of a tensor view

```
cuda_tile.get_tensor_shape %src
```

Parameters

- **src** (tensor_view) - The source tensor view. 13.1

- **result** (Variadic < tile < any > >) - The shape of the tensor, each value representing the size of the corresponding dimension. 13.1

The `get_tensor_shape` operation returns the shape of the tensor backing the provided tensor view.

The result values should be interpreted as unsigned integers.

Warning

If the tensor dimensions do not fit in the result tile's element type the behavior is undefined.

Constraints

- The operation is pure and does not perform any memory side effects.

Examples

```
%dim0, %dim1 = get_tensor_shape %tensor_view : tensor_view<32x32xf32,
    strides=[32,1]> -> tile<i64>
```

See cuda_tile.get_tensor_shape_0 for the full example listing.

8.11.3 cuda_tile.load_view_tko

Load a tile from a tile view

```
cuda_tile.load_view_tko %memory_ordering_semantics %memory_scope %view %index %
    token %optimization_hints
```

Parameters

- **memory_ordering_semantics** (MemoryOrderingSemantics) - The memory ordering semantics for the load operation. 13.1
- **memory_scope** (MemoryScope) - The memory scope for the atomic operation. 13.1
- **view** (view_type) - The view from which the tile will be loaded. 13.1
- **index** (Variadic < tile < any > >) - The n-dimensional index of the desired element to load from the view. 13.1
- **token** (token) - The optional token for the load operation. 13.1
- **optimization_hints** (OptimizationHints) - Optimization hints for operation 13.1

- **tile** (tile) - The loaded tile. 13.1
- **result_token** (token) - The result token. 13.1

The `load_view_tko` operation loads a tile from a tile view.

A view is mapping from view-space indices to a particular element in the view, each view type has a defined mapping from view-space indices to tiles produced from elements of the view.

For example, the Partition View partitions a Tensor View into a grid of equally sized tiles. The view indexes one of the partitioned tiles in the grid.

For a given view the rank of the indices must match the rank of the view's index space. The space of valid indices depends on which view is passed to the operation.

For example the index space of a Partition View is equal to the rank of the partitioned tiles.

The `index` operands are interpreted as unsigned integers.

Out of bounds accesses are handled according to the semantics of Partition View .

The `memory_ordering_semantics` attribute specifies the concurrency assumption between memory accesses in different threads, which controls the synchronization required.

For example, `weak` ordering allows the compiler to assume that there are no concurrent accesses to any accessed location.

For more information, refer to the memory model section of the specification.

- `weak` - No concurrent accesses to the source/destination location.
- `relaxed` - There may be concurrent access to the location, but this access does not establish a happens-before relationship.
- `acquire` - There may be concurrent accesses to the location. If this acquire observes a release operation, then *happens before* is established.

Note: The following variants are not supported by this operation: `release` , `acq_rel` .

The `memory_scope` attribute specifies a communication scope for memory operations.

When communicating with other concurrent threads in the system, the scope must be broad enough to encompass all other threads which are participating in the communication, or data races may occur.

- `tl_blk` - There may be concurrent accesses from within the same tile block.
- `device` - There may be concurrent accesses from within the same device (i.e., GPU).
- `sys` - There may be concurrent accesses from anywhere within the system (i.e., all devices).

The `optimization_hints` attribute provides architecture-specific compiler hints in the form of nested dictionaries.

The hints are specified for each architecture (e.g., `sm_100` , `sm_120`) and for each architecture the user can specify specific hints for each operation.

- `num_cta_in_cga` - suggest the number of CTAs in a CGA (which must be the power of 2 less than or equal to 16) for cuda_tile.entry .
- `allow_tma` - suggest whether to use TMA for cuda_tile.load_view_tko and cuda_tile.store_view_tko .
- `latency` - latency hint for cuda_tile.load_view_tko and cuda_tile.store_view_tko .

For example they can be annotated as:

```
optimization_hints=<
  sm_100 = {num_cta_in_cga = 8},
  sm_120 = {num_cta_in_cga = 16}
>
```

Constraints

- The operation must encode variadic operand segment sizes in attributes.

Examples

```
%tensor_view = make_tensor_view %ptr, shape=[8192, 128], strides=[128, 1]
  : tensor_view<8192x128xf32, strides=[128,1]>
// This example uses the PartitionView on a 8192x128xf32 tensor_view,
// dividing the tensor_view in tiles of 64x64.
%view = make_partition_view %tensor_view : partition_view<tile=(64x64),
    tensor_view<8192x128xf32, strides=[128,1]>>
%c0 = constant <i32: 0> : tile<i32>
%c1 = constant <i32: 1> : tile<i32>
// Load a tile at index (0, 0) in the view's index space.
// For this PartitionView, this is the rectangular tile such that
```

```
// X=[0,64) and Y=[0,64), in the coordinates of tiles.
%tile0, %res_token0 = load_view_tko weak %view[%c0, %c0]
  : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token
// Load a tile at index (0, 1) in the view's index space.
// For this PartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[64,128), in the coordinates of tiles.
%tile1, %res_token1 = load_view_tko weak %view[%c0, %c1]
  : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token
// Same example as above but with memory token as input.
%token = make_token : token
%tile2, %res_token2 = load_view_tko weak %view[%c0, %c1] token = %token
  : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token
// Loads a tile at the dynamic index (%index, %index) in the view's index
    space.
%tile3, %res_token3 = load_view_tko weak %view[%index, %index]
  : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token
```

See cuda_tile.load_view_tko_0 for the full example listing.

8.11.4 cuda_tile.make_partition_view

Create a partition view from a tensor view

```
cuda_tile.make_partition_view %tensor_view
```

Parameters

- **tensor_view** (tensor_view) - The source tensor view to create a partition view from. 13.1
- **result** (partition_view) - The created partition view. 13.1

The `make_partition_view` operation creates a partition_view from a tensor_view . For more details about partition views see Partition View .

The operation uses the type constraints of the input tensor view and the annotated return type to perform the partitioning. The tensor view's type contains its physical layout in the form of shapes and strides and the partition view contains the logical size of a single tile.

The resulting partition view can be loaded from using cuda_tile.load_view_tko and stored to using cuda_tile.store_view_tko .

The view memory options act on the computed index space of the partition view see Tensor View and Partition View for detailed semantics.

Constraints

- The operation is conditionally speculatablebased on the specific operands and attributes.
- The operation may be speculatively executed without side effects.
- The operation is pure and does not perform any memory side effects.

Examples

```
%tensor_view0 = make_tensor_view %ptr, shape=[8192, 8192, 64], strides
    =[524288,64,1]
  : tensor_view<8192x8192x64xf32, strides=[524288,64,1]>
// Creates a partition with 32-bit-indexed tiles of size (1024x1x32) over
// the provided tensor_view.
make_partition_view %tensor_view0 :
  partition_view<
    tile=(1024x1x32),
    tensor_view<8192x8192x64xf32, strides=[524288,64,1]>
  >
%s0 = constant <i32: 8192> : tile<i32>
%str0 = constant <i32: 524288> : tile<i32>
// These seems very wrong.
%tensor_view1 = make_tensor_view %ptr, shape=[%s0, 8192, 64], strides=[%
    str0, 64, 1]
  : tile<i32> -> tensor_view<?x8192x64xf32, strides=[?,64,1]>
// Creates a partition with 32-bit-indexed tiles of size (1024x1x32) over
// the provided tensor_view, with masking. The provided tensor_view has a
// dynamically-sized dimension.
make_partition_view %tensor_view1 :
  partition_view<tile=(1024x1x32), tensor_view<?x8192x64xf32, strides
      =[?,64,1]>>
```

See cuda_tile.make_partition_view_0 for the full example listing.

8.11.5 cuda_tile.make_tensor_view

Create :code:___PLACEHOLDER_0___ from a pointer to global memory

```
cuda_tile.make_tensor_view %base %dynamicShape %dynamicStrides
```

Parameters

- **base** (tile < ptr >) - The scalar base pointer to a portion of global memory. 13.1
- **dynamicShape** (Variadic < tile < any > >) - The array of values representing the shape of the view, may be fully dynamic. 13.1
- **dynamicStrides** (Variadic < tile < any > >) - The array of values representing the strides of the view, may be fully dynamic. 13.1
- **result** (tensor_view) - The constructed tensor_view. 13.1

The `make_tensor_view` operation constructs a `tensor_view` from a global memory pointer, a dynamic shape and dynamic strides. See Tensor View for more details.

The constructor supports taking dynamic arrays for shapes and strides as part of the constructor enabling workloads to take global memory tensors of dynamic shape and strides. If these arguments are static they will be statically reflected in the type of the resulting `tensor_view` , if they are dynamic they will appear as `?` in the type. See below for concrete examples.

The `dynamicShape` and `dynamicStrides` operands are interpreted as unsigned integers.

Constraints

- The operation must encode variadic operand segment sizes in attributes.
- The operation is pure and does not perform any memory side effects.

Examples

```
  // tensor_view to a scalar tile of f32
  %a0 = make_tensor_view %base,
      shape = [], strides = [] : tensor_view<f32>
  // tensor_view to a tile of static shape and strides
  %a1 = make_tensor_view %base,
      shape = [32, 32], strides = [32, 1]
      : tensor_view<32x32xf32, strides=[32,1]>
%sh0 = constant <i32: 32> : tile<i32>
%sh1 = constant <i32: 32> : tile<i32>
%st0 = constant <i32: 32> : tile<i32>
%st1 = constant <i32: 1> : tile<i32>
  // tensor_view to a tile with partially dynamic shape and strides
  // all dynamic values must be of the same type, here tile<i32>
  %a2 = make_tensor_view %base,
          shape = [%sh0, %sh1], strides = [%st0, %st1]
          : tile<i32> -> tensor_view<?x?xf32, strides=[?,?]>
```

See cuda_tile.make_tensor_view_0 for the full example listing.

8.11.6 cuda_tile.store_view_tko

Stores a tile into a tile view

```
cuda_tile.store_view_tko %memory_ordering_semantics %memory_scope %tile %view %
    index %token %optimization_hints
```

Parameters

- **memory_ordering_semantics** (MemoryOrderingSemantics) - The memory scope for the store operation. 13.1
- **memory_scope** (MemoryScope) - The memory scope for the store operation. 13.1
- **tile** (tile) - The tile to store. 13.1
- **view** (view_type) - The view to store the tile to. 13.1
- **index** (Variadic < tile < any > >) - The indices of the desired target tile within the view. 13.1
- **token** (token) - The optional token for operation ordering. 13.1
- **optimization_hints** (OptimizationHints) - Optimization hints for operation 13.1

- **result_token** (token) - The result token for synchronization. 13.1

The `store_view_tko` operation stores a tile to a view indexing into a tile view.

A view is mapping from view-space indices to a particular element in the view, each view type has a defined mapping from view-space indices to tiles produced from elements of the view.

For example, the Partition View partitions a Tensor View into a grid of equally sized tiles. The view indexes one of the partitioned tiles in the grid.

For a given view the rank of the indices must match the rank of the view's index space. The space of valid indices depends on which view is passed to the operation.

For example the index space of a Partition View is equal to the rank of the partitioned tiles.

The index space of the view is computed a function of the requested tile size and the shape of the view.

The `index` operands are interpreted as unsigned integers.

Out of bounds accesses are handled according to the semantics of Partition View .

The `memory_ordering_semantics` attribute specifies the concurrency assumption between memory accesses in different threads, which controls the synchronization required.

For example, `weak` ordering allows the compiler to assume that there are no concurrent accesses to any accessed location.

For more information, refer to the memory model section of the specification.

- `weak` - No concurrent accesses to the source/destination location.
- `relaxed` - There may be concurrent access to the location, but this access does not establish a happens-before relationship.
- `release` - There may be concurrent access to the location. If this release is observed with an acquire operation, then *happens before* is established.

Note: The following variants are not supported by this operation: `acquire` , `acq_rel` .

The `memory_scope` attribute specifies a communication scope for memory operations.

When communicating with other concurrent threads in the system, the scope must be broad enough to encompass all other threads which are participating in the communication, or data races may occur.

- `tl_blk` - There may be concurrent accesses from within the same tile block.
- `device` - There may be concurrent accesses from within the same device (i.e., GPU).
- `sys` - There may be concurrent accesses from anywhere within the system (i.e., all devices).

The `optimization_hints` attribute provides architecture-specific compiler hints in the form of nested dictionaries.

The hints are specified for each architecture (e.g., `sm_100` , `sm_120`) and for each architecture the user can specify specific hints for each operation.

- `num_cta_in_cga` - suggest the number of CTAs in a CGA (which must be the power of 2 less than or equal to 16) for cuda_tile.entry .
- `allow_tma` - suggest whether to use TMA for cuda_tile.load_view_tko and cuda_tile.store_view_tko .
- `latency` - latency hint for cuda_tile.load_view_tko and cuda_tile.store_view_tko .

For example they can be annotated as:

```
optimization_hints=<
  sm_100 = {num_cta_in_cga = 8},
  sm_120 = {num_cta_in_cga = 16}
>
```

Constraints

- The operation must encode variadic operand segment sizes in attributes.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
%tensor_view = make_tensor_view %ptr, shape=[8192, 128], strides=[128,1] :
  tensor_view<8192x128xf32, strides=[128,1]>
// This example uses the PartitionView on a 8192x128xf32 tensor_view,
// dividing the tensor_view in tiles of 64x64.
%view = make_partition_view %tensor_view :
  partition_view<tile=(64x64), tensor_view<8192x128xf32, strides=[128,1]>>
%c0 = constant <i32: 0> : tile<i32>
%c1 = constant <i32: 1> : tile<i32>
%tile = constant <f32: 0.0> : tile<64x64xf32>
// Store a tile at index (0, 0) in the view's index space.
```

```
// For this TilePartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[0,64), in the coordinates of tiles.
%res_token0 = store_view_tko weak %tile, %view[%c0, %c0]
  : tile<64x64xf32>, partition_view<tile=(64x64), tensor_view<8192x128xf32,
      strides=[128,1]>>, tile<i32> -> token
// Store a tile at index (0, 1) in the view's index space.
// For this PartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[64,128), in the coordinates of tiles.
%res_token1 = store_view_tko weak %tile, %view[%c0, %c1]
  : tile<64x64xf32>, partition_view<tile=(64x64), tensor_view<8192x128xf32,
      strides=[128,1]>>, tile<i32> -> token
// Same example as above but with input token.
%token = make_token : token
%res_token2 = store_view_tko weak %tile, %view[%c0, %c1] token = %token
  : tile<64x64xf32>, partition_view<tile=(64x64), tensor_view<8192x128xf32,
      strides=[128,1]>>, tile<i32> -> token
```

See cuda_tile.store_view_tko_0 for the full example listing.

8.12 Miscellaneous

The set of miscellaneous operations in **Tile IR** are operations which do not have a specific category.

8.12.1 cuda_tile.assume

Attach static information to an SSA value

```
cuda_tile.assume %value %predicate
```

Parameters

- **value** (Any) - The value to attach the predicate to. 13.1
- **predicate** (AssumePredicate) - The predicate to attach to the value. 13.1
- **result** (Any) - The value with the attached predicate. 13.1

The `assume` operation passes through `value` as the result and attaches a predicate to it. The assumed predicate is a property of `result` .

This operation can be used to inject static information into the compiler, potentially resulting in more efficient code generation.

`predicate` must implement the `AssumePredicateAttrInterface` .

Note

`assume` does not check the correctness of the predicate.

Incorrect predicates may inject incorrect static information and cause miscompilation. If an incorrect predicate is attached to an SSA value, the behavior of the program is undefined.

AssumePredicate Implementers

Bounded

```
#bounded<(lb|?), (ub|?)>
```

The `bounded` attribute must be used as a predicate for `cuda_tile.assume` . The predicated value must be a tile of integers.

`bounded` specifies a lower and upper bound for all elements of the predicated tile when interpreted as signed integers. Bounds are optional:

it is possible to leave a bound unspecified, as indicated by "?" in the assembly format. E.g., `#bounded<0, ?>` . Both lower bound and upper bound are inclusive.

The lower bounds must be less than or equal to the upper bound. A lower/
upper bound that exceeds the range of valid values of the predicated value is invalid.

```
%1 = cuda_tile.assume #cuda_tile.bounded<0, ?>, %0
    : !cuda_tile.tile<4x8xi16>
```

See cuda_tile.assume_example_Bounded_0 for the full example listing.

DivBy

```
div_by< $divisor (, every $every^ along $along)?>
```

The `div_by` attribute must be used as a predicate for `cuda_tile.assume` ops. The predicated value must be a `tile` of integers or pointers, or a `tensor_view` .

If the predicated value is a `tile` , the attribute indicates that some elements of the `tile` are divisible by `divisor` . If the predicated value is a `tensor_view` the attribute indicates that the base address of the `tensor_view` is divisible by `divisor` . `divisor` must be a positive power of `2` .

The `every` and `along` attributes control which elements are assumed to satisfy the divisibility property. When splitting the tensor in groups of size `every` along dimension `along` , the first element of each group is assumed to satisfy the divisibility property. The other elements are assumed to be monotonically increasing by `1` within the group. In case of a `tile` of pointers, the elements are assumed to be monotonically increasing by the byte width of the pointee type. The size of the last group may be smaller than `every` .

The `every` and `along` attributes are optional. When missing, they are assumed to have a default value of `1` and `0` in case of a `tile` .

I.e., all elements of the `tile` are assumed to satisfy the divisibility property. (The value of `along` does not matter in that case.) If the predicated value is a `tensor_view` or a 0D `tile` , `every` and `along` cannot be used.

`every` , and `along` must be used together. If one is specified, so must be the other.

Note

If the predicated value is a tile of integers, `every` is a property of the signed interpretation of the integer values. Otherwise, it is a property of the unsigned integer interpretation. E.g., `every = 4` is incorrect for the following sequence of "i8" values (written in binary form) because they wrap around when interpreted as signed integers: `[01111110, 01111111, 10000000, 10000001]` . `every = 2` would be correct.

The examples below demonstrate tensors that satisfy the assumed properties.

```
// Example 1: Each pointer is divisible by 16.
// [ 0x10, 0x20, 0x80, 0x10, 0x0, 0x120, ... ]
%0 = cuda_tile.assume #cuda_tile.div_by<16>, %ptrs
    : !cuda_tile.tile<128x!cuda_tile.ptr<f32>>
// Note: Equivalent to #cuda_tile.div_by<16, every 1 along 0>.
```

See cuda_tile.assume_example_DivBy_0 for the full example listing.

```
// Example 2: Each integer is divisible by 4.
// [ 16, 24, 8, 4, 12, 12, 0, 16, ... ]
%0 = cuda_tile.assume #cuda_tile.div_by<4>, %t
    : !cuda_tile.tile<128xi32>
```

See cuda_tile.assume_example_DivBy_1 for the full example listing.

```
// Example 3: Group size [4].
// [7, 8, 9, 10, 23, 24, 25, 26, 0, 1, 2, 3, ...]
%0 = cuda_tile.assume #cuda_tile.div_by<1, every 4 along 0>, %t
    : !cuda_tile.tile<128xi32>
```

See cuda_tile.assume_example_DivBy_2 for the full example listing.

```
// Example 4: 2-d Group size [1, 4] with divisibility 4.
// [ [  4,  5,  6,  7, 12, 13, 14, 15 ],
//   [  8,  9, 10, 11, 24, 25, 26, 27 ],
```

```
//   [ 24, 25, 26, 27, 64, 65, 66, 67 ],
//   [  0,  1,  2,  3,  4,  5,  6,  7 ] ]
%0 = cuda_tile.assume #cuda_tile.div_by<4, every 4 along 1>, %t
    : !cuda_tile.tile<4x8xi32>
```

See cuda_tile.assume_example_DivBy_3 for the full example listing.

```
// Example 5: 2-d Group size [4, 1] with divisibility 32.
// Note that the elements within each column are monotonically increasing
// by the byte width of the pointee type f32, e.g., 0x20, 0x24, 0x28, 0x2c.
// [ [  0x20, 0x100,  0x40,  0x60,  0x40, 0x200, 0x340,  0x40 ],
//   [  0x24, 0x104,  0x44,  0x64,  0x44, 0x204, 0x344,  0x44 ],
//   [  0x28, 0x108,  0x48,  0x68,  0x48, 0x208, 0x348,  0x48 ],
//   [  0x2c, 0x10c,  0x4c,  0x6c,  0x4c, 0x20c, 0x34c,  0x4c ] ]
%0 = cuda_tile.assume #cuda_tile.div_by<32, every 4 along 0>, %ptrs
    : !cuda_tile.tile<4x8x!cuda_tile.ptr<f32>>
```

See cuda_tile.assume_example_DivBy_4 for the full example listing.

SameElements

```
#same_elements< $values >
```

The `same_elements` attribute must be used as a predicate for `cuda_tile.assume` . The predicated value must be a tensor of integers or pointers.

`same_elements` is specified for each dimension. A value of C for a dimension of size N indicates that, after dividing the respective dimension into N/C groups of size C, each group consists of the same elements. As N/C may not divide evenly, the last group may have fewer than C elements.

If the "same elements" property does not hold along a dimension, the respective value should be set to 1. `#cuda_tile.same_elements<[1, 1, ..., 1]>` is a correct predicate for any tensor of integers or pointers, where the number of ones matches the rank of the tensor. (Size-1 groups always have the same elements.)

```
// Integer tensor with same elements.
%0 = cuda_tile.constant <i16: [[0, 0, 0, 0, 10, 10, 10, 10],
                               [0, 0, 0, 0, 10, 10, 10, 10],
                               [5, 5, 5, 5, 93, 93, 93, 93],
                               [5, 5, 5, 5, 93, 93, 93, 93]]>
    : tile<4x8xi16>
%1 = cuda_tile.assume #cuda_tile.same_elements<[2, 4]>, %0
    : !cuda_tile.tile<4x8xi16>
// Pointer tensor with same elements.
%2 = cuda_tile.constant <i64: [[ 0,  0,  0,  0,  8,  8,  8,  8],
                               [ 0,  0,  0,  0,  8,  8,  8,  8],
                               [64, 64, 64, 64, 32, 32, 32, 32],
                               [64, 64, 64, 64, 32, 32, 32, 32]]>
    : tile<4x8xi64>
%3 = cuda_tile.bitcast %2
    : !cuda_tile.tile<4x8xi64>
      -> !cuda_tile.tile<!cuda_tile.ptr<f32>>
%4 = cuda_tile.assume #cuda_tile.same_elements<[2, 4]>, %3
    : !cuda_tile.tile<!cuda_tile.ptr<f32>>
```

See cuda_tile.assume_example_SameElements_0 for the full example listing.

Constraints

- `value` and `result` must have the same shape and element type (Any).
- The operation's result type may be inferred from its operands and attributes.

Examples

```
// Assume that all integers are divisible by 32.
%int_tile = constant <i16: [32, 64, 0, 0, 32, -32, 1024, 0]> : tile<8xi16>
%div_by_1 = assume div_by<32>, %int_tile : tile<8xi16>
```

```
// Assume that every 4th element (starting with element 0) along
// dimension 0 is divisible by 32 that and all integers are
// montonically increasing by 1 within each group of 4.
%int_tile_2 = constant <i16: [96, 97, 98, 99, 64, 65, 66, 67]> : tile<8xi16
    >
%div_by_2 = assume div_by<32, every 4 along 0>, %int_tile_2 : tile<8xi16>
// Assume that every rectangular chunk of size [1, 4, 2] has the same
// values.
%same_elem = assume same_elements<[1, 4, 2]>, %ptr_3d : tile<1x8x8xptr<f32
    >>
// Assume that every value is greater or equal to 5.
%int_tile_3 = constant <i16: [5, 9, 10, 11, 6, 5, 5, 7]> : tile<8xi16>
%bounded = assume bounded<5, ?>, %int_tile_3 : tile<8xi16>
```

See cuda_tile.assume_0 for the full example listing.

8.12.2 cuda_tile.print_tko

Print a formatted string (token-ordered)

```
cuda_tile.print_tko %str %args %token
```

Parameters

- **str** (String) - The format string. 13.1
- **args** (Variadic < tile >) - The arguments to format and print. 13.1
- **token** (token) - The optional token for operation ordering. 13.2
- **result_token** (token) - The result token for synchronization. 13.2

The `print_tko` operation prints a C-printf-style format string, interleaved with the given operands. The number of format expressions

(starting with the `%` character) must match the number of operands.

If a format expression is not applicable to its respective operand, then the output is undefined.

Token-ordered print operations are not constrained by program order. The compiler may reorder them (i.e., move them earlier or later in the program)

unless further constrained by tokens.

This operation is meant for debugging. Its implementation is not optimized for performance, so it should not be used in production mode. Prints are not guaranteed to be atomic. I.e., the output of prints that execute simultaneously may be interleaved.

Note

This op was renamed from `print` to `print_tko` in 13.2. The op code did not change.

Constraints

- The operation must encode variadic operand segment sizes in attributes.
- The operation's result type may be inferred from its operands and attributes.

Examples

```
print_tko "Hello world: %f\n", %arg : tile<4xf32> -> token
print_tko "%+08.3f", %arg : tile<4xf32> -> token
```

See cuda_tile.print_tko_0 for the full example listing.

9 Debug Info

Debug info is essential for understanding and diagnosing the behavior of a **Tile IR** program. Debug info is metadata that maps generated code back to the original source code, creating a more informative and intuitive debugging experience for the **Tile IR** user and allowing the user to quickly identify and fix issues in a **Tile IR** program. This section explains how to include and use debug info in a **Tile IR** program when using the MLIR dialect. For a description of how to produce this information in the bytecode directly, see the bytecode section .

Tile IR supports control flow based debugging of unoptimized code with features such as breakpoints, stepping through code and inspecting stack frames. **Tile IR** debug info exposes scope metadata describing the files, functions and blocks that comprise a **Tile IR** program along with information on how that program was compiled. This scope metadata is included in the **Tile IR** program by fusing it to location information. Details and examples below. **Tile IR** does not support value inspection of user variables, only control flow based debugging as described above is supported at this time.

9.1 Location Information

Tile IR requires that scope metadata is attached to all operations within a **Tile IR** module to generate debug info of any kind. Scope metadata is required in addition to simple file, line, column location information. Providing simple file, line, column location information alone without scope metadata results in a very poor user experience. Scoped metadata is required to generate DWARF information and DWARF information is required by all developer tools including debuggers and profilers.

Example of a simple file, line, column location: `loc("/tmp/foo.py":1:4)`

Example of a location fused with scope metadata: `#cuda_tile.di_loc<loc("/tmp/foo.py":1:4) in #subprogram`

9.1.1 Location Types

Two location types are supported by **Tile IR** :

1. A **Tile IR** location represented by a `#cuda_tile.di_loc` attribute which combines file, line, column information with scope metadata as shown in the example above. This is the preferred method for generating location information fused with scope metadata.

2. A `CallSiteLoc` where both the callee and caller are supported location types. This respresents a function call e.g. in the case where the **Tile IR** producer supports inlining.

Any location containing a supported location such as a `FusedLoc` , `NamedLoc` or `OpaqueLoc` will be converted to that supported location. If no supported location is found, the location will be converted to `UnknownLoc` indicating that the location is unknown or not available. All **Tile IR** global variables will have `UnknownLoc` given that `#cuda_tile.di_loc` supports local scope only and there is no support for `FileLineColLoc` .

Location Type	CudaTile Bytecode Support
CudaTile DILocAttr	Supported
CallSiteLoc	Supported if BOTH callee and caller are supported, else UnknownLoc
FusedLoc	Converted to first supported sub location, else UnknownLoc
NameLoc	Converted to child location if supported, else UnknownLoc
OpaqueLoc	Converted to fallback location if supported, else UnknownLoc
FileLineColLoc	Unsupported; always treated as UnknownLoc

9.1.2 Generating Scope Metadata

Below are the options, trade-offs and mechanisms for generating scope metadata in **Tile IR** programs.

9.1.3 Options

Tile IR producers have two options for generating location information fused with scope metadata:

1. The **Tile IR** producer can generate location information fused with scope metadata directly using `#cuda_tile.di_loc` as shown in the example above.

2. The **Tile IR** producer can generate a `loc` with file, line, column location information and then use a **Tile IR** pass (via `--synthesize-debug-info-scopes`) to synthesize scope metadata from that file, line, column location information.

Note

File, line, column location information must be converted to a **Tile IR** location prior to bytecode generation as all `FileLineColLoc` attributes are encoded as `UnknownLoc` attributes in the bytecode.

9.1.4 Trade-offs

When deciding between the two options above, the **Tile IR** producer should consider the following trade-offs:

1. Providing location information with scope metadata may be harder for the **Tile IR** producer to implement, however, the **Tile IR** producer retains full control to describe its source code to the compiler which may result in a smoother debugging experience for the end user.
2. Providing simple file line, location information is very simple for the **Tile IR** producer to implement but cedes control to the pass which may not be able to accurately synthesize debug info for all cases possibly resulting in a degraded debugging experience for the end user. E.g. the pass may be able to synthesize function name, guess at linkage name, but not do a very good job of getting the line or column number of the function.

9.1.5 Mechanism

To synthesize scope metadata, the SynthesizeDebugInfoScopes pass can be added to any **Tile IR** pass pipeline or invoked via Python as follows:

```
pm = passmanager.PassManager(context=module.context)
full_pass_string = f"cuda_tile.module({"synthesize-debug-info-scopes"})"
pm = pm.parse(full_pass_string, context=module.context)
pm.run(module.operation.regions[0].blocks[0].operations[0])
```

9.2 Scope Metadata

Tile IR debug info is comprised of the following scope metadata expressed as attributes and fields:

9.2.1 Compile Unit

A compile unit represents the root scope of all objects declared within a specific compilation unit. It specifies the source file associated with the compilation unit and encompasses all the other debug information elements such as files, lexical blocks, and subprograms. This scope is fundamental for organizing and correlating debug information with the original source code, facilitating efficient debugging.

Fields:

- **File** : The source file associated with the compilation unit.

The following fields are set by the **Tile IR** compiler and listed for reference:

- **Is Optimized?** : Set to `false` if compiler option `--opt-level` is `0` else `true` .
- **Emission Kind** : Set based on compiler options: `--line-info` to generate line tables only or `--device-debug` alias `-g` for generating full debug info.

9.2.2 File

A file represents a source file used in the compilation process. It specifies the file name and directory of the source file. This information is used to map the generated code back to the original source, allowing developers to trace and debug code effectively.

Fields:

- **Name** : The name of the file.
- **Directory** : The directory containing the file.

9.2.3 Lexical Block

A lexical block represents a nested scope within a subprogram, specifying the scope, file, line number, and optional column number of the block. Lexical blocks are used to represent various nested scopes in the source code, such as conditional statements, loops, and other code blocks. This detailed scope information helps in understanding the program's structure and control flow during debugging.

Fields:

- **Scope** : The scope in which the lexical block is defined.
- **File** : The source file containing the lexical block.
- **Line** : The line number where the lexical block starts.
- **Column** : The column number where the lexical block starts.

9.2.4 Subprogram

A subprogram represents a function within the source language. It specifies the scope, file, line number, name, and linkage name of the subprogram. Optionally, it can include the line number within the scope. Subprogram information is used for function level debugging, allowing developers to set breakpoints, step through code, and inspect stack frames within specific functions.

Fields:

- **File** : The source file containing the subprogram.
- **Line** : The line number where the subprogram starts.
- **Name** : The name of the subprogram.
- **Linkage Name** : The linkage name of the subprogram.
- **Compile Unit** : The compilation unit containing the subprogram.
- **Scope Line** : The line number where the scope of the subprogram starts. [Optional]

The following fields are set by the **Tile IR** compiler and listed for reference:

- **Subroutine Type** : Set to `() -> ()` meaning that all functions appear to have no arguments and no return value.

9.3 Example Usage

Below is an example showing how to use **Tile IR** debug info:

```
#file= #cuda_tile.di_file<"foo.py" in "/tmp/">
#compile_unit= #cuda_tile.di_compile_unit<
  file = #file
>
#subprogram= #cuda_tile.di_subprogram<
  file = #file,
  line = 1,
  name = "test_kernel",
  linkageName = "kernel",
  compileUnit = #compile_unit,
  scopeLine = 1
>
#block= #cuda_tile.di_lexical_block<
  scope = #subprogram,
  file = #file,
  line = 1,
  column = 4
>
#loc_fn= #cuda_tile.di_loc<loc("/tmp/foo.py":1:4) in #subprogram>
#loc_return= #cuda_tile.di_loc<loc("/tmp/foo.py":5:4) in #block>
```

```
cuda_tile.module @kernels attributes { } {
  entry @kernel() {
    return loc(#loc_return)
  } loc(#loc_fn)
}
```